

Dispensa di Algorithm Engineering

Basata su *Pearls of Algorithm Engineering*
di Paolo Ferragina (Cambridge University Press, 2023)

Università di Pisa
Corso di Laurea Magistrale in Informatica
A.A. 2025–26

Nota: questa dispensa integra le lezioni, gli appunti e il materiale didattico del corso.
Non sostituisce lo studio diretto del libro di testo.

Contents

1	Introduction	1
1.1	What is an Algorithm	1
1.2	The RAM Model	2
1.3	The Memory Hierarchy	2
1.3.1	I/O Complexity and the Two-Level Model	3
1.3.2	Cache-Oblivious Algorithms	3
1.4	Asymptotic Notation	3
1.5	Exercises	4
2	Warm-Up: Maximum Subarray Sum	5
2.1	The Problem	5
2.2	Cubic Algorithm – $\Theta(n^3)$	5
2.3	Quadratic Algorithm – $\Theta(n^2)$	6
2.4	Linear Algorithm – $\Theta(n)$ (Kadane’s Algorithm)	6
2.5	Second Linear Algorithm – Prefix Sums	7
2.6	Advanced Variants [∞]	8
2.6.1	CG-rich Segments in DNA	8
2.7	Exercises	8
3	Random Sampling	10
3.1	The Problem	10
3.2	Disk Model with Known Length	10
3.3	Streaming Model with Known Length	11
3.4	Streaming Model with Unknown Length	11
3.5	Exercises	12
4	List Ranking	13
4.1	The Problem	13
4.2	Pointer Jumping	13
4.3	Two-Level Memory Simulation	14
4.3.1	Divide-and-Conquer Algorithm	14

4.4	Exercises	14
5	Sorting Atomic Items	16
5.1	The Merge-Based Paradigm	17
5.1.1	Binary MergeSort and I/O Complexity	17
5.1.2	Optimization 1: Early Recursion Stop	17
5.1.3	Optimization 2: The Snow-Plow Technique	18
5.1.4	Optimization 3: Multi-Way MergeSort	18
5.2	Lower Bounds	19
5.2.1	Sorting Lower Bound	19
5.2.2	Permutation Lower Bound	20
5.3	The Distribution-Based Paradigm: QuickSort	21
5.3.1	Three-Way Partitioning	21
5.3.2	Pivot Selection	22
5.3.3	RandSelect: k -th Element Selection	23
5.4	Exercises	23
6	Set Intersection	25
6.1	Merge-Based Approach	25
6.2	Mutual Partitioning	26
6.3	Doubling Search (Exponential Search)	27
6.4	Two-Level Storage Approach	27
6.5	Exercises	28
7	String Sorting	30
7.1	Lower Bound	30
7.2	RadixSort	31
7.2.1	MSD-First RadixSort	31
7.2.2	LSD-First RadixSort	31
7.3	Multi-Key QuickSort	32
7.4	Exercises	33
8	The Dictionary Problem	34
8.1	Direct-Access Tables	34
8.2	Hash Tables with Chaining	34
8.2.1	Designing Good Hash Functions	35
8.3	Universal Hashing	36
8.4	Perfect (Static) Hash Tables	36
8.5	Cuckoo Hashing	37
8.6	Bloom Filter	38

8.7 Exercises	39
9 Prefix Search	40
9.1 Array of String Pointers	40
9.1.1 Contiguous Allocation and Two-Level Storage	41
9.1.2 Front Coding	41
9.2 Interpolation Search	41
9.3 Compacted Trie (Patricia Trie)	42
9.4 Exercises	43
10 Substring Search	44
10.1 Notation and Terminology	44
10.2 The Suffix Array	45
10.2.1 The Substring Search Problem	45
10.2.2 LCP-Optimized Search	46
10.2.3 The LCP Array and Its Construction	47
10.2.4 Suffix Array Construction	48
10.2.5 The Skew Algorithm (DC3)	48
10.3 The Suffix Tree	49
10.4 Exercises	50
11 Integer Coding	52
11.1 Unary Code	53
11.2 Codici di Elias: γ e δ	54
11.2.1 Il codice γ (gamma)	54
11.2.2 Il codice δ (delta)	54
11.3 Rice Code	55
11.4 PForDelta	55
11.5 Variable-Byte Code and (s, c) -Dense Codes	56
11.6 Interpolative Code	57
11.7 Elias–Fano Encoding	57
11.8 Summary and Comparison	59
11.9 Exercises	59
12 Statistical Coding	60
12.1 Huffman Coding	61
12.1.1 Huffman Tree Construction	61
12.1.2 Optimality of Huffman Coding	61
12.1.3 Canonical Huffman	62
12.2 Arithmetic Coding	63

12.3 Higher-Order Entropy and PPM	64
12.4 Exercises	65
13 Dictionary-Based Compression	66
13.1 LZ77	66
13.1.1 Implementation: gzip	67
13.2 LZ78	68
13.3 LZW	68
13.4 LZ77 vs. LZ78 Comparison	69
13.5 Exercises	70
14 Block-Sorting Compression	71
14.1 The Burrows–Wheeler Transform	71
14.1.1 Forward Transform	72
14.1.2 Backward Transform	73
14.1.3 Connection with the Suffix Array	75
14.2 Two Auxiliary Transforms: MTF and RLE	75
14.2.1 Move-to-Front (MTF)	76
14.2.2 Run-Length Encoding (RLE)	77
14.3 The bzip2 Pipeline	77
14.4 Theoretical Results on BWT	78
14.5 Exercises	78
15 Compressed Data Structures	80
15.1 Compressed Representation of Binary Arrays	80
15.1.1 Rank and Select Definitions	81
15.1.2 Succinct Rank Implementation	81
15.1.3 Select Implementation	82
15.2 The FM-index	83
15.2.1 Backward Search for Count	83
15.2.2 Locate and Extract	84
15.3 Summary and Perspectives	85
15.4 Exercises	85
16 Conclusions	87

Chapter 1

Introduction

1.1 What is an Algorithm

L'attore principale di questo libro è l'*algoritmo*. Il dizionario Oxford English Dictionary definisce informalmente un algoritmo come “*un processo, o insieme di regole, solitamente espresso in notazione algebrica, usato in informatica, traduzione automatica e linguistica.*”

In informatica, però, la definizione richiede maggiore precisione.

Seguiamo la definizione di Donald Knuth, risalente agli anni '60:

Definizione 1.1 (Algoritmo). Un algoritmo è una **procedura finita, definitiva, effettiva, con un input e un output.**

Analizziamo le caratteristiche fondamentali:

- **Finito:** L'algoritmo deve terminare dopo un numero **finito** (e ragionevole) di passi. La **ragionevolezza** è legata all'**efficienza**: vogliamo algoritmi *buoni*, non solo corretti.
- **Definitivo:** Ogni passo deve essere specificato in modo **rigoroso** e non ambiguo. Chiunque legga la descrizione dell'algoritmo deve poterlo eseguire nello stesso modo.
- **Effettivo:** Ogni **operazione** deve essere **atomica**, ovvero eseguibile in tempo **costante** (piccolo) con carta e matita. Questo implica una comprensione profonda del problema.
- **Input:** Quantità fornite all'algoritmo **prima dell'esecuzione**, prese da insiemi specificati.
- **Output:** Quantità **prodotte dall'algoritmo**, che hanno una relazione specificata con l'input.

Oltre alla correttezza, puntiamo all'*eleganza* e all'*efficienza*. L'efficienza si misura tradizionalmente come **complessità temporale** $T(n)$: il numero massimo di passi che l'algoritmo esegue su input di dimensione n (caso peggiore).

1.2 The RAM Model

Il modello classico per analizzare gli algoritmi è il **modello RAM** (*Random Access Machine*), noto anche come modello di Von Neumann.

Definizione 1.2 (Modello RAM). Una RAM è composta da:

- una CPU che esegue istruzioni in tempo costante,
- una memoria di dimensione infinita, con accesso in tempo costante a ogni cella.

Il **costo** di un algoritmo si valuta **contando il numero di passi** eseguiti.

Il modello RAM è stato dominante per decenni grazie alla sua semplicità, ma presenta **limitazioni** significative per grandi moli di dati:

1. L'**architettura** dei PC moderni è diventata sempre **più sofisticata** (multi-core, GPU, cache multilivello).
2. I **dati** sono esplosi in dimensione: " $n \rightarrow +\infty$ " è una realtà, non solo un'astrazione teorica.

1.3 The Memory Hierarchy

Un PC moderno non è semplicemente CPU + memoria. È dotato di una **gerarchia di livelli di memoria** (Figura 1.1), ciascuno con capacità, latenza e larghezza di banda proprie:

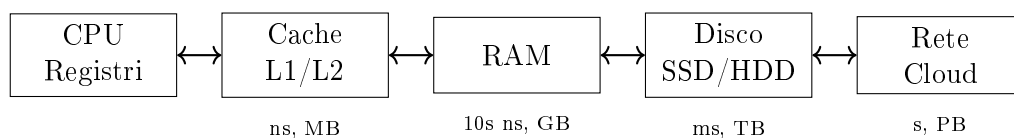


Figure 1.1: Gerarchia della memoria in un PC moderno. Il divario tra disco e RAM è dell'ordine di 10^5 – 10^6 .

Osservazione 1.3. Il divario di velocità tra CPU e disco (meccanico) è paragonabile alla differenza tra affilare una matita sulla propria scrivania e prendere un aereo dall'altra parte del mondo per farlo su una scrivania altrui. Questo **I/O bottleneck** è il problema centrale dell'**Algorithm Engineering**.

1.3.1 I/O Complexity and the Two-Level Model

Per catturare la realtà dell'I/O bottleneck, usiamo il **modello a due livelli di memoria** (anche detto *disk model* o *external-memory model*):

Definizione 1.4 (Modello a Due Livelli). Il modello a due livelli comprende:

- Una memoria interna (RAM) di dimensione M .
- Una memoria esterna (disco) di dimensione illimitata.
- I dati vengono trasferiti in **blocchi** (pagine disco) di dimensione B .
- Il costo dell'algoritmo è il numero di **I/O** (accessi a blocchi del disco).

Un I/O legge o scrive B elementi in un'unica operazione. Il modello estende a **D dischi in parallelo**, dove ogni I/O legge $D \times B$ elementi totali. Le due **regole** d'oro per algoritmi efficienti su grandi dati sono:

- **Località spaziale**: organizzare i dati sul disco in modo che gli accessi a pagine vicine siano il più possibile utili.
- **Località temporale**: fare più lavoro possibile sui dati già caricati in memoria interna prima di scriverli.

1.3.2 Cache-Oblivious Algorithms

Un algoritmo è **cache-oblivious** se non conosce esplicitamente i valori di B (dimensione del blocco) e M (dimensione della memoria interna), eppure ottimizza automaticamente il numero di I/O su qualsiasi configurazione della memoria.

Osservazione 1.5. Il vantaggio degli algoritmi cache-oblivious è la **portabilità**: funzionano in modo ottimale su qualsiasi livello della gerarchia di memoria, senza necessità di parametrizzazione.

1.4 Asymptotic Notation

Usiamo la notazione asintotica standard:

- $\mathcal{O}(f(n))$: limite superiore asintotico ("al più dell'ordine di $f(n)$ ").
- $\Omega(f(n))$: limite inferiore asintotico ("almeno dell'ordine di $f(n)$ ").
- $\Theta(f(n))$: limite stretto ($\mathcal{O}(f(n))$ e $\Omega(f(n))$ simultaneamente).

1.5 Exercises

1. Dimostrare che se $C(n) = n^a$ (con $a > 0$) e il computer migliora di un fattore k , il numero di dati processabili nello stesso tempo aumenta di un fattore $k^{1/a}$. Cosa succede se $C(n) = 2^n$?
2. Supponiamo che un'algoritmo $\mathcal{A}_{s,b}$ scandisca un array $A[1, n]$ di blocchi di dimensione b , saltando s blocchi ad ogni passo. Calcolare il numero di I/O in funzione di s , b , n e B (dimensione della pagina disco).
3. Discutere perché il modello RAM non è adeguato per analizzare algoritmi su dataset di dimensione $n = (1 + \varepsilon)M$, con M dimensione della memoria interna e $\varepsilon > 0$ piccolo.
4. Definire formalmente la probabilità di I/O fault $p(\varepsilon) = \varepsilon/(1 + \varepsilon)$ per un algoritmo con accessi random a $n = (1 + \varepsilon)M$ dati. Calcolare il rallentamento medio se il costo di un I/O è $c = 10^5$ rispetto a un accesso in RAM, e la frazione di passi che accede alla memoria è $f = 0.35$.

Chapter 2

Warm-Up: Maximum Subarray Sum

2.1 The Problem

Consideriamo il seguente problema classico, ispirato dalla finanza:

Problema (Massima Somma di Sottoarray). Dato un array $D[1, n]$ di numeri interi (positivi e negativi), trovare il sottoarray $D[b, s]$ (con $1 \leq b \leq s \leq n$) che massimizza la somma dei suoi elementi.

Motivazione pratica: $D[i]$ rappresenta la variazione giornaliera del prezzo di un'azione alla Borsa di New York. Vogliamo determinare il periodo $[b, s]$ che avrebbe massimizzato il guadagno comprando il giorno b e vendendo il giorno s .

Esempio 2.1. Con $n = 11$ e $D[1, 11] = [+4, -6, +3, +1, +3, -2, +3, -4, +1, -9, +6]$, il sottoarray ottimo è $D[3, 7] = [+3, +1, +3, -2, +3]$ con somma = 8.

Osservazione 2.2. Se tutti gli elementi sono positivi, il sottoarray ottimo è l'intero array. Se tutti sono negativi, scegliamo il sottoarray di lunghezza 1 con il valore meno negativo.

2.2 Cubic Algorithm – $\Theta(n^3)$

La soluzione più semplice **esamina tutte le coppie (b, s)** e per ciascuna calcola la somma da zero.

Algoritmo 2.1: Algoritmo **cubico** per la massima somma di sottoarray

Input: Array $D[1, n]$

Output: $\langle MaxSum, b_o, s_o \rangle$ con $MaxSum = \max_{b \leq s} \sum_{i=b}^s D[i]$

$MaxSum \leftarrow -\infty$ **for** $b \leftarrow 1$ **to** n **do** **for** $s \leftarrow b$ **to** n **do** $TmpSum \leftarrow 0$ **for** $i \leftarrow b$ **to** s **do** $TmpSum \leftarrow TmpSum + D[i]$ **if** $MaxSum < TmpSum$ **then** $MaxSum \leftarrow TmpSum$; $b_o \leftarrow b$; $s_o \leftarrow s$ **return** $\langle MaxSum, b_o, s_o \rangle$

Teorema 2.3. *L'algoritmo cubico ha complessità $\Theta(n^3)$.*

Dimostrazione. Il numero di coppie (b, s) è $\binom{n}{2} + n = \mathcal{O}(n^2)$. Per ciascuna coppia, il calcolo della somma costa $\mathcal{O}(n)$ nel caso peggiore. Quindi il costo totale è $\mathcal{O}(n^3)$.

Per il limite inferiore $\Omega(n^3)$: consideriamo i sottoarray di lunghezza $L \in [n/4, n/2]$. Ci sono $\Theta(n/4 - n/4) = \Theta(n)$ tali valori di L , e per ciascuno ci sono $(n - L + 1) \geq n/2$ sottoarray. Ogni sottoarray costa $L \geq n/4$ operazioni. Il costo totale è almeno $\Omega(n \cdot n/2 \cdot n/4) = \Omega(n^3)$. $\square$

2.3 Quadratic Algorithm – $\Theta(n^2)$

La inefficienza cubica deriva dal **ricalcolo completo della somma ad ogni cambio di s** . Notiamo che quando s aumenta di 1, la somma di $D[b, s + 1]$ si ottiene da quella di $D[b, s]$ **aggiungendo $D[s + 1]$** .

Algoritmo 2.2: Algoritmo **quadratico**

Input: Array $D[1, n]$

Output: $\langle \text{MaxSum}, b_o, s_o \rangle$

$\text{MaxSum} \leftarrow -\infty$ **for** $b \leftarrow 1$ **to** n **do** $\text{TmpSum} \leftarrow 0$ **for** $s \leftarrow b$ **to** n **do** $\text{TmpSum} \leftarrow \text{TmpSum} + D[s]$ **if** $\text{MaxSum} < \text{TmpSum}$ **then** $\text{MaxSum} \leftarrow \text{TmpSum};$ $b_o \leftarrow b;$ $s_o \leftarrow s$ **return** $\langle \text{MaxSum}, b_o, s_o \rangle$

Teorema 2.4. *L'algoritmo quadratico ha complessità $\Theta(n^2)$.*

Dimostrazione. Il numero totale di addizioni eseguite è:

$$\sum_{b=1}^n \sum_{s=b}^n 1 = \sum_{b=1}^n (n - b + 1) = n + (n - 1) + \dots + 1 = \frac{n(n + 1)}{2} = \Theta(n^2). \quad \square$$

2.4 Linear Algorithm – $\Theta(n)$ (Kadane's Algorithm)

Possiamo fare molto meglio. La chiave sono **due proprietà strutturali** del sottoarray ottimo.

Proposizione 2.5 (Proprietà 1). **La somma degli elementi di qualsiasi sottoarray $D[x, b_o - 1]$, con $x < b_o$, non può essere strettamente positiva.**

Dimostrazione. Se la somma fosse positiva, potremmo estendere $D[b_o, s_o]$ verso sinistra ottenendo un sottoarray con somma maggiore, contraddicendo l'ottimalità di $D[b_o, s_o]$. $\square$

Proposizione 2.6 (Proprietà 2). *La somma degli elementi di qualsiasi sottoarray $D[b_o, y]$, con $y \leq s_o$, non può essere strettamente negativa.*

Dimostrazione. Se la somma fosse negativa, potremmo troncare il sottoarray ottimo ottenendo $D[y + 1, s_o]$ con somma maggiore, contraddicendo l'ottimalità. $\square$

Queste due proprietà guidano l'algoritmo lineare: manteniamo un sottoarray candidato corrente e lo aggiorniamo in base al segno della somma parziale.

Algoritmo 2.3: Algoritmo **lineare** (Kadane)

Input: Array $D[1, n]$

Output: $\langle MaxSum, b_o, s_o \rangle$

```

 $MaxSum \leftarrow -\infty$ ;  $TmpSum \leftarrow 0$ ;  $b \leftarrow 1$  for  $s \leftarrow 1$  to  $n$  do  $TmpSum \leftarrow$ 
 $TmpSum + D[s]$  if  $MaxSum < TmpSum$  then  $MaxSum \leftarrow TmpSum$ ;  $b_o \leftarrow b$ ;
 $s_o \leftarrow s$  if  $TmpSum < 0$  then  $TmpSum \leftarrow 0$ ;  $b \leftarrow s + 1$  return  $\langle MaxSum, b_o, s_o \rangle$ 

```

Teorema 2.7. *L'algoritmo di Kadane ha complessità $\Theta(n)$ in tempo e $\mathcal{O}(1)$ in spazio aggiuntivo.*

Dimostrazione. Il ciclo esegue esattamente n iterazioni, ciascuna in tempo $\mathcal{O}(1)$. La correttezza si basa sul fatto che i sottoarray candidati esaminati formano una *partizione* di $D[1, n]$: ogni elemento è esaminato esattamente una volta. Non possiamo scartare nessuno di questi sottoarray senza esaminarne la somma (altrimenti potremmo perdere l'ottimo). La correttezza segue dalla Proprietà 1 (step 6–7: se $TmpSum < 0$, per la Proprietà 1 nessun ottimo può iniziare prima di $s + 1$) e dalla Proprietà 2 (step 4–5: aggiorniamo $MaxSum$ solo quando $TmpSum$ migliora). $\square$

Osservazione 2.8 (Cache-Obliviousness). L'algoritmo di Kadane **scansiona D una sola volta da sinistra a destra: esegue n/B I/O, che è ottimale.** Notare che B non appare nel codice: **l'algoritmo è cache-oblivious.**

2.5 Second Linear Algorithm – Prefix Sums

Esiste una **formulazione alternativa** basata sulle **somme prefisse**:

$$\max_s \max_{b \leq s} \text{Sum}_D[b, s] = \max_s \left(P[s] - \min_{b \leq s} P[b - 1] \right),$$

dove $P[i] = \sum_{j=1}^i D[j]$ è la somma prefissa (con $P[0] = 0$). Questo permette di calcolare $\min_{b \leq s} P[b-1]$ in modo incrementale.

Algoritmo 2.4: Secondo algoritmo lineare (prefix sums)

Input: Array $D[1, n]$

Output: $\langle MaxSum, b_o, s_o \rangle$

```

     $MaxSum \leftarrow -\infty$ ;  $b_{tmp} \leftarrow 1$   $TmpSum \leftarrow 0$ ;  $MinTmpSum \leftarrow 0$  for  $s \leftarrow 1$ 
to  $n$  do  $TmpSum \leftarrow TmpSum + D[s]$  if  $MaxSum < TmpSum - MinTmpSum$ 
then  $MaxSum \leftarrow TmpSum - MinTmpSum$ ;  $s_o \leftarrow s$ ;  $b_o \leftarrow b_{tmp}$  if  $TmpSum <$ 
 $MinTmpSum$  then  $MinTmpSum \leftarrow TmpSum$ ;  $b_{tmp} \leftarrow s+1$  return  $\langle MaxSum, b_o, s_o \rangle$ 

```

2.6 Advanced Variants[∞]

2.6.1 CG-rich Segments in DNA

In bioinformatica, le stringhe del DNA usano l'alfabeto $\{A, T, C, G\}$. Si vuole trovare il segmento più ricco in nucleotidi C e G. Due formulazioni:

1. **Massima somma con peso:** assegnare $-p$ a A e T, $1-p$ a C e G. Il segmento di lunghezza ℓ con x occorrenze di C o G ha somma $x - p \cdot \ell$. Tutti gli algoritmi precedenti si applicano.
2. **Massima densità:** trovare il *segmento più lungo* con densità $x/\ell \geq t$. Si riduce a trovare il segmento più lungo con somma ≥ 0 nell'array $D'[i] = D[i] - t$. Soluzione in $\mathcal{O}(n)$.

2.7 Exercises

1. Implementare l'algoritmo di Kadane e verificarne la correttezza sull'esempio $D = [+4, -6, +3, +1, +3, -2, +3, -4, +1, -9, +6]$.
2. Modificare l'algoritmo di Kadane per trovare il sottoarray di somma massima tra tutti quelli di lunghezza esattamente k .
3. Dimostrare che il lower bound temporale per il problema della massima somma di sottoarray è $\Omega(n)$.
4. Considerare la variante 2D: dato un array $n \times n$, trovare il sottomatite rettangolare di somma massima. Proporre un algoritmo in $\mathcal{O}(n^3)$ e discutere il lower bound.

5. Dato un array circolare $D[1, n]$ (dove $D[n]$ e $D[1]$ sono adiacenti), estendere l'algoritmo di Kadane per trovare il sottoarray circolare di somma massima.

Chapter 3

Random Sampling

3.1 The Problem

Problema (Campionamento Casuale). Data una sequenza S di n elementi, estrarre un campione di k elementi *uniformemente a caso* (senza reinserimento), cioè in modo tale che ogni sottoinsieme di k elementi abbia la stessa probabilità $\binom{n}{k}^{-1}$ di essere selezionato.

Studieremo tre varianti basate sul modello computazionale:

3.2 Disk Model with Known Length

In questo modello, n è noto e la sequenza è memorizzata su disco. Possiamo accedere agli elementi in ordine e caricarli in memoria interna.

Algoritmo 3.1: Campionamento con n noto (Knuth)

Input: Sequenza $S[1, n]$ su disco, intero $k \leq n$

Output: Campione casuale di k elementi

```
sample  $\leftarrow \emptyset$ ;  r  $\leftarrow n$ ;  s  $\leftarrow k$  for i  $\leftarrow 1$  to n do if random(0,1)  $\leq s/r$  then  
  Aggiungi  $S[i]$  al campione  s  $\leftarrow s - 1$   r  $\leftarrow r - 1$  if s = 0 then break  return  
sample
```

Teorema 3.1. L'algoritmo di Knuth produce un campione uniforme di k elementi in $\mathcal{O}(n)$ tempo e $\mathcal{O}(k)$ spazio, con n/B I/O.

Sketch. Al passo i , abbiamo già visto $i - 1$ elementi e ne mancano $n - i + 1$; ne abbiamo scelti $k - s$ e ne mancano s . La probabilità di includere $S[i]$ è $s/(n - i + 1) = s/r$, che è esattamente la probabilità che un elemento non ancora visto debba essere incluso nel

campione dato il numero rimanente. Si verifica per induzione che ogni sottoinsieme di k elementi ha la stessa probabilità di essere scelto. $\square$

3.3 Streaming Model with Known Length

Nel modello *streaming*, gli elementi arrivano in sequenza e non possiamo tornare indietro. Se n è noto, si può usare lo stesso algoritmo di Knuth, che già funziona in modalità streaming.

3.4 Streaming Model with Unknown Length

Il caso più interessante e pratico: n è sconosciuto. Gli elementi arrivano in streaming e vogliamo mantenere un campione di k elementi uniformemente distribuito in qualsiasi momento.

Algoritmo 3.2: Reservoir Sampling (Vitter)

Input: Stream S di lunghezza ignota, intero k

Output: Campione casuale di k elementi

```
// Fase 1: riempi il reservoir con i primi k elementi
for i ← 1 to k do R[i] ← S[i] // Fase 2: per ogni nuovo elemento, rimpiazza con
// prob. k/i
i ← k+1 while S non è terminato do j ← random(1, i) if j ≤ k then R[j] ← S[i]
i ← i + 1 return R
```

Teorema 3.2 (Reservoir Sampling). *Al termine della lettura di n elementi, il reservoir R contiene un campione uniforme di k elementi tra gli n visti, con probabilità $1/\binom{n}{k}$ per ogni sottoinsieme di k elementi.*

Dimostrazione. Dimostriamo per induzione su i che dopo aver letto i elementi, ogni elemento ha probabilità k/i di trovarsi nel reservoir.

Caso base: $i = k$. Tutti e k gli elementi sono nel reservoir, quindi $P[\text{elemento } j \text{ nel reservoir}] = 1 = k/k$. ✓

Passo induttivo: Supponiamo che dopo i elementi, ciascuno abbia probabilità k/i di essere nel reservoir. Al passo $i + 1$:

- L'elemento $S[i + 1]$ entra nel reservoir con probabilità $k/(i + 1)$.
- Un elemento $S[j]$ con $j \leq i$ già nel reservoir (con prob. k/i) rimane se $S[i + 1]$ non lo sostituisce: la sostituzione avviene solo se $S[i + 1]$ è selezionato ($k/(i + 1)$)

E la posizione casuale r è quella di $S[j]$ ($1/k$). Quindi la probabilità di rimanere è:

$$\frac{k}{i} \cdot \left(1 - \frac{k}{i+1} \cdot \frac{1}{k}\right) = \frac{k}{i} \cdot \frac{i}{i+1} = \frac{k}{i+1}. \quad \square$$

Osservazione 3.3 (Complessità). Reservoir Sampling usa $\mathcal{O}(k)$ spazio e $\mathcal{O}(n)$ tempo. Ogni elemento dello stream è esaminato esattamente una volta: il numero di I/O è n/B (ottimale e cache-oblivious).

3.5 Exercises

1. Implementare il Reservoir Sampling e verificare empiricamente l'uniformità della distribuzione con $n = 1000$, $k = 10$, ripetendo l'esperimento 10000 volte.
2. Modificare il Reservoir Sampling per campionare con probabilità non uniforme: l'elemento $S[i]$ deve avere peso $w_i > 0$ e la probabilità di inclusione nel campione deve essere proporzionale al suo peso.
3. Dimostrare che il Reservoir Sampling usa esattamente n/B I/O nel modello a due livelli, e che questo è ottimale.
4. Considerare il caso in cui vogliamo campionare k elementi *con* reinserimento. Progettare un algoritmo in $\mathcal{O}(k+n)$ tempo e $\mathcal{O}(k)$ spazio.

Chapter 4

List Ranking

4.1 The Problem

Problema (List Ranking). Data una lista concatenata L di n elementi su disco (non necessariamente contigui in memoria), determinare per ogni elemento x la sua posizione $\text{rank}(x)$ nella lista (numero di elementi successori di x , incluso x stesso).

L'interesse algoritmico sta nel fatto che la scansione naïve è $\mathcal{O}(n)$ in RAM ma richiede $\mathcal{O}(n)$ I/O in memoria esterna (ogni elemento potrebbe essere in una pagina diversa), il che è catastrofico per grandi dataset.

4.2 Pointer Jumping

Il *pointer jumping* è una tecnica classica per algoritmi su liste/alberi. Ad ogni passo, ogni elemento salta al successore del proprio successore, dimezzando la distanza.

Algoritmo 4.1: Pointer Jumping per List Ranking

Input: Lista L con puntatori $\text{next}[i]$

Output: $\text{rank}[i]$ per ogni i

// Inizializzazione

for ogni nodo i in L **do if** $\text{next}[i] = \text{NIL}$ **then** $\text{rank}[i] \leftarrow 1$ $\text{rank}[i] \leftarrow 0$; $w[i] \leftarrow 1$

// Iterazioni di pointer jumping

while esistono nodi con $\text{next}[i] \neq \text{NIL}$ **do for** ogni nodo i con $\text{next}[i] \neq \text{NIL}$ **do**
 $\text{rank}[i] \leftarrow \text{rank}[i] + w[\text{next}[i]]$ $w[i] \leftarrow w[i] + w[\text{next}[i]]$ $\text{next}[i] \leftarrow \text{next}[\text{next}[i]]$ **return**
 rank

Teorema 4.1. Il *Pointer Jumping* risolve List Ranking in $\mathcal{O}(\log n)$ passi paralleli, ciascuno di $\mathcal{O}(n)$ operazioni, per un totale di $\mathcal{O}(n \log n)$ operazioni sequenziali.

4.3 Two-Level Memory Simulation

Eseguire pointer jumping in memoria esterna è problematico: ogni salto del puntatore può causare un I/O fault. Abbiamo bisogno di un approccio divide-and-conquer.

4.3.1 Divide-and-Conquer Algorithm

L'idea è di selezionare un sottoinsieme indipendente di nodi $I \subseteq L$ (*independent set*), rimuoverli dalla lista, risolvere ricorsivamente, e poi reintegrare.

Definizione 4.2 (Insieme Indipendente). Un insieme I di nodi è *indipendente* se nessun nodo in I è successore di un altro nodo in I .

Algoritmo 4.2: List Ranking Divide-and-Conquer (versione esterna)

Input: Lista L con n nodi

Output: rank[i] per ogni nodo

if $n \leq M$ (entra in memoria interna) **then** Calcola rank direttamente **return** //

Passo 1: costruisci un insieme indipendente casuale I

Ogni nodo si inserisce in I con probabilità $1/2$ se il suo predecessore non è già in I
 // *Passo 2: rimuovi I dalla lista, aggiorna pesi*

for ogni $v \in I$ **do** $w[\text{prev}(v)] += w[v]$ Collega $\text{prev}(v) \rightarrow \text{next}(v)$ // *Passo 3: risolvi ricorsivamente su $L \setminus I$*

ListRanking($L \setminus I$) // *Passo 4: reintegra i nodi di I*

for ogni $v \in I$ **do** rank[v] $\leftarrow$ rank[$\text{prev}(v)$] + $w[v]$ **return** rank

Teorema 4.3. L'algoritmo divide-and-conquer per List Ranking esegue $\mathcal{O}(\text{sort}(n))$ I/O in attesa, dove $\text{sort}(n) = \mathcal{O}\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$ è il costo di ordinare n elementi.

Sketch. Ad ogni livello di ricorsione, la lista si riduce di un fattore costante in media (l'insieme indipendente ha $n/4$ nodi in attesa). Il numero di livelli è $\mathcal{O}(\log n)$. Ogni livello richiede un'operazione di ordinamento per allineare i puntatori ai blocchi su disco: $\mathcal{O}(\text{sort}(n))$ I/O. Il costo totale è $\mathcal{O}(\log n \cdot \text{sort}(n))$. Con un'analisi più accurata usando tecniche di compaction si ottiene $\mathcal{O}(\text{sort}(n))$. $\square$

4.4 Exercises

1. Dimostrare che la scansione naïve di una lista concatenata memorizzata in modo non contiguo su disco richiede $\mathcal{O}(n)$ I/O nel caso peggiore.

2. Mostrare che il Pointer Jumping richiede $\lceil \log_2 n \rceil$ iterazioni per risolvere List Ranking su una lista di n nodi.
3. Dimostrare che un insieme indipendente casuale (ogni nodo entra con probabilità $1/2$ se il predecessore non è già dentro) ha dimensione attesa $\geq n/4$.
4. Descrivere come estendere l'algoritmo divide-and-conquer per calcolare il *prefix sum* di valori $v[i]$ associati ai nodi della lista: $P[i] = \sum_{j \text{ precede } i} v[j]$.

Chapter 5

Sorting Atomic Items

We adore chaos because we love to produce order.

— *M. C. Escher*

In questo capitolo affrontiamo il classico problema dell'ordinamento di un insieme di **elementi atomici**: elementi che occupano un numero costante e fisso di celle di **memoria** (tipicamente interi o reali a 32 o 64 bit). Il caso di elementi a lunghezza variabile (stringhe) è trattato nel capitolo successivo.

Problema 5.1 (Sorting Problem). Data una sequenza di **n elementi atomici $S[1, n]$** e un **ordinamento totale $\leq$ tra ogni coppia, ordinare S in ordine crescente**.

Esamineremo due paradigmi complementari: il paradigma **merge-based** (che sottende MERGESORT) e il paradigma **distribution-based** (che sottende QUICKSORT). Adatteremo entrambi al modello a due livelli di memoria, analizzeremo la loro complessità I/O e dimostreremo che gli algoritmi ottenuti sono **I/O-ottimali**, stabilendo un **lower bound** sofisticato. Introdurremo anche la tecnica Snow-Plow e la compressione come strumenti per aumentare virtualmente la memoria M .

Collegeremo infine l'ordinamento al *problema della permutazione*:

Problema 5.2 (Permuting Problem). Data una sequenza di n elementi atomici $S[1, n]$ e una **permutazione $\pi[1, n]$** degli interi $\{1, 2, \dots, n\}$, **permutare S secondo π** , ottenendo la sequenza **$S[\pi[1]], S[\pi[2]], \dots, S[\pi[n]]$** .

Nel modello **RAM**, **ordinare** richiede **$\Theta(n \log n)$** e **permutare $\Theta(n)$** : esiste un gap logaritmico. Sorprendentemente, nel modello su **disco** i due problemi hanno la **stessa complessità I/O** sotto condizioni ragionevoli sui parametri n, M, B .

5.1 The Merge-Based Paradigm

5.1.1 Binary MergeSort and I/O Complexity

MERGESORT si basa sul paradigma **divide-et-impera**. L'array S viene diviso a metà, le due **metà** vengono **ordinate ricorsivamente**, e infine **fuse** con la procedura **MERGE**.

Algoritmo 5.1: Binary MergeSort: $\text{MERGESORT}(S, i, j)$

Input: Array S , indici $i \leq j$

Output: $S[i, j]$ ordinato

if $i < j$ **then** $m \leftarrow \lfloor (i + j)/2 \rfloor$ $\text{MERGESORT}(S, i, m - 1)$ $\text{MERGESORT}(S, m, j)$
 $\text{MERGE}(S, i, m, j)$

Analisi nel modello RAM. La relazione di ricorrenza è $T(n) = 2T(n/2) + O(n) = O(n \log n)$, come noto da qualsiasi corso di algoritmi base.

Analisi I/O nel modello a due livelli. Assumiamo $n > M$ (altrimenti non ci sono I/O). La fusione di due sottosequenze di dimensione totale z richiede $O(z/B)$ I/O: questo vale perché $M \geq 3B$ garantisce che le due disk page correnti e la pagina di output siano tenute in memoria interna, e ogni volta che un puntatore avanza su una nuova pagina viene effettuato un I/O. Dunque la relazione di ricorrenza per la complessità I/O è:

$$T(n) = 2T(n/2) + O(n/B) = \Theta\left(\frac{n}{B} \log n\right) \text{ I/O.}$$

Questo bound è però ottimistico rispetto alla realtà: **non sfrutta ancora la disponibilità della memoria interna di dimensione M** . Quando una sottosequenza di dimensione $z = O(M)$ entra interamente in memoria interna, il suo ordinamento costa solo $O(z/B)$ I/O (per caricarla), non $\Theta((z/B) \log z)$. Questa osservazione risparmia $\Theta((n/B) \log M)$ I/O in totale, riducendo la complessità a:

$$\Theta\left(\frac{n}{B} \log \frac{n}{M}\right) \text{ I/O.}$$

5.1.2 Optimization 1: Early Recursion Stop

La prima ottimizzazione introduce una soglia sulla dimensione della sottosequenza: quando $j - i < cM$ (con $c \leq 1$ costante che dipende dallo spazio usato dallo sorter interno), si carica l'intera sottosequenza in memoria e si applica un sorter interno efficiente (es. HEAPSORT o QUICKSORT).

Il parametro c vale 1 per sorter in-place come INSERTIONSORT e HEAPSORT, ed è circa 0.5 per MERGESORT (che usa un array ausiliario di dimensione n). È quindi preferibile usare **sorter in-place per massimizzare c e minimizzare il fattore logaritmico**. La **complessità** diventa $\Theta(\frac{n}{B} \log \frac{n}{cM})$, che **asintoticamente equivale a $\Theta(\frac{n}{B} \log \frac{n}{M})$** .

5.1.3 Optimization 2: The **Snow-Plow** Technique

La tecnica Snow-Plow, attribuita a Donald Knuth, consente di **aumentare virtualmente la dimensione della memoria di un fattore 2 in media**. Questa tecnica scansiona la sequenza S e genera *sorted run* di lunghezza variabile, mai inferiore a M e pari a $2M$ in media.

L'idea si basa su un **min-heap $\mathcal{H}$ di dimensione M e un array ausiliario non ordinato $\mathcal{U}$** . Una *fase* dell'algoritmo procede come segue:

Algoritmo 5.2: Una fase della **tecnica Snow-Plow**

Require: $\mathcal{U}$ è un array non ordinato di M elementi

Costruisci $\mathcal{H}$ come min-heap sugli elementi di $\mathcal{U}$ Poni $\mathcal{U} \leftarrow \emptyset$ **while** $\mathcal{H} \neq \emptyset$ **do**
 $\text{min} \leftarrow$ estrai il minimo da $\mathcal{H}$ Scrivi min nel run di output $\text{next} \leftarrow$ leggi il prossimo
 elemento da S **if** $\text{next} < \text{min}$ **then...** **else** inserisci next in $\mathcal{U}$ inserisci next in $\mathcal{H}$

Correttezza. Durante l'esecuzione di una fase il minimo di $\mathcal{H}$ è non decrescente, quindi il run di output è ordinato. Gli elementi in $\mathcal{H}$ all'inizio di una fase vengono tutti scritti prima della fine.

Lunghezza media dei run. Supponiamo che una fase legga τ elementi in totale. All'inizio ci sono M elementi in $\mathcal{U}$ (che diventano $\mathcal{H}$). Durante la fase, $\tau - M$ elementi aggiuntivi vengono letti: ognuno va in $\mathcal{H}$ o in $\mathcal{U}$ con probabilità $1/2$ ciascuno (assumendo distribuzione casuale dell'input). Poiché $|\mathcal{U}| \leq M$, si conclude che in media $\tau/2$ elementi vanno in $\mathcal{U}$ e $\tau/2$ in $\mathcal{H}$. Ma $|\mathcal{U}| = M$ implica $\tau/2 = M$, quindi $\tau = 2M$.

Fatto 5.3 (Fatto 5.1). Snow-Plow genera **$O(n/M)$ run ordinati, ciascuno di lunghezza $> M$ e mediamente $2M$** . Usando Snow-Plow per la generazione dei run, MERGESORT ha complessità I/O media $O(\frac{n}{B} \log \frac{n}{2M})$.

5.1.4 Optimization 3: Multi-Way MergeSort

Le precedenti ottimizzazioni hanno ridotto il numero di livelli di ricorsione, ma la fusione rimane binaria: processa **2 run alla volta**. Questo **limita** la base del logaritmo a 2.

Osserviamo però che la memoria interna contiene $M/B \gg 3$ blocchi, quasi tutti inutilizzati durante una fusione binaria. L'idea è eseguire una *fusione k -way* che processa $k = (M/B) - 1$ run simultaneamente: si mantiene un blocco per ogni run da leggere e uno per il run di output.

Per trovare efficientemente il minimo tra k candidati si usa un min-heap di k coppie $\langle \text{item}, \text{run_index} \rangle$. Ogni estrazione e inserimento nell'heap richiede $O(\log k)$ tempo, per un costo totale di $O(\log k)$ per elemento e $O(z/B)$ I/O per fondere k run di dimensione totale z .

La struttura ricorsiva è un albero k -ario con $O(n/M)$ foglie (run di dimensione $\geq M$), quindi il numero di livelli di fusione è $\log_k(n/M) = \log_{(M/B)}(n/M)$. Il costo totale è:

Teorema 5.4 (MergeSort Multi-Way — Teorema 5.1). **MERGESORT** *multi-way* esegue

$$O\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$$

I/O e $O(n \log n)$ confronti per ordinare n elementi atomici in un modello a due livelli con memoria interna M e dimensione di pagina B .

Dimostrazione. Il numero di livelli di fusione è $\log_k(n/M)$ con $k = (M/B) - 1 = \Theta(M/B)$. Ad ogni livello si leggono e scrivono tutti gli n elementi: costo $O(n/B)$ I/O per livello. Il costo totale è $O\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$. Poiché $\log_{M/B} M = \Theta(\log_{M/B} B)$ asintoticamente, possiamo anche scrivere $O\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$. Il numero di confronti segue dall'analisi classica di MERGESORT. $\square$

Osservazione 5.5. In pratica, con $B = 4\text{KB}$ e $M = 4\text{GB}$, si ha $M/B = 2^{20}$, quindi bastano $\log_{2^{20}}(n/M)$ livelli: tipicamente 1 o 2 livelli per dataset di decine di GB.

5.2 Lower Bounds

5.2.1 Sorting Lower Bound

Usiamo la tecnica dell'albero di decisione per dimostrare che il MERGESORT multi-way è I/O-ottimale.

Nel modello RAM, ogni nodo dell'albero è un confronto tra due elementi; le $n!$ foglie corrispondono alle permutazioni dell'input. La profondità dell'albero binario più basso con $n!$ foglie è $\lceil \log_2 n! \rceil = \Omega(n \log n)$ per l'approssimazione di Stirling.

Nel modello a due livelli, ogni nodo corrisponde a un singolo I/O. Il fan-out di un nodo è il numero di risultati distinti che un I/O può generare: leggendo B nuovi elementi dal disco e distribuendoli tra gli $M - B$ elementi già presenti in memoria,

si possono generare al più $\binom{M}{B}(B!)$ risultati distinti. Dopo t I/O, il numero di foglie raggiungibili è dunque al massimo $(\binom{M}{B}(B!))^t$.

Per ordinare, occorre distinguere tutte le $n!$ permutazioni, quindi:

$$\left(\binom{M}{B}(B!)\right)^{t-n/B} \times \left(n\binom{M}{B}\right)^{n/B} \geq n!$$

dove n/B dei nodi leggono nuove pagine (untouched) e i restanti leggono pagine già processate. Risolvendo rispetto a t si ottiene:

Teorema 5.6 (Lower Bound per l'Ordinamento — Teorema 5.3). *In un modello a due livelli con memoria interna M , dimensione di pagina B e D dischi, qualsiasi algoritmo di ordinamento comparison-based deve eseguire almeno*

$$\Omega\left(\frac{n}{DB} \log_{M/B} \frac{n}{DB}\right) \text{ I/O.}$$

Sketch. L'argomento è quello dell'albero di decisione sopra descritto. Per $D = 1$: dopo t I/O, il numero di ordinamenti distinguibili è al più $(\binom{M}{B}(B!))^t$. Imponendo che questo sia $\geq n!$, e applicando l'approssimazione di Stirling, si ricava $t = \Omega\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$. Il caso $D > 1$ introduce il fattore $1/(DB)$ per il parallelismo sui dischi. $\square$

Quindi MERGESORT multi-way (Teorema 5.1) è asintoticamente ottimale per $D = 1$.

5.2.2 Permutation Lower Bound

Teorema 5.7 (Complessità della Permutazione — Teorema 5.2). *Permutare n elementi richiede $O(\min(n, \frac{n}{B} \log_{M/B} \frac{n}{M}))$ I/O in un modello a due livelli.*

Il lower bound per la permutazione distingue tre tipi di I/O: lettura di una pagina nuova (untouched), lettura di una pagina già toccata (touched), scrittura. Denotando con P_t il numero massimo di permutazioni generate da un algoritmo con t I/O e con t_r , t_w i read e write I/O ($t = t_r + t_w$):

$$P_t \leq \left(\frac{n}{B} \binom{M}{B}(B!)\right)^{n/B} \times \left(n \binom{M}{B}\right)^{t_r - n/B} \times n^{t_w}.$$

Imponendo $P_t \geq n!$ e risolvendo, si ottiene:

Teorema 5.8 (Lower Bound per la Permutazione — Teorema 5.4). *In un modello a due livelli con memoria M , dimensione di pagina B e D dischi, permutare n elementi richiede almeno $\Omega(\min(\frac{n}{D}, \frac{n}{DB} \log_{M/B} \frac{n}{DB}))$ I/O.*

Osservazione 5.9. I due bound (Teoremi 5.3 e 5.4) sono asintoticamente equivalenti quando $B = \Omega(\log_{M/B}(n/M))$, condizione soddisfatta in pratica per qualsiasi n fino agli exabyte con i valori correnti di B e M . **Dunque nel modello su disco ordinare = permutare in termini di complessità I/O.**

5.3 The Distribution-Based Paradigm: QuickSort

QUICKSORT è basato sul paradigma **divide-et-impera**, come MERGESORT, ma il suo passo di divisione è sofisticato (sceglie un pivot e partiziona) mentre il passo di combinazione è assente. La qualità dell'algoritmo dipende fortemente dalla **scelta del pivot**.

Algoritmo 5.3: **Binary QuickSort:** QUICKSORT(S, i, j)

Input: Array S , indici $i \leq j$

Output: $S[i, j]$ ordinato

if $i < j$ **then** $r \leftarrow$ seleziona la posizione di un “buon pivot” scambia $S[r]$ con $S[i]$ $p \leftarrow$ PARTITION(S, i, j) QUICKSORT($S, i, p - 1$) QUICKSORT($S, p + 1, j$)

PARTITION(S, i, j) partiziona $S[i, j]$ in due parti: gli elementi $\leq$ pivot vengono posti prima, quelli $>$ pivot dopo. Alla fine, il pivot si trova nella posizione p (la sua posizione definitiva), e la procedura restituisce p .

5.3.1 Three-Way Partitioning

Il partizionamento classico può essere migliorato con un **partizionamento a tre vie** che gestisce esplicitamente gli elementi uguali al pivot, collocandoli in una sottosequenza centrale che non richiede ulteriori ricorsioni.

Algoritmo 5.4: **Three-way partitioning:** PARTITION(S, i, j)

$P \leftarrow S[i]; \quad l \leftarrow i; \quad r \leftarrow i + 1$ **for** $c \leftarrow r$ **downto** j **do** **if** $S[c] = P$ **then**... **else** scambia $S[c]$ con $S[r]$ $r++$ **if** $S[c] < P$ **then** scambia $S[c]$ con $S[l]$ scambia $S[c]$ con $S[r]$ $r++$; $l++$ **return** $\langle l, r - 1 \rangle$

L'invariante mantenuto è: $S[i, l - 1]$ contiene elementi $< P$, $S[l, r - 1]$ contiene elementi $= P$, $S[r, c - 1]$ contiene elementi $> P$. Al termine, la sottosequenza centrale $S[l, r - 1]$ è già nella posizione definitiva: la ricorsione opera solo su $S[i, l - 1]$ e $S[r, j]$.

Il partizionamento a tre vie è ottimale per sequenze con molti duplicati (caso frequente in pratica) e offre accessi *stream-like* all'array S , favorendo il prefetch del processore.

5.3.2 Pivot Selection

La scelta del pivot determina il bilanciamento delle partizioni. Un pivot che divide S in due metà uguali produce $O(n \log n)$ confronti; uno che separa un elemento dagli altri produce $O(n^2)$.

Pivot casuale. La scelta naturale è selezionare il pivot *uniformemente a caso* tra gli elementi di $S[i, j]$.

Teorema 5.10 (Teorema 5.5). *La selezione casuale del pivot fa eseguire a QUICKSORT al più $2n \ln n$ confronti in attesa.*

Dimostrazione. Sia $X_{u,v}$ la variabile aleatoria indicatore dell'evento che $S[u]$ e $S[v]$ vengano confrontati durante l'esecuzione. Il numero atteso di confronti è:

$$E \left[\sum_{u,v} X_{u,v} \right] = \sum_{u=1}^n \sum_{v=u+1}^n p_{u,v}$$

dove $p_{u,v}$ è la probabilità che $S[u]$ e $S[v]$ vengano confrontati. Due elementi vengono confrontati solo se uno di essi è il pivot prima che uno qualsiasi degli elementi con valore strettamente tra $S[u]$ e $S[v]$ venga scelto come pivot. Sia b il numero di tali elementi intermedi; allora ci sono 2 scelte “buone” e b “cattive”, quindi:

$$p_{u,v} = \frac{2}{b+2} = \frac{2}{v' - u' + 1}$$

dove $u' < v'$ sono i ranghi di $S[u]$ e $S[v]$ nella sequenza ordinata. Sommando su tutte le coppie:

$$\sum_{u'=1}^n \sum_{v'=u'+1}^n \frac{2}{v' - u' + 1} = 2 \sum_{u'=1}^n \sum_{k=2}^{n-u'+1} \frac{1}{k} \leq 2 \sum_{u'=1}^n \sum_{k=1}^n \frac{1}{k} \leq 2nH_n \leq 2n \ln n,$$

dove $H_n = \sum_{k=1}^n 1/k$ è il numero armonico. □

Mediana di campioni. Per rendere la selezione più robusta, si campionano $2s+1$ elementi a caso e si prende la loro mediana come pivot.

Teorema 5.11 (Teorema 5.6). *Se QUICKSORT partiziona intorno alla mediana di $2s+1$ elementi casuali, ordina n elementi distinti in $\frac{2nH_s}{H_{2s+2}-H_{s+1}} + O(n)$ confronti attesi, dove $H_z = \sum_{i=1}^z 1/i$.*

Aumentando s si avvicina il numero atteso di confronti a $n \log n + O(n)$, al costo di una selezione del pivot più costosa ($O(s \log s)$ confronti).

5.3.3 RandSelect: k -th Element Selection

RANDSELECT seleziona l'elemento di rango k in una sequenza non ordinata, sfruttando la stessa struttura di partizionamento di QUICKSORT ma ricorrendo solo su una delle tre parti.

Algoritmo 5.5: Selezione del k -esimo elemento: **RANDSELECT**(S, k)

Input: Sequenza S di n elementi, rango k

Output: L'elemento di rango k in S

$r \leftarrow$ posizione casuale in $\{1, 2, \dots, n\}$ $S_{<} \leftarrow$ elementi di S minori di $S[r]$ $S_{>} \leftarrow$ elementi di S maggiori di $S[r]$ $n_{<} \leftarrow |S_{<}|$; $n_{=} \leftarrow |S| - |S_{<}| - |S_{>}|$ **if** $k \leq n_{<}$ **then...**
else return **RANDSELECT**($S_{<}, k$) **if** $k \leq n_{<} + n_{=}$ **then...** **else return** $S[r]$ **return** **RANDSELECT**($S_{>}, k - n_{<} - n_{=}$)

Teorema 5.12 (Teorema 5.7). **RANDSELECT** seleziona il k -esimo elemento in $O(n)$ tempo atteso nel modello RAM e $O(n/B)$ I/O attesi nel modello a due livelli.

Dimostrazione. Definiamo come “buona selezione” quella in cui $n_{<}$ e $n_{>}$ sono entrambi $\leq 2n/3$: questo avviene quando $S[r]$ ha rango in $[n/3, 2n/3]$, che si verifica con probabilità $1/3$ scegliendo il pivot uniformemente a caso. Sia $\hat{T}(n)$ la complessità attesa. Si può scrivere:

$$\hat{T}(n) \leq O(n) + \frac{1}{3}\hat{T}(2n/3) + \frac{2}{3}\hat{T}(n).$$

Il secondo termine (prob. $1/3$) è una chiamata ricorsiva su un'istanza di dimensione $\leq 2n/3$ dopo una buona selezione; il terzo termine (prob. $2/3$) è un limite superiore nel caso peggiore, trattando le selezioni cattive come ricorsioni sull'intera sequenza. Sottraendo $\frac{2}{3}\hat{T}(n)$ da entrambi i lati:

$$\frac{1}{3}\hat{T}(n) \leq O(n) + \frac{1}{3}\hat{T}(2n/3) \implies \hat{T}(n) \leq O(n) + \hat{T}(2n/3) = O(n).$$

Nel modello a due livelli, la costruzione di $S_{<}$, $S_{=}$, $S_{>}$ richiede una scansione di S : $O(n/B)$ I/O. La ricorsione opera su una sottosequenza di dimensione $\leq 2n/3$: $\hat{T}(n) = O(n/B) + \hat{T}(2n/3) = O(n/B)$. $\square$

5.4 Exercises

1. Dimostrare che la complessità I/O di MERGESORT binario su una sequenza di n elementi è $\Theta(\frac{n}{B} \log \frac{n}{M})$, partendo dalla relazione di ricorrenza $T(n) = 2T(n/2) + O(n/B)$ e applicando la correzione per l'utilizzo della memoria interna.

2. La tecnica Snow-Plow produce run di lunghezza media $2M$. Mostrare che, se invece di input casuale l'input è già ordinato in modo crescente, Snow-Plow produce run di lunghezza n (un unico run). Cosa succede se l'input è ordinato in modo decrescente?
3. Considerare un MERGESORT multi-way con $k = M/B - 1$ vie. Calcolare il numero esatto di livelli di fusione necessari per ordinare $n = 10^9$ elementi con $M = 1$ GB e $B = 4$ KB. Confrontare con MERGESORT binario.
4. (Esame 2022) Dimostrare il lower bound $\Omega\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$ I/O per l'ordinamento nel modello a due livelli, usando la tecnica dell'albero di decisione. In particolare, calcolare il fan-out massimo di un nodo dell'albero nel modello su disco.
5. Analizzare la complessità attesa di QUICKSORT con pivot casuale nel modello a due livelli. Mostrare che la complessità I/O attesa è $O\left(\frac{n}{B} \log n\right)$, peggiore di $O\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$ del MERGESORT multi-way. Perché allora QUICKSORT è spesso preferito in pratica?
6. Modificare RANDSELECT per trovare simultaneamente i k -esimi elementi per $k = \lfloor n/4 \rfloor$ e $k = \lfloor 3n/4 \rfloor$ (i quartili). Qual è la complessità I/O attesa?

Chapter 6

Set Intersection

Sharing is caring!

Questo capitolo affronta il problema dell'intersezione di insiemi, che è il nucleo computazionale di ogni motore di ricerca testuale. Un motore di ricerca mantiene un *indice invertito*: per ogni parola del vocabolario, una *posting list* $\mathcal{L}[w]$ contiene i docID dei documenti che contengono w . Data una query $Q = w_1 w_2 \cdots w_k$, trovare i documenti che contengono tutte le parole equivale a calcolare $\mathcal{L}[w_1] \cap \mathcal{L}[w_2] \cap \cdots \cap \mathcal{L}[w_k]$.

Problema 6.1 (Sorted Set Intersection). Date due sequenze intere ordinate $A = a_1 a_2 \cdots a_n$ e $B = b_1 b_2 \cdots b_m$ (con $a_i < a_{i+1}$ e $b_i < b_{i+1}$), **calcolare gli interi comuni a entrambe le sequenze** $A \cap B$.

Assumiamo senza perdita di generalità $m \leq n$. L'approccio banale (confronto incrociato) richiede $O(nm)$ passi: impraticabile per posting list di milioni di docID.

6.1 Merge-Based Approach

L'idea è semplice: poiché A e B sono ordinate, le scansioni parallele da sinistra a destra permettono di trovare i comuni senza ripetere confronti. Si mantengono due puntatori i e j , inizialmente a 1.

- Se $a_i < b_j$: incrementa i .
- Se $a_i > b_j$: incrementa j .
- Se $a_i = b_j$: aggiungi a_i all'intersezione, incrementa entrambi.

Teorema 6.2 (Teorema 6.1). *L'algoritmo merge-based risolve il problema di intersezione ordinata in $O(m + n)$ passi. Questa complessità è I/O-ottimale nel modello cache-oblivious, richiedendo $O(n/B)$ I/O.*

Dimostrazione. Ad ogni passo eseguiamo un confronto e avanziamo almeno un puntatore. Poiché $i \leq n$ e $j \leq m$, il totale è $\leq n + m$ passi. Nel modello a due livelli, A e B vengono scansionate sequenzialmente: costo $O((n + m)/B) = O(n/B)$ I/O. Il lower bound $\Omega(\min(n, m))$ segue dal fatto che ogni elemento di B deve essere processato almeno una volta. $\square$

Quando $m \ll n$, possiamo fare meglio. L'approccio con *ricerca binaria* cerca ogni $b \in B$ nella lista A (lunga n), richiedendo $O(m \log n)$ confronti.

Teorema 6.3 (Teorema 6.2). *L'algoritmo basato sulla ricerca binaria risolve il problema di intersezione ordinata in $O(m \log n)$ passi.*

Questo è meglio di $O(m + n)$ quando $m = o(n / \log n)$.

6.2 Mutual Partitioning

Il *partizionamento mutuo* combina i vantaggi di entrambi gli approcci. L'idea: usare la *mediana* di B (la sequenza più corta) come pivot e cercarlo in A tramite ricerca binaria. Il pivot divide ricorsivamente entrambe le sequenze in due parti che vengono intersecate in corrispondenza.

Algoritmo 6.1: Set Intersection tramite Mutual Partitioning

Input: $A[1, n]$ e $B[1, m]$ ordinate, $m \leq n$

Output: $A \cap B$

if $m = 0$ **then return** $\emptyset$ $p \leftarrow b_{\lfloor m/2 \rfloor}$ (mediana di B) Ricerca binaria di p in A : trovare j t.c. $a_j \leq p < a_{j+1}$ **if** $p = a_j$ **then stampa** p Calcola ricorsivamente $A[1, j] \cap B[1, m/2 - 1]$ Calcola ricorsivamente $A[j + 1, n] \cap B[m/2 + 1, m]$

Teorema 6.4 (Teorema 6.3). *L'algoritmo mutual partitioning risolve il problema di intersezione ordinata in $O(m(1 + \log \frac{n}{m}))$ passi. Questa complessità è ottimale nel modello di confronto.*

Dimostrazione. La relazione di ricorrenza è $T(n, m) = O(\log n) + 2T(n/2, m/2)$ con caso base $T(n, 0) = O(1)$. Risolvendo: $T(n, m) = O(m \log n / m)$ perché si hanno m livelli di ricorsione e ogni livello esegue ricerche binarie sulla porzione corrente di A .

Ottimalità: Ci sono $\binom{n}{m}$ soluzioni possibili (sottoinsiemi di A di dimensione al più m). L'albero di decisione deve avere $\geq \binom{n}{m}$ foglie, quindi profondità $\geq \log \binom{n}{m} = \Omega(m \log(n/m))$. $\square$

6.3 Doubling Search (Exponential Search)

Il mutual partitioning è ottimale ma fa uso intensivo di ricerca binaria e chiamate ricorsive, che generano accessi non sequenziali alla memoria — inefficienti su disco. La *ricerca esponenziale* (o *galloping search*) è un approccio iterativo che ottiene la stessa complessità con accessi più cache-friendly.

Idea. Supponiamo di aver già trovato la posizione di b_{j-1} in A : essa si trova in posizione i (cioè $a_i \leq b_{j-1} < a_{i+1}$). Per trovare b_j nel suffisso $A[i+1, n]$, invece di una ricerca binaria immediata, effettuiamo una *ricerca per raddoppio*: confrontiamo b_j con $A[i+2^0]$, $A[i+2^1]$, $A[i+2^2]$, ... fino a trovare k tale che $b_j < A[i+2^k]$ o fino a uscire dai limiti. Poi eseguiamo una ricerca binaria nell'intervallo $A[i+2^{k-1}, \min(i+2^k, n)]$.

Algoritmo 6.2: Set Intersection tramite Doubling Search

Input: $A[1, n]$ e $B[1, m]$ ordinate, $m \leq n$

Output: $A \cap B$

```

     $i \leftarrow 0$  for  $j \leftarrow 1, 2, \dots, m$  do  $k \leftarrow 0$  while  $(i + 2^k \leq n)$  e  $(B[j] > A[i + 2^k])$ 
do  $k \leftarrow k + 1$   $i' \leftarrow$  ricerca binaria di  $B[j]$  in  $A[i + 2^{k-1} + 1, \min(i + 2^k, n)]$  if  $A[i'] = B[j]$ 
then stampa  $B[j]$   $i \leftarrow i'$ 

```

Teorema 6.5 (Teorema 6.4). *L'algoritmo doubling search risolve il problema di intersezione ordinata in $O(m(1 + \log \frac{n}{m}))$ passi. Questa complessità è ottimale nel modello di confronto.*

Dimostrazione. Sia $\Delta_j = \min(2^{k_j}, n) \leq 2(i_j - i_{j-1})$ la dimensione del sottovettore su cui si esegue la ricerca binaria per b_j , dove i_j è la posizione finale. L'algoritmo esegue $O(1 + \log \Delta_j)$ passi per ogni b_j . Sommando su tutti gli m elementi di B :

$$\sum_{j=1}^m O(1 + \log \Delta_j) = O\left(m + \log \prod_{j=1}^m \Delta_j\right) \leq O\left(m + m \log \frac{\sum \Delta_j}{m}\right) = O\left(m \log \frac{n}{m}\right),$$

dove si usa la disuguaglianza di Jensen ($\log$ è concavo). Il vantaggio rispetto al mutual partitioning è che l'accesso alla sequenza A è *iterativo* (senza ricorsione), il che riduce overhead e migliora la cache locality. $\square$

6.4 Two-Level Storage Approach

Gli approcci precedenti richiedono accessi non sequenziali ad A (salti durante la ricerca binaria), inefficienti su disco. L'approccio *two-level storage* organizza le posting list in

modo gerarchico per ottimizzare gli I/O.

Preprocessing. Ogni lista A di lunghezza n viene suddivisa in $h = n/L$ blocchi $A_1, A_2, \dots, A_h$ di dimensione L ciascuno. Si crea una *sequenza ausiliaria* $A' = (A_1[1], A_2[1], \dots, A_h[1])$ di dimensione $h = n/L$ contenente il primo elemento di ogni blocco.

Intersezione (Fase 1 + Fase 2). Dati A (lungo, $|A| = n$) e B (corto, $|B| = m$):

1. **Fase 1:** Fondi A' e B con l'algoritmo merge-based. Per ogni elemento di B , questo identifica il blocco A_i di A dove potrebbe essere presente. Costo: $O(n/L + m)$.
2. **Fase 2:** Per ogni coppia (A_i, B_i) con B_i non vuoto (sottoinsiemi di B che cadono nel blocco A_i), calcola $A_i \cap B_i$ con l'algoritmo merge-based. Costo: $O(L + |B_i|)$ per coppia, $O(mL + m)$ in totale (poiché $\sum |B_i| = m$ e i blocchi non vuoti sono al più m).

Teorema 6.6 (Teorema 6.5). *L'algoritmo two-level storage risolve il problema di intersezione ordinata in $O(n/L + mL)$ passi e $O(n/(LB) + m/B + m)$ I/O, dove B è la dimensione di pagina del disco. Scegliendo $L = \sqrt{n/m}$, il tempo è $O(\sqrt{nm})$ e gli I/O sono $O(\sqrt{n/m}/B + m/B + m)$.*

Dimostrazione. Il tempo di Fase 1 è $O(n/L + m)$ (merge di due liste di dimensione n/L e m). Quello di Fase 2 è $O(\sum_i (L + |B_i|)) = O(mL + m)$ (al più m blocchi non vuoti, ognuno di dimensione L). Totale: $O(n/L + mL + m) = O(n/L + mL)$.

Gli I/O: la Fase 1 accede sequenzialmente ad A' e B ($O(n/(LB) + m/B)$); la Fase 2 accede ai blocchi selezionati di A ($O(m \cdot L/B)$)... ma nel modello a due livelli ogni blocco è una pagina, quindi $O(m)$ I/O). Sommando: $O(n/(LB) + m/B + m)$.

Il parametro ottimale $L = \sqrt{n/m}$ bilancia n/L e mL . □

Osservazione 6.7. La complessità $O(\sqrt{nm})$ migliora sia $O(m + n)$ (quando $m \ll n$) sia $O(m \log n)$ (quando m è vicino a n). Nel contesto di motori di ricerca, il two-level storage consente inoltre di sfruttare la compressione differenziale degli interi nelle posting list (gap-coding), riducendo ulteriormente gli I/O.

6.5 Exercises

1. Dimostrare formalmente che l'algoritmo merge-based per l'intersezione (due puntatori) è corretto: se $a_i < b_j$ allora $a_i \notin B$, e quindi può essere scartato.
2. Generalizzare l'algoritmo di intersezione al caso di k posting list $\mathcal{L}_1, \dots, \mathcal{L}_k$ di dimensioni $n_1 \leq n_2 \leq \dots \leq n_k$. Proporre una strategia efficiente e analizzarne la complessità.

3. (Esame 2021) Dimostrare il lower bound $\Omega(m \log(n/m))$ per l'intersezione di insiemi nel modello di confronto, usando un argomento basato sul numero di soluzioni possibili.
4. Nell'approccio two-level storage, il parametro L determina il trade-off tra la Fase 1 e la Fase 2. Trovare il valore ottimale di L minimizzando $f(L) = n/L + mL$ e verificare che produce complessità $O(\sqrt{nm})$.
5. Descrivere come implementare la doubling search in modo da minimizzare le cache miss, supponendo che ogni accesso a una posizione di A non ancora caricata in cache costi un'unità. Confrontare con la ricerca binaria pura.
6. (Collegamento al Capitolo 5) Spiegare perché l'algoritmo two-level storage per l'intersezione richiede che le posting list siano ordinate, e discutere il trade-off tra il costo di ordinamento preventivo e il beneficio sulle interrogazioni.

Chapter 7

String Sorting

Problema 7.1 (String Sorting Problem). Data una sequenza $S[1, n]$ di stringhe, con lunghezza totale N e alfabeto di dimensione σ , **ordinare S in ordine lessicografico crescente**.

Indichiamo con $L = N/n$ la lunghezza media delle stringhe. Un sorter comparison-based applicato direttamente (ad es. MERGESORT o QUICKSORT) richiederebbe $O(Ln \log n) = O(N \log n)$ confronti nel caso medio, perché ogni confronto tra due stringhe costa $O(L)$ caratteri. Questo è *subottimale*.

Il problema fondamentale è che le stringhe di S sono tipicamente implementate come array di puntatori: ogni confronto richiede di accedere a due posizioni non contigue in memoria (cache miss o I/O fault), rendendo l'algoritmo lento su memoria gerarchica.

7.1 Lower Bound

Definiamo il *prefisso distinguente* d_s di una stringa $s \in S$ come la lunghezza del più corto prefisso di s che la distingue da tutte le altre stringhe di S . La somma $d = \sum_{s \in S} d_s$ è il *prefisso distinguente totale* dell'insieme S .

Lemma 7.2 (Lemma 7.1). *Qualsiasi algoritmo che risolve il problema dell'ordinamento di stringhe deve eseguire $\Omega(d + n \log n)$ confronti di caratteri.*

Il termine d quantifica il costo minimo per distinguere tutte le stringhe (ogni stringa deve essere confrontata per almeno d_s caratteri), mentre $n \log n$ è il costo minimo per determinare l'ordine tra le n stringhe (argomento dell'albero di decisione). Gli algoritmi basati su MERGESORT o QUICKSORT impiegano $O(N \log n)$ confronti, distanti dall'ottimo di un fattore $\Theta(\log n)$ quando $d \ll N$.

7.2 RadixSort

RADIXSORT tratta le stringhe come sequenze di caratteri su un alfabeto intero $\{0, 1, \dots, \sigma-1\}$. Esistono due varianti principali.

7.2.1 MSD-First RadixSort

L'approccio MSD-first (Most Significant Digit first) processa i caratteri dall'inizio alla fine delle stringhe, distribuendole ricorsivamente in σ bucket in base al carattere corrente. La struttura dati sottostante è un *trie*: ogni nodo è un array di dimensione σ , e il percorso dalla radice a un nodo u descrive il prefisso condiviso da tutte le stringhe nel sottoalbero di u .

Teorema 7.3 (Teorema 7.1). *MSD-first RADIXSORT ordina un insieme di stringhe con alfabeto di dimensione σ e prefisso distinguente d in $O(d\sigma)$ tempo e spazio, usando un trie con array di dimensione σ in ogni nodo.*

Dimostrazione. Ogni stringa s percorre un cammino nel trie di lunghezza d_s . In ogni nodo visitato si alloca un array di dimensione σ : costo $O(\sigma)$ per nodo. Il numero totale di nodi (interni) è al più d (i nodi unari corrispondono ai caratteri dei prefissi distinguenti, i nodi di branching sono al più n). Il costo totale è $O(d\sigma)$. $\square$

Usando tabelle hash invece di array σ -ari nei nodi del trie si ottiene:

Teorema 7.4 (Teorema 7.2). *MSD-first RADIXSORT con hashing nei nodi del trie ordina le stringhe in $O(d \log \sigma)$ tempo atteso e $O(d)$ spazio.*

7.2.2 LSD-First RadixSort

L'approccio LSD-first (Least Significant Digit first) processa le stringhe dalla cifra meno significativa alla più significativa. Richiede un sorter *stabile* come COUNTINGSORT. Assumiamo che le stringhe abbiano tutte lunghezza L (o vengano logicamente completate con un carattere minore di qualsiasi altro).

Lemma 7.5 (Lemma 7.2). *LSD-first RADIXSORT risolve il problema dell'ordinamento di stringhe in $O(L(n + \sigma)) = O(N + L\sigma)$ tempo e $O(N)$ spazio. Il sorter è in-place se e solo se COUNTINGSORT è in-place ($\sigma = O(1)$).*

Dimostrazione. L'algoritmo esegue L fasi, ognuna ordinando le n stringhe secondo la loro i -esima cifra con COUNTINGSORT in $O(n + \sigma)$. La stabilità di COUNTINGSORT garantisce che le fasi successive preservano l'ordine stabilito da quelle precedenti. La correttezza segue dalla stabilità: dati $\alpha < \beta$ lessicograficamente, il primo carattere di mismatch è $a < b$; la fase corrispondente pone α prima di β , e la stabilità delle fasi successive preserva questo ordine. $\square$

Ottimizzazione: processare più cifre (detti *gruppi*) alla volta riduce il numero di fasi al costo di aumentare la dimensione dell'alfabeto.

Lemma 7.6 (Lemma 7.3). *LSD-first RADIXSORT richiede $\Theta(\frac{b}{r}(n + 2^r))$ tempo e $O(nb) = O(N \log \sigma)$ spazio per ordinare n stringhe di b bit ciascuna, raggruppando r bit alla volta.*

Minimizzando rispetto a r : poiché COUNTINGSORT richiede $O(n + 2^r)$ per fase, è inutile usare $r < \log n$ (si sprecherebbero le fasi extra). Scegliendo $r = \Theta(\log n)$, si ottiene il minimo:

Teorema 7.7 (Teorema 7.3 + Corollario 7.1). *LSD-first RADIXSORT ordina n stringhe di b bit ciascuna in $O(bn/\log n)$ tempo e $O(nb)$ spazio. In termini dell'alfabeto di dimensione σ (con $b = L \log \sigma$ e $N = nb/\log \sigma$), l'algoritmo richiede $O(N \log \sigma / \log n)$ tempo e $O(N \log \sigma)$ bit di spazio.*

7.3 Multi-Key QuickSort

Il *Multi-key QuickSort* è una variante di QUICKSORT ottimizzata per stringhe. A differenza del QuickSort classico dove il pivot è un intero, qui il pivot è un *carattere* in una posizione fissa, e la partizione è a tre vie.

Algoritmo 7.1: Multi-Key QuickSort: MULTIKEYQS(R, i)

Input: Insieme R di stringhe (prefix-free), posizione $i \geq 0$

Output: R ordinato lessicograficamente

if $|R| \leq 1$ **then return** R Scegli una stringa pivot $p \in R$ (casualmente) $R_{<} \leftarrow \{s \in R \mid s[i] < p[i]\}$ $R_{=} \leftarrow \{s \in R \mid s[i] = p[i]\}$ $R_{>} \leftarrow \{s \in R \mid s[i] > p[i]\}$
 $A \leftarrow \text{MULTIKEYQS}(R_{<}, i)$ $B \leftarrow \text{MULTIKEYQS}(R_{=}, i + 1)$ $C \leftarrow \text{MULTIKEYQS}(R_{>}, i)$
return concatenazione A, B, C

L'invariante è: tutte le stringhe in R sono già ordinate lessicograficamente fino al loro $(i - 1)$ -esimo carattere. La ricorsione su $R_{=}$ avanza il puntatore i (le stringhe uguali sul carattere i devono essere distinte dai successivi); quella su $R_{<}$ e $R_{>}$ mantiene i invariato.

Teorema 7.8 (Teorema 7.4). *MULTIKEYQS risolve il problema dell'ordinamento di stringhe eseguendo $O(d + n \log n)$ confronti di caratteri in attesa. Questo è ottimale nel modello comparison-based (Lemma 7.1). Il bound può essere reso worst-case adottando la mediana come pivot.*

Sketch. Per ogni stringa $s \in R$ e ogni chiamata ricorsiva $\text{MULTIKEYQS}(R, i)$: se $s \in R_{<} \cup R_{>}$, il carattere $s[i]$ viene confrontato con $p[i]$ e s va in un sottoinsieme di dimensione $\leq \alpha n$ (per $\alpha < 1$ costante, con pivot casuale bilanciato). Se $s \in R_{=}$, il confronto avviene e poi i avanza. Il numero totale di confronti che coinvolgono s nel primo caso è $O(\log n)$ in attesa (come nel QuickSort classico); nel secondo caso è d_s (perché i avanza solo quando $s[i] = p[i]$, e può avanzare al massimo d_s volte prima che s raggiunga un singleton). Sommando su tutte le stringhe: $O(n \log n + d)$. $\square$

Connessione ai Ternary Search Tree. C'è un'elegante corrispondenza tra MULTIKEYQS e i *ternary search tree* (TST): strutture dati in cui ogni nodo contiene un carattere *split*, con tre puntatori (low/equal/high). Un TST è ottenuto da un trie comprimendo ogni nodo con un solo figlio. La ricerca di un pattern $P[1, p]$ in un TST bilanciato richiede $O(p + \log n)$ confronti di caratteri.

Teorema 7.9 (Teorema 7.5). *La ricerca di un pattern $P[1, p]$ in un ternary search tree bilanciato che rappresenta n stringhe richiede $O(p + \log n)$ confronti di caratteri. Questo è ottimale nel modello comparison-based.*

7.4 Exercises

1. Dimostrare il Lemma 7.1: qualsiasi algoritmo comparison-based per l'ordinamento di stringhe richiede $\Omega(d + n \log n)$ confronti di caratteri. Discutere i due termini separatamente.
2. Confrontare MSD-first e LSD-first RADIXSORT in termini di: (a) numero di confronti; (b) utilizzo della memoria; (c) comportamento su sequenze con molti prefissi comuni.
3. (Esame 2020) Descrivere e analizzare un algoritmo per ordinare n stringhe binarie di lunghezza b che sia ottimale per $b = \Theta(\log n)$.
4. Implementare MULTIKEYQS con selezione del pivot come mediana tra tre elementi casuali. Analizzare come cambia la complessità attesa.
5. Descrivere il funzionamento di un ternary search tree su un esempio di 5 stringhe, e spiegare il collegamento con MULTIKEYQS .
6. Adattare LSD-first RADIXSORT al caso in cui le stringhe abbiano lunghezze diverse. Analizzare la complessità.

Chapter 8

The Dictionary Problem

Impossible is a word to be found only in the dictionary of fools.
— *Napoleone Bonaparte*

Problema 8.1 (Dictionary Problem). Sia $\mathcal{D}$ un insieme di n oggetti, chiamato il *dizionario*, univocamente identificati da chiavi tratte da un universo U . Il problema del dizionario consiste nel progettare una struttura dati che supporta efficientemente tre operazioni:

- **Search**(k): verifica se $\mathcal{D}$ contiene un oggetto con chiave k ; restituisce **true/false** (o il valore associato, se richiesto).
- **Insert**(x): inserisce in $\mathcal{D}$ l'oggetto x indicizzato dalla chiave $k = \text{key}[x]$.
- **Delete**(k): rimuove da $\mathcal{D}$ l'oggetto indicizzato dalla chiave k , se presente.

Assumeremo $U = \{0, 1, 2, \dots\}$ come universo di interi non negativi.

8.1 Direct-Access Tables

La struttura dati più semplice è una *tabella binaria* T di dimensione $u = |U|$ bit: $T[k] = 1$ se e solo se $k \in S$. Le operazioni richiedono $O(1)$ nel caso peggiore. Il limite è lo spazio: $O(u)$ bit, non dipende da n , quindi è impraticabile se $u \gg n$ (es. $u = 2^{32}$ per chiavi intere a 32 bit).

8.2 Hash Tables with Chaining

Le *tabelle hash* risolvono il problema dello spazio. Una tabella hash con chaining consiste in un array $T[0, m-1]$ di liste (bucket). Una *funzione hash* $h : U \rightarrow$

$\{0, 1, \dots, m-1\}$ mappa ogni chiave a una posizione della tabella. Le operazioni si riducono a operazioni sulla lista $T[h(k)]$.

L'efficienza dipende dalla capacità di h di distribuire uniformemente le chiavi tra i bucket. Il caso peggiore (tutte le chiavi nello stesso bucket) produce ricerca in $\Theta(n)$.

Definizione 8.2 (Simple Uniform Hashing). Una funzione hash h soddisfa l'*hashing uniforme semplice* se ogni chiave $k \in S$ è egualmente probabile di essere mappata in qualsiasi degli m slot di T , indipendentemente dalle altre chiavi.

Teorema 8.3 (Teorema 8.1). *Sotto l'ipotesi di hashing uniforme semplice, esiste una tabella hash con chaining di dimensione m tale che l'operazione $\text{Search}(k)$ su un dizionario di n oggetti richiede $\Theta(1 + \alpha)$ tempo atteso, dove $\alpha = n/m$ è il fattore di carico della tabella.*

Dimostrazione. Per una ricerca senza successo ($k \notin S$), il tempo è proporzionale alla lunghezza della lista $T[h(k)]$. Per l'hashing uniforme, la lunghezza attesa di $T[i]$ è $\sum_{k \in S} \mathcal{P}(h(k) = i) = |S|/m = \alpha$. Aggiungendo 1 per il calcolo di $h(k)$, il tempo è $O(1 + \alpha)$.

Per una ricerca con successo ($k \in S$, dove k è la j -esima chiave inserita), il tempo è proporzionale alla posizione di k nella lista $T[h(k)]$, che dipende da quanti elementi inseriti dopo k vanno nello stesso bucket. Il numero atteso è $(j-1)/m$. Mediando su tutte le n chiavi: $\frac{1}{n} \sum_{j=1}^n (1 + \frac{j-1}{m}) = 1 + \frac{\alpha}{2} - \frac{1}{2m} = O(1 + \alpha)$. $\square$

Corollario 8.4 (Corollario 8.1). *Una tabella hash con chaining occupa $O((m+n) \log_2 n + n \log_2 u)$ bit.*

Corollario 8.5 (Corollario 8.2). *Con $m = \Theta(n)$ e global rebuilding (raddoppio/dimezzamento della tabella), esiste una tabella hash con chaining dinamica che usa $O(n)$ spazio e garantisce Search in $O(1)$ tempo atteso e Insert/Delete in $O(1)$ tempo ammortizzato atteso.*

8.2.1 Designing Good Hash Functions

L'hashing uniforme semplice è difficile da garantire computazionalmente. I due schemi pratici più diffusi sono:

- **Hashing per divisione:** $h(k) = k \bmod m$. Semplice e veloce. Si evitano potenze di 2 per m ; si preferiscono numeri primi lontani dalle potenze di 2.
- **Hashing per moltiplicazione:** $h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$ con $0 < A < 1$. La scelta di m non è critica; suggerito $A = (\sqrt{5} - 1)/2 \approx 0,618$.

Nessuno di questi schemi garantisce l'hashing uniforme semplice per qualsiasi input: è sempre possibile costruire un dizionario avversariale che li degrade. La soluzione è l'hashing universale.

8.3 Universal Hashing

Definizione 8.6 (Classe Universale — Definizione 8.1). Sia $\mathcal{H}$ una collezione finita di funzioni hash che mappano U in $\{0, 1, \dots, m-1\}$. $\mathcal{H}$ è una *classe universale* se e solo se, per qualsiasi coppia di chiavi distinte $x, y \in U$:

$$|\{h \in \mathcal{H} : h(x) = h(y)\}| \leq \frac{|\mathcal{H}|}{m}.$$

In altre parole, una funzione hash scelta a caso da $\mathcal{H}$ collide su x e y con probabilità $\leq 1/m$.

Teorema 8.7 (Teorema 8.2). Sia $T[0, m-1]$ una tabella hash con chaining con h scelta a caso da una classe universale $\mathcal{H}$. La lunghezza attesa di ogni lista in T è al più $1 + \alpha$, qualsiasi sia il dizionario di input S .

Dimostrazione. Per ogni coppia di chiavi $x, y \in S$ con $x \neq y$, definiamo la variabile indicatore $I_{xy} = 1$ se $h(x) = h(y)$. Per la definizione di classe universale: $E[I_{xy}] = \mathcal{P}(h(x) = h(y)) \leq 1/m$. Per linearità: $E[N_x] = \sum_{y \in S, y \neq x} E[I_{xy}] \leq (n-1)/m < \alpha$. $\square$

Teorema 8.8 (Teorema 8.3). Con hashing universale in una tabella $T[0, n-1]$ di dimensione $m = n$, la lunghezza della lista più lunga in T è $O(\log n / \log \log n)$ con alta probabilità.

Esistenza di Classi Universali

Teorema 8.9 (Teorema 8.4). Un esempio di classe universale è l'insieme delle funzioni hash

$$h_a(k) = \sum_{i=0}^{r-1} a_i k_i \bmod m,$$

dove m è primo, $a = [a_0, a_1, \dots, a_{r-1}]$ con $a \in [1, |U| - 1]$, e $k = [k_0, \dots, k_{r-1}]$ sono le parti di k in base m . La dimensione di $\mathcal{H}$ è $|U| - 1 = m^r - 1$.

Varianti pratiche non richiedono operazioni modulo su numeri arbitrari. Lo schema *multiply-add-shift* usa $h_{a,b}(k) = ((ak + b) \bmod 2^w) \gg 2^{w-\ell}$ con a dispari e $b \geq 0$, implementabile con soli shift e moltiplicazioni su word di $2w$ bit.

8.4 Perfect (Static) Hash Tables

Definizione 8.10 (Funzione Hash Perfetta — Definizione 8.2). Una funzione hash $h : U \rightarrow \{0, 1, \dots, m-1\}$ è *perfetta* rispetto a un dizionario S di n chiavi se e solo se $h(k') \neq h(k'')$ per ogni coppia di chiavi distinte $k', k'' \in S$.

Per il caso statico (il dizionario non cambia), si usa uno schema a *due livelli*: una tabella primaria $T_1[0, n-1]$ con funzione hash universale h_1 , e n tabelle secondarie $T_{2,j}$ di dimensione $m_j = n_j^2$ per gestire le collisioni in $T_1[j]$.

Teorema 8.11 (Teorema 8.5). *Se si memorizzano q chiavi in una tabella hash di dimensione $s = q^2$ usando una funzione hash universale, il numero atteso di collisioni è $< 1/2$. Quindi la probabilità di avere almeno una collisione è $< 1/2$.*

Dimostrazione. Ci sono $\binom{q}{2} < q^2/2$ coppie di chiavi che possono collidere. Con $s = q^2$, ogni coppia collide con probabilità $\leq 1/s = 1/q^2$ (per la classe universale). Per linearità: numero atteso di collisioni $< (q^2/2) \cdot (1/q^2) = 1/2$. Per Markov: $\mathcal{P}(X \geq 1) \leq E[X] < 1/2$. $\square$

Teorema 8.12 (Teorema 8.6). *Se si memorizzano n chiavi in una tabella T_1 di dimensione n con funzione hash universale, allora $E\left[\sum_{j=0}^{n-1} (n_j)^2\right] < 2n$, dove $n_j = |S_j|$.*

Combinando: lo spazio totale delle tabelle secondarie è $\sum_j m_j = \sum_j n_j^2 < 2n$ in attesa. La ricerca richiede esattamente 2 lookup nella tabella ($O(1)$ nel caso peggiore).

8.5 Cuckoo Hashing

Per dizionari dinamici, il *cuckoo hashing* garantisce tempo costante nel caso peggiore per la ricerca e tempo costante ammortizzato atteso per l'inserimento.

Struttura. Due funzioni hash h_1 e h_2 , una tabella $T[0, m-1]$. Ogni chiave k è memorizzata in $T[h_1(k)]$ oppure in $T[h_2(k)]$.

Ricerca e cancellazione. Guardare in $T[h_1(k)]$ e $T[h_2(k)]$: $O(1)$ nel caso peggiore.

Inserimento (Schema dell'evizione a cascata): se $T[h_1(k)]$ è libero, inserire k lì. Se $T[h_2(k)]$ è libero, inserire k lì. Altrimenti, scegliere una delle due posizioni (es. $T[h_1(k)]$), *scacciare* la chiave k' ivi presente, inserire k al suo posto, e tentare di ricollocare k' nella sua posizione alternativa. Ripetere. Se il loop supera m passi, si esegue il rehashing con nuove funzioni hash.

Algoritmo 8.1: Cuckoo Hashing: Insert($k \notin S$) in $T[0, m-1]$

if $T[h_1(k)] = \text{NULL}$ **then** $T[h_1(k)] \leftarrow k$; **return** **if** $T[h_2(k)] = \text{NULL}$ **then** $T[h_2(k)] \leftarrow k$; **return** $\text{count} \leftarrow 0$; $s \leftarrow 2$ **while** $\text{count} < m$ **do** $d \leftarrow \{1, 2\} \setminus \{s\}$; $\text{pos} \leftarrow h_d(k)$ scambia k con $T[\text{pos}]$ **if** $k = \text{NULL}$ **then return** $s \leftarrow h_s(k) = \text{pos}$; $\text{count}++$ Rehash con nuove funzioni hash (possibilmente raddoppiando m); ripeti l'inserimento

La struttura matematica sottostante è il *cuckoo graph*: nodi = slot della tabella, archi = chiavi (un arco per chiave da $T[h_1(k)]$ a $T[h_2(k)]$). Un inserimento fallisce (ciclo infinito) se e solo se il cuckoo graph contiene un ciclo.

Teorema 8.13 (Teorema 8.7). *Per qualsiasi coppia di nodi i e j e costante $c > 1$, se $m \geq 2cn$, la probabilità che nel cuckoo graph (con m nodi e n archi) esista un cammino da i a j di lunghezza $L \geq 1$ è al più c^{-L}/m .*

Corollario 8.14 (Corollario 8.3). *Con $c = 3$, l'inserimento di en chiavi in un dizionario di dimensione n implementato via cuckoo hashing su una tabella di dimensione $m \geq 6n(1 + \epsilon)$ richiede $\Theta(n)$ tempo ammortizzato atteso.*

Sketch. La probabilità di esistenza di un ciclo nel cuckoo graph (evento di fallimento del rehashing) è al più $\frac{m}{m(c-1)} = \frac{1}{c-1}$. Con $c = 3$, questa probabilità è $\leq 1/2$, quindi è necessario un numero costante di rehashing. Il costo di un singolo rehashing è $O(n)$, e questo costo viene ammortizzato su $O(n)$ inserimenti. $\square$

8.6 Bloom Filter

I *Bloom filter* sono strutture dati probabilistiche per dizionari in cui le chiavi sono molto lunghe (es. URL), rendendo il costo di memorizzazione delle chiavi stesse proibitivo. La proprietà chiave è che **le chiavi non vengono memorizzate esplicitamente**: solo un “fingerprint” è conservato.

Struttura. Un vettore di bit $B[0, m-1]$ (inizialmente tutto a 0) e r funzioni hash universali $h_i : U \rightarrow \{0, \dots, m-1\}$ per $i = 1, \dots, r$.

Inserimento di $k \in S$: porre $B[h_i(k)] = 1$ per tutti $i = 1, \dots, r$. Costo: $O(r)$.

Ricerca di y : rispondere “ $y \in S$ ” se e solo se $B[h_i(y)] = 1$ per tutti i . Costo: $O(r)$.

Cancellazione: *non supportata* (richiederebbe un contatore per bit, non un bit solo).

Correttezza: Se $y \in S$, tutti i bit di y sono stati impostati a 1: la risposta è sempre corretta (nessun *false negative*). Se $y \notin S$, potrebbe accadere che tutti i bit $B[h_i(y)]$ siano stati impostati a 1 da altre chiavi: *false positive*.

Probabilità di false positive. La probabilità che un bit rimanga a 0 dopo l'inserimento di tutte le n chiavi è $p_0 \approx e^{-rn/m}$. La probabilità di false positive è:

$$\mathcal{P}(\text{false positive}) \approx (1 - e^{-rn/m})^r.$$

Ottimizzando rispetto a r (a m/n fissato): il valore ottimale è $r = (m/n) \ln 2$, che dà:

$$\mathcal{P}(\text{false positive}) \approx \left(\frac{1}{2}\right)^r = 2^{-(m/n) \ln 2}.$$

Questa probabilità decresce *esponenzialmente* con m/n : con soli 10 bit per chiave ($m/n = 10$) e $r = 7$ funzioni hash, la probabilità di errore è circa 0,8%.

Osservazione 8.15 (Bloom filter mantra). “Ovunque si usi una lista o un insieme e lo spazio è una risorsa critica, un Bloom filter dovrebbe essere considerato, tenendo conto dei possibili effetti dei false positive.”

8.7 Exercises

1. Dimostrare il Teorema 8.4: la famiglia $\mathcal{H} = \{h_a : a \in [1, |U| - 1]\}$ con $h_a(k) = \sum_{i=0}^{r-1} a_i k_i \bmod m$ è universale. In particolare, mostrare che per ogni coppia $x \neq y$, esiste al più un a_0 che causa la collisione.
2. (Esame 2023) Descrivere la struttura dello schema di hash perfetto a due livelli. Provare che lo spazio totale delle tabelle secondarie è $O(n)$ in attesa.
3. Analizzare il cuckoo hashing con il *cuckoo graph*: mostrare che una collisione durante l'inserimento corrisponde a un ciclo nel grafo. Calcolare la probabilità di tale ciclo in funzione di m e n .
4. Calcolare la probabilità ottimale di false positive di un Bloom filter con m bit e n chiavi: trovare il valore ottimale di r minimizzando $(1 - e^{-rn/m})^r$, e ricavare la formula $2^{-(m/n) \ln 2}$.
5. Un Bloom filter con $m = 1\,000\,000$ bit memorizza $n = 100\,000$ chiavi. Quante funzioni hash r sono ottimali? Calcolare la probabilità di false positive.
6. Spiegare perché il cuckoo hashing garantisce ricerca in $O(1)$ nel caso peggiore mentre l'hashing con chaining garantisce solo $O(1)$ in attesa. Quali sono i trade-off?

Chapter 9

Prefix Search

Most discoveries even today are a combination of serendipity and of searching.

— Siddhartha Mukherjee

Il problema della *prefix search* è fondamentale per i motori di ricerca moderni (si pensi all'auto-completamento di Google/Bing) e per molti database testuali.

Problema 9.1 (Prefix-Search Problem). Dato un dizionario $\mathcal{D}$ di n stringhe, con lunghezza totale N e alfabeto di dimensione σ , preprocessare $\mathcal{D}$ per rispondere efficientemente a query del tipo: dato un pattern P di lunghezza p , trovare (o contare) le stringhe di $\mathcal{D}$ che hanno P come prefisso.

La prefix search si riduce algoritmicamente alla *substring search* (ricerca di sottostringhe): basta cercare P tra tutti i *suffissi* delle stringhe del dizionario. Questa connessione sarà approfondita nel Capitolo 10.

9.1 Array of String Pointers

La soluzione più semplice consiste in un array $A[1, n]$ di puntatori alle stringhe, *ordinati indirettamente* secondo l'ordine lessicografico delle stringhe puntate. La prefix search si riduce a due ricerche lessicografiche (per P e $P\#$, dove $\#$ è un carattere più grande di qualsiasi altro): queste identificano l'intervallo $A[l, r]$ delle stringhe prefissate da P .

Teorema 9.2 (Teorema 9.1). *La prefix search sull'array di puntatori a stringhe richiede $O(p \log n)$ tempo e $O(p \log n/B)$ I/O. Lo spazio totale è $N + (1 + w)n$ byte. Il recupero delle n_{occ} stringhe prefissate da P richiede $\Omega(n_{\text{occ}})$ I/O.*

Il bound $\Omega(n_{\text{occ}})$ nel recupero è dovuto alla *non contiguità* delle stringhe su disco: ogni stringa puntata può richiedere un I/O fault.

9.1.1 Contiguous Allocation and Two-Level Storage

L'idea è memorizzare le stringhe *contiguamente* su disco in ordine lessicografico, suddivise in blocchi di dimensione B . L'array A punta solo al primo elemento di ogni blocco ("sampled strings", $n_B \leq N/B$). La prefix search procede in due fasi:

1. **Fase 1 (Ricerca lessicografica):** cerca la posizione di Q ($= P$ o $P\#$) tra le stringhe campionate $\mathcal{D}_B$, usando ricerca binaria su A . Costo: $O((p/B) \log(N/B))$ I/O.
2. **Fase 2 (Scansione):** scansiona il blocco identificato per trovare le stringhe prefissate. Costo: $O(N_{\text{occ}}/B)$ I/O.

Teorema 9.3 (Teorema 9.2). *Con two-level storage, la prefix search richiede $O((p/B) \log(N/B))$ I/O. Lo spazio è $N + n + n_B w$ byte.*

9.1.2 Front Coding

Il *front coding* è una tecnica di compressione δ : ogni stringa s_i è codificata come la coppia $(\ell_i, \hat{s}_i)$, dove ℓ_i è la lunghezza del prefisso condiviso con la stringa precedente s_{i-1} , e $\hat{s}_i$ è il suffisso rimanente. Questa tecnica è efficace perché stringhe lessicograficamente vicine condividono spesso prefissi lunghi (fino al 70% per dizionari di URL).

Applicando il front coding con bucketing (un blocco per disk page, la prima stringa di ogni blocco non compressa), si ottiene:

Teorema 9.4 (Teorema 9.3). *Con front coding e bucketing (FC_B), la prefix search su $\mathcal{D}$ richiede $O\left(\frac{p}{B} \log \frac{FC_B(\mathcal{D})}{B}\right)$ I/O. Il recupero delle stringhe prefissate richiede $O\left(\frac{FC_B(\mathcal{D}_{\text{occ}})}{B}\right)$ I/O.*

9.2 Interpolation Search

Quando le stringhe del dizionario (o i loro puntatori campionati) possono essere interpretate come interi uniformemente distribuiti, la *interpolation search* offre un miglioramento esponenziale rispetto alla ricerca binaria.

L'idea: dati m interi ordinati $x_1 < \dots < x_m$, per cercare y si suddivide l'intervallo $[x_1, x_m]$ in m bin di ugual dimensione $b = (x_m - x_1 + 1)/m$. Il bin candidato per y è $j = \lfloor (y - x_1)/b \rfloor + 1$. Si effettua poi una ricerca binaria all'interno del bin.

Teorema 9.5 (Teorema 9.5). *L'interpolation search su m interi richiede $O(\log \min(\Delta, m))$ nel caso peggiore, dove $\Delta = \max_i(x_i - x_{i-1}) / \min_i(x_i - x_{i-1})$ è il rapporto tra gap massimo e minimo.*

Lemma 9.6 (Lemma 9.1). *Se gli m interi sono distribuiti uniformemente in $[0, U - 1]$, l'interpolation search richiede $O(\log \log m)$ tempo con alta probabilità.*

Applicando l'interpolation search sulle stringhe campionate del two-level storage, il numero di I/O per la prefix search scende a $O(\frac{p}{B} \log \log \frac{N}{B})$: un miglioramento esponenziale rispetto a $O(\frac{p}{B} \log \frac{N}{B})$.

9.3 Compacted Trie (Patricia Trie)

Il *trie* è un albero multi-via i cui archi sono etichettati con caratteri e i percorsi radice-foglia corrispondono alle stringhe del dizionario. Il *trie non compattato* ha al più N nodi interni (uno per carattere) e n foglie, con spazio $O(N\sigma)$ nel caso peggiore (o $O(N)$ con hash table nei nodi).

Il *trie compattato* (detto anche *Patricia trie*) è ottenuto comprimendo ogni cammino unario in un singolo arco, la cui etichetta è una sottostringa della stringa originale (rappresentata come una tripla $\langle s, i, j \rangle$). Il numero di nodi interni si riduce a $O(n)$ e lo spazio a $O(n)$ (più le stringhe stesse).

Teorema 9.7 (Teorema 9.6). *Il trie non compattato risolve la prefix search in $O(p + n_{\text{occ}})$ tempo e $O(p + n_{\text{occ}}/B)$ I/O. Ha $O(N)$ nodi e occupa $O(N)$ spazio. Supporta anche la ricerca lessicografica in $O(p + \log \sigma)$ tempo.*

Teorema 9.8 (Teorema 9.7). *Il trie compattato risolve la prefix search in $O(p + n_{\text{occ}})$ tempo e $O(p + n_{\text{occ}}/B)$ I/O. Ha $O(n)$ nodi e occupa $O(n)$ spazio (più le stringhe). La ricerca lessicografica richiede $O(p + \log \sigma)$ tempo nel caso peggiore.*

Il *Patricia trie* è una variante del trie compattato in cui le etichette degli archi sono ridotte al solo primo carattere, e i nodi interni contengono la lunghezza della stringa associata. La ricerca nel Patricia trie usa l'algoritmo *blind search*: scende nel trie seguendo solo il primo carattere, poi verifica la stringa candidata con un singolo accesso alla foglia.

Teorema 9.9 (Teorema 9.8 (Blind Search)). *La blind search nel Patricia trie richiede un solo accesso a una stringa del dizionario (1 I/O), indipendentemente dalla lunghezza del pattern P .*

Combinando il Patricia trie (per indicizzare le stringhe campionate $\mathcal{D}_B$) con il front coding a bucketing (per le stringhe su disco), si ottiene una soluzione two-level con prefix search in $O(p)$ tempo + $O(1)$ I/O per la Fase 1, e $O(N_{\text{occ}}/B)$ I/O per la Fase 2.

9.4 Exercises

1. Descrivere in dettaglio la struttura dati two-level storage per la prefix search. Discutere i trade-off tra la dimensione del blocco B , lo spazio della tabella A , e il numero di I/O.
2. Implementare il front coding su un esempio di 6 stringhe ordinate lessicograficamente. Calcolare il risparmio percentuale in spazio rispetto alla memorizzazione non compressa.
3. (Esame 2023) Spiegare l'algoritmo di blind search nel Patricia trie. Perché basta un solo accesso alla stringa del dizionario, indipendentemente dalla lunghezza di P ?
4. Confrontare le tre soluzioni per la prefix search: array di puntatori, trie non compattato, e trie compattato (Patricia trie) in termini di tempo, I/O e spazio. Quale è preferibile per dizionari di URL?
5. Dimostrare il Lemma 9.1: se gli interi sono uniformemente distribuiti in $[0, U - 1]$, il costo dell'interpolation search è $O(\log \log m)$ con alta probabilità. Collegare alla dimensione massima di un bin nel modello balls-into-bins.

Chapter 10

Substring Search

In questo capitolo affrontiamo il problema della *ricerca full-text* (o *substring searching*): dato un testo $T[1, n]$ su un alfabeto Σ di dimensione σ , lo si vuole pre-elaborare in modo da poter successivamente recuperare (o contare) tutte le posizioni in cui un pattern $P[1, p]$ occorre come sottostringa di T .

Problema 10.1 (Substring-search). Dato un testo $T[1, n]$ su alfabeto Σ di dimensione σ , pre-elaborarlo affinché, per ogni pattern $P[1, p]$, si possano determinare efficientemente tutte le posizioni del testo in cui P occorre come sottostringa.

L'approccio banale confronta P con ogni sottostringa di T , impiegando $\mathcal{O}(np)$ tempo nel caso peggiore: troppo lento per collezioni massive di testi soggette a un gran numero di query (banche dati genomiche, motori di ricerca). L'idea chiave è costruire una *struttura dati di indicizzazione* sul testo *prima* che le ricerche inizino, ammortizzando il costo di costruzione sulle ricerche successive.

Le due strutture fondamentali che studieremo sono il **suffix array** (SA) e il **suffix tree** (ST), entrambe basate su array oppure alberi.

10.1 Notation and Terminology

Assumiamo che T termini con un carattere speciale $T[n] = \$$, lessicograficamente più piccolo di ogni altro carattere dell'alfabeto. Questo garantisce che nessun suffisso sia prefisso di un altro. Definiamo il i -esimo suffisso di T come $\text{suff}_i = T[i, n]$.

Osservazione 10.2. Se $P = T[i, i + p - 1]$, allora P occorre alla posizione i nel testo, e possiamo dire che P è un *prefisso* del suffisso suff_i . Pertanto:

$$\text{cercare } P \text{ come sottostringa di } T \iff \text{cercare } P \text{ come prefisso in } \text{SUF}(T),$$

dove $\text{SUF}(T) = \{\text{suff}_1, \text{suff}_2, \dots, \text{suff}_n\}$ è il dizionario di tutti i suffissi del testo.

Dato che i suffissi prefissati da P occorrono contigui nell'ordinamento lessicografico di $SUF(T)$, e la posizione lessicografica di P precede immediatamente il blocco di tali suffissi, il problema si riconduce esattamente alla prefix search studiata nel Capitolo ??.

10.2 The Suffix Array

Definizione 10.3 (Suffix Array). Il *suffix array* di un testo $T[1, n]$ è l'array $SA[1, n]$ di interi che elenca le posizioni iniziali di tutti i suffissi di T in ordine lessicografico:

$$suff_{SA[1]} < suff_{SA[2]} < \cdots < suff_{SA[n]},$$

dove $<$ denota l'ordine lessicografico. SA consiste di n interi nell'intervallo $[1, n]$ e occupa $\Theta(n \log n)$ bit.

Esempio 10.4. Per $T = \text{mississippi\$}$ ($n = 12$), il suffix array è $SA = [12, 11, 8, 5, 2, 1, 10, 9, 7, 4, 6, 3]$ corrispondente ai suffissi ordinati: \$, i\$, ippi\$, issippi\$, ississippi\$, mississippi\$, pi\$, ppi\$, sippi\$, sissippi\$, ssissippi\$.

Un altro concetto fondamentale è il **longest common prefix** (LCP) tra suffissi consecutivi nel suffix array.

Definizione 10.5 (Array LCP). L'array $lcp[1, n-1]$ è definito da $lcp[i] = lcp(suff_{SA[i]}, suff_{SA[i+1]})$, dove $lcp(s_1, s_2)$ denota la lunghezza del più lungo prefisso comune tra le stringhe s_1 e s_2 . Ogni valore è un intero in $[0, n)$.

10.2.1 The Substring Search Problem

Poiché la ricerca di P come sottostringa si riduce a una prefix search sul dizionario $SUF(T)$, essa si può risolvere con una **ricerca binaria** sul suffix array.

Algoritmo 10.1: SUBSTRINGSEARCH(P, T, SA)

```

     $L \leftarrow 1; R \leftarrow n$  while  $L \neq R$  do  $M \leftarrow \lfloor (L+R)/2 \rfloor$  if  $\text{strncmp}(P, suff_{SA[M]}, p) > 0$ 
then  $L \leftarrow M + 1$   $R \leftarrow M$  if  $\text{strncmp}(P, suff_{SA[L]}, p) = 0$  then return  $L$  return
     $-1$ 

```

Lemma 10.6 (Lemma 10.1). Dato il testo $T[1, n]$ e il suo suffix array SA , possiamo contare le occorrenze di un pattern $P[1, p]$ in T in tempo $\mathcal{O}(p \log n)$ e $\mathcal{O}(\log n)$ accessi in memoria. Il recupero delle posizioni delle occorrenze richiede un ulteriore $\mathcal{O}(\text{occ})$ tempo. Lo spazio totale richiesto è $\Theta(n(\log n + \log \sigma))$ bit, dove il primo termine è per il suffix array e il secondo per il testo.

Dimostrazione. L'Algoritmo 10.1 esegue una ricerca binaria standard con $\mathcal{O}(\log n)$ confronti tra stringhe. Ogni confronto richiede $\mathcal{O}(p)$ tempo (si confrontano al più p caratteri). Trovato l'intervallo $SA[i, j]$ dei suffissi prefissati da P , le occorrenze sono $occ = j - i + 1$ e le posizioni sono $SA[i], SA[i + 1], \dots, SA[j]$. $\square$

10.2.2 LCP-Optimized Search

Il costo $\mathcal{O}(p \log n)$ della ricerca base può essere migliorato osservando che, nei confronti successivi della ricerca binaria, molti caratteri di P vengono ri-confrontati inutilmente. L'idea è sfruttare tre array ausiliari per evitare la riscansione:

- L'array $\text{lcp}[1, n - 1]$, che memorizza la lunghezza del più lungo prefisso comune tra suffissi consecutivi $\text{suff}_{SA[i]}$ e $\text{suff}_{SA[i+1]}$.
- Due array $\text{Llcp}[1, n - 1]$ e $\text{Rlcp}[1, n - 1]$, definiti per ogni tripla (L, M, R) che può emergere durante la ricerca binaria: $\text{Llcp}[M] = \text{lcp}(\text{suff}_{SA[L]}, \text{suff}_{SA[M]})$ e $\text{Rlcp}[M] = \text{lcp}(\text{suff}_{SA[M]}, \text{suff}_{SA[R]})$.

Si osserva che il numero totale di triple (L, M, R) è $\Theta(n)$ (corrispondono ai nodi di un albero binario completo), e gli array Llcp e Rlcp possono essere costruiti in tempo $\mathcal{O}(n)$ sfruttando la relazione $\text{lcp}[L, R] = \min\{\text{lcp}[L, M], \text{lcp}[M, R]\}$.

Ad ogni passo della ricerca binaria, si mantengono i valori $l = \text{lcp}(P, \text{suff}_{SA[L]})$ e $r = \text{lcp}(P, \text{suff}_{SA[R]})$, che indicano quanti caratteri P condivide con i suffissi agli estremi del range corrente. Supponendo $l \geq r$ (caso simmetrico se $l < r$), si distinguono tre casi:

1. Se $l < \text{Llcp}[M]$: P condivide meno caratteri con $\text{suff}_{SA[L]}$ di quanti ne condividano $\text{suff}_{SA[L]}$ e $\text{suff}_{SA[M]}$, dunque $P > \text{suff}_{SA[M]}$ e la ricerca prosegue nel sotto-array $SA[M, R]$ senza alcun confronto di caratteri.
2. Se $l > \text{Llcp}[M]$: analogamente, $P < \text{suff}_{SA[M]}$ e la ricerca prosegue in $SA[L, M]$ senza confronti.
3. Se $l = \text{Llcp}[M]$: P e $\text{suff}_{SA[M]}$ condividono l caratteri, quindi il confronto può iniziare dalla posizione $l + 1$ anziché dall'inizio.

Lemma 10.7 (Lemma 10.2). *Dati i tre array lcp , Llcp e Rlcp , in aggiunta al suffix array SA costruito su $T[1, n]$, possiamo contare le occorrenze di $P[1, p]$ nel testo T in tempo $\mathcal{O}(p + \log n)$ nel caso peggiore. Il recupero delle posizioni richiede un ulteriore $\mathcal{O}(occ)$ tempo. Lo spazio totale è $\mathcal{O}(n)$ e tutti i bound sono ottimali nel modello comparison-based.*

Dimostrazione. Ad ogni passo della ricerca binaria, o si avanza nella comparazione di un carattere di P (può accadere al più p volte in totale), oppure si dimezza il range senza confronto di caratteri (al più $\mathcal{O}(\log n)$ volte). Le due ricerche lessicografiche (per P e per $P\#$) delimitano l'intervallo $SA[i, j]$, e $occ = j - i + 1$. $\square$

10.2.3 The LCP Array and Its Construction

La costruzione naive dell'array $\text{lcp}[1, n - 1]$ richiederebbe di confrontare carattere per carattere le $n - 1$ coppie di suffissi consecutivi in SA , con costo totale $\Theta(\sum_{i=1}^{n-1} (\text{lcp}[i] + 1))$, che in media è $\mathcal{O}(n\bar{\ell})$ dove $\bar{\ell}$ è la media dei valori LCP, ma nel caso peggiore può essere $\Theta(n^2)$ (ad esempio su $T = a^n$).

Algoritmo di Kasai. Nel 2001, Kasai e collaboratori proposero un elegante algoritmo lineare per calcolare l'array lcp , dato il suffix array SA (e il suo inverso SA^{-1}). L'idea chiave è *evitare la riscansione dei caratteri* del testo, sfruttando la seguente proprietà:

Fatto 10.1. Il LCP tra i suffissi consecutivi $SA[y - 1]$ e $SA[y]$ è non minore del LCP tra $SA[y]$ e qualsiasi altro suffisso precedente $SA[x]$, per $x = 1, \dots, y - 1$.

Fatto 10.2. Se $\text{lcp}(\text{suff}_{SA[p-1]}, \text{suff}_{SA[p]}) > 0$, allora:

$$\text{lcp}(\text{suff}_{SA[p-1]+1}, \text{suff}_{SA[p]+1}) = \text{lcp}(\text{suff}_{SA[p-1]}, \text{suff}_{SA[p]}) - 1.$$

Combinando i Fatti 10.1 e 10.2, si ottiene la proprietà chiave utilizzata dall'algoritmo di Kasai: $\text{lcp}[q - 1] \geq \max\{\text{lcp}[p - 1] - 1, 0\}$, dove $q = SA^{-1}[i]$ e $p = SA^{-1}[i - 1]$. L'algoritmo scansiona i suffissi del testo *da sinistra a destra* (nell'ordine testuale $T[1, n], T[2, n], \dots$), e per ciascuno calcola il valore lcp corrispondente partendo dall'offset dell'iterazione precedente.

Algoritmo 10.2: LCP-BUILD(T, SA)

```

     $h \leftarrow 0$  for  $i = 1$  to  $n$  do  $q \leftarrow SA^{-1}[i]$  if  $q > 1$  then  $k \leftarrow SA[q - 1]$  if  $h > 0$ 
then  $h \leftarrow h - 1$  while  $T[k + h] = T[i + h]$  do  $h \leftarrow h + 1$   $\text{lcp}[q - 1] \leftarrow h$ 

```

Teorema 10.8 (Teorema 10.1). *Dato un testo $T[1, n]$ e il suo suffix array SA_T , l'algoritmo di Kasai (Algoritmo 10.2) calcola l'array lcp in tempo e spazio $\mathcal{O}(n)$. Nel modello a due livelli di memoria, questo algoritmo può essere inefficiente e richiedere $\mathcal{O}(n)$ I/O.*

Dimostrazione. La variabile h viene decrementata al più una volta per iterazione del ciclo `for` (linea 7), e incrementata nella linea 9. Poiché ci sono n iterazioni e h non può mai superare n né scendere sotto 0, il numero totale di incrementi è al più $2n$. Quindi il tempo totale è $\mathcal{O}(n)$. L'algoritmo non è I/O-efficiente perché accede all'array `lcp` in ordine arbitrario (determinato da SA^{-1}). $\square$

10.2.4 Suffix Array Construction

Approccio comparison-based. L'approccio più semplice per costruire SA è ordinare i suffissi con un algoritmo basato su confronti. Si inizializzano i puntatori ai suffissi e si usa un sorter come QuickSort con una funzione di confronto che calcola l'ordine lessicografico.

Teorema 10.9 (Teorema 10.2). *Nel caso peggiore, l'uso di un sorter basato su confronti per costruire il suffix array di $T[1, n]$ richiede $\mathcal{O}((n/B)n \log n)$ I/O e $\mathcal{O}(n \log n)$ bit di spazio di lavoro.*

I due problemi principali sono: (i) ciascuno degli $\mathcal{O}(n \log n)$ confronti coinvolge stringhe di lunghezza variabile fino a $\Theta(n)$, e (ii) l'ordinamento permuta *puntatori* ai suffissi, il che non preserva la località e genera molti I/O.

10.2.5 The Skew Algorithm (DC3)

Nel 2003, Kärkkäinen e Sanders mostrarono che la costruzione del suffix array può essere *ridotta* al problema di ordinare un insieme di triplette le cui componenti sono interi in $[1, \mathcal{O}(n)]$. Poiché questa riduzione richiede tempo e spazio lineare, e sappiamo ordinare n interi in tempo lineare nel modello RAM, otteniamo una costruzione del suffix array in tempo $\mathcal{O}(n)$.

L'algoritmo si chiama **DC3** (Difference Cover modulo 3) o **Skew** per la suddivisione asimmetrica $\frac{2}{3} : \frac{1}{3}$ (invece della più naturale $\frac{1}{2} : \frac{1}{2}$), che semplifica il passo di merge.

Sia $T[1, n] = t_1 t_2 \dots t_n$ con caratteri da un alfabeto intero di dimensione $\sigma = \mathcal{O}(n)$, e $t_n = \$$. L'algoritmo si articola in tre passi:

Passo 1: Costruire $SA^{2,0}$. Si costruisce il suffix array limitato ai suffissi che iniziano in posizioni i tali che $i \bmod 3 \in \{0, 2\}$. Si definisce $P_{2,0} = \{i : i \bmod 3 = 2 \text{ oppure } i \bmod 3 = 0\}$.

1. Si costruisce una stringa speciale $T^{2,0}$ di lunghezza $(2/3)n$ che codifica compattamente tutti i suffissi di T che iniziano nelle posizioni $P_{2,0}$, utilizzando triplette di caratteri consecutivi.
2. Si costruisce *ricorsivamente* il suffix array SA' di $T^{2,0}$.

3. Si deriva $SA^{2,0}$ da SA' .

Passo 2: Costruire SA^1 . Si costruisce il suffix array dei suffissi rimanenti, quelli che iniziano nelle posizioni $P_1 = \{i : i \bmod 3 = 1\}$.

1. Si assume di aver precomputato l'array $\text{pos}[j]$ che fornisce la posizione del j -esimo suffisso $T[j, n]$ in $SA^{2,0}$.
2. Per ogni $i \in P_1$, si rappresenta il suffisso $T[i, n]$ con la coppia $\langle T[i], \text{pos}[i + 1] \rangle$ (notiamo che $i + 1 \in P_{2,0}$).
3. Si ordina con RadixSort sulle $\mathcal{O}(n)$ coppie.

Passo 3: Fondere $SA^{2,0}$ e SA^1 . Si fondono i due suffix array parziali in un unico SA completo, con un merge lineare. La chiave è che il confronto tra un suffisso di P_1 e uno di $P_{2,0}$ può essere effettuato in tempo $\mathcal{O}(1)$ grazie alle posizioni relative (i “rank”) già calcolati nel Passo 1:

- Se $i \in P_1$ e j con $j \bmod 3 = 0$: si confrontano $\langle T[i], T[i + 1], \text{pos}[i + 2] \rangle$ con $\langle T[j], T[j + 1], \text{pos}[j + 2] \rangle$.
- Se $i \in P_1$ e j con $j \bmod 3 = 2$: si confrontano $\langle T[i], \text{pos}[i + 1] \rangle$ con $\langle T[j], \text{pos}[j + 1] \rangle$.

La ricorsione ha profondità $\mathcal{O}(\log_{3/2} n)$ e il lavoro ad ogni livello è $\mathcal{O}(n)$, dato che la dimensione del sottoproblema è $(2/3)n$. La complessità totale è quindi:

$$T(n) = T(2n/3) + \mathcal{O}(n) = \mathcal{O}(n).$$

Osservazione 10.10. L'algoritmo DC3 funziona in *qualsiasi* modello di computazione che disponga di un ordinamento efficiente. In particolare, nel modello a due livelli di memoria, l'I/O-complessità della costruzione del suffix array diventa quella dell'ordinamento di $\mathcal{O}(n)$ interi, ovvero $\mathcal{O}(\text{sort}(n))$ I/O, dove $\text{sort}(n) = \frac{n}{B} \log_{M/B} \frac{n}{M}$.

10.3 The Suffix Tree

Il *suffix tree* di T è un trie compattato costruito sull'insieme $SUF(T)$ di tutti i suffissi di T . Grazie al carattere terminale \$, ogni suffisso corrisponde esattamente a una foglia. Il suffix tree ha le seguenti proprietà:

- Ha esattamente n foglie, una per ogni suffisso.
- Ogni nodo interno (eccetto la radice) ha almeno 2 figli.

- Ogni arco è etichettato con una sottostringa non vuota di T , e le etichette degli archi uscenti da uno stesso nodo iniziano con caratteri distinti.
- La concatenazione delle etichette dalla radice a una foglia i è esattamente suffix_i .

Spazio. Il suffix tree ha al più $n - 1$ nodi interni (oltre alla radice) e n foglie. Ogni arco può essere rappresentato dalla coppia (i, j) di posizioni nel testo che ne definiscono l'etichetta $T[i, j]$. Lo spazio totale è quindi $\mathcal{O}(n)$.

Relazione con il Suffix Array. Una visita in-ordine (DFS) del suffix tree produce i suffissi nell'ordine lessicografico, ovvero genera il suffix array. Viceversa, il suffix tree può essere costruito a partire dal suffix array e dall'array LCP in tempo $\mathcal{O}(n)$ con un algoritmo bottom-up basato su stack.

Applicazioni del Suffix Tree. Oltre alla ricerca per sottostringa, il suffix tree supporta numerose operazioni avanzate: trovare la più lunga sottostringa ripetuta, la più lunga sottostringa comune a due testi, e varie statistiche sulle sottostringhe. La visita dell'albero permette di risolvere in tempo lineare molti problemi su stringhe che sarebbero altrimenti costosi.

Nota 10.11. Il suffix tree è una struttura estremamente potente ma richiede in pratica 10–20 byte per carattere del testo, rendendolo proibitivo per testi molto grandi. Il suffix array, con i suoi 4–8 byte per carattere (a seconda della dimensione degli interi), è più pratico e per molte applicazioni offre prestazioni comparabili.

10.4 Exercises

1. Costruire manualmente il suffix array e l'array LCP per il testo $T = \text{banana\$}$. Mostrare ogni passo dell'Algoritmo 10.2 (Kasai) su questo esempio.
2. Utilizzando il suffix array di $T = \text{abracadabra\$}$, eseguire la ricerca binaria (Algoritmo 10.1) per il pattern $P = \text{abra}$. Quanti confronti di caratteri sono necessari? Come migliora con l'uso degli array $Llcp$ e $Rlcp$?
3. (Esame 2022) Spiegare nel dettaglio l'algoritmo DC3/Skew per la costruzione del suffix array. Perché la suddivisione $\frac{2}{3} : \frac{1}{3}$ è preferibile a $\frac{1}{2} : \frac{1}{2}$? Analizzare la complessità della ricorrenza $T(n) = T(2n/3) + \mathcal{O}(n)$.
4. Dimostrare il Fatto 10.2: se $\text{lcp}(\text{suffix}_{SA[p-1]}, \text{suffix}_{SA[p]}) > 0$, allora $\text{lcp}(\text{suffix}_{SA[p-1]+1}, \text{suffix}_{SA[p]+1}) = \text{lcp}(\text{suffix}_{SA[p-1]}, \text{suffix}_{SA[p]}) - 1$. Spiegare come questa proprietà garantisce la linearità dell'algoritmo di Kasai.

5. Descrivere come si può costruire il suffix tree a partire dal suffix array e dall'array LCP in tempo $\mathcal{O}(n)$, utilizzando uno stack. Confrontare le prestazioni pratiche (spazio e tempo) di suffix array e suffix tree per la ricerca di sottostringhe.
6. Dato il suffix array, progettare un algoritmo per trovare la *più lunga sottostringa ripetuta* in T in tempo $\mathcal{O}(n)$. Suggerimento: qual è il legame con il valore massimo nell'array LCP?

Chapter 11

Integer Coding

In questo capitolo affrontiamo un problema di codifica fondamentale che compare in moltissimi contesti applicativi e il cui impatto sull'occupazione di memoria e sulle prestazioni del sistema è spesso sottovalutato.

Problema 11.1. Sia $S = s_1, s_2, \dots, s_n$ una sequenza di interi positivi, eventualmente ripetuti. L'obiettivo è rappresentare gli interi di S come sequenze binarie *auto-delimitanti* (self-delimiting) e il più corte possibile.

La richiesta di auto-delimitazione è cruciale: le codifiche binarie dei singoli interi devono poter essere concatenate in un unico flusso di bit dal quale il decodificatore possa ricostruire univocamente la sequenza originale, identificando dove finisce un intero e inizia il successivo. Un codice con questa proprietà è detto *prefix-free*.

Motivazioni e applicazioni. Le codifiche di interi compaiono in almeno tre scenari fondamentali:

1. **Posting list nei motori di ricerca.** Per ogni termine t il motore di ricerca memorizza la lista dei documenti che lo contengono (*posting list*), rappresentata come sequenza di docID interi. Per ridurre spazio si ordinano i docID e si codificano i *gap* (differenze tra docID consecutivi) con un codice per interi piccoli (*gap coding*).
2. **Fase finale dei compressor.** Compressor come LZ77, BWT, MTF producono sequenze intermedie di interi con valori piccoli molto probabili. La fase finale converte questi interi in un flusso di bit minimizzando il numero totale di bit.
3. **Compressione di testi come sequenze di token.** Ogni token (parola o carattere) può essere associato a un *tokenID* intero assegnato in ordine decrescente di frequenza. In testi linguistici la frequenza del t -esimo token più frequente segue una distribuzione di Zipf: $\mathcal{P}[t] \approx c/t^\alpha$.

Il problema della codifica a lunghezza variabile. La soluzione più semplice è la codifica a lunghezza fissa: se $m = \max_j s_j$, ogni intero richiede $\lceil \log_2(m+1) \rceil$ bit. Questo è efficiente solo quando gli interi sono uniformemente distribuiti. In generale, se $m \gg s_i$ per la maggior parte degli elementi, molti bit vengono sprecati.

Si potrebbe pensare di usare direttamente la rappresentazione binaria di ogni s_i con $\lceil \log_2(s_i+1) \rceil$ bit, ma questa codifica *non* è auto-delimitante: concatenando le rappresentazioni, il decodificatore non sa dove finisce un intero e inizia il successivo. Ad esempio, la sequenza $S = \{1, 2, 3\}$ produce il flusso 11011, che ammette molteplici decomposizioni.

La domanda centrale è quindi: come progettare una rappresentazione binaria a lunghezza variabile per interi (illimitati) che sia il più corta possibile, prefix-free, e concatenabile?

Lunghezza ideale di un codice. Dal teorema di Shannon, la lunghezza ideale del codice per un simbolo c è $L(c) = \log_2 \frac{1}{\mathcal{P}[c]}$ bit, dove $\mathcal{P}[c]$ è la probabilità di c . Per la codifica di interi, il “simbolo” è l’intero x , e la lunghezza ideale è $\log_2 \frac{1}{\mathcal{P}[x]}$ bit.

11.1 Unary Code

Il codice più semplice e più noto è il *codice unario*. Per un intero $x \geq 1$, il codice unario $U(x)$ è definito come:

$$U(x) = \underbrace{00 \cdots 0}_{x-1 \text{ zeri}} 1$$

ovvero una sequenza di $x-1$ bit a 0 seguita da un bit delimitatore a 1. È chiaro che $|U(x)| = x$ bit, il che rende il codice *esponenzialmente più lungo* della rappresentazione binaria $B(x)$ di lunghezza $\Theta(\log x)$. Il codice unario è quindi efficiente solo per interi molto piccoli.

Teorema 11.2 (Ottimalità del codice unario). *Il codice unario di un intero positivo x richiede x bit, ed è ottimale per la distribuzione $\mathcal{P}[x] = 2^{-x}$.*

Dimostrazione. Dal criterio di Shannon, il codice ideale per x con distribuzione $\mathcal{P}[x] = 2^{-x}$ ha lunghezza $\log_2 \frac{1}{2^{-x}} = x$ bit, che è esattamente $|U(x)|$. $\square$

Teorema 11.3 (Codifica a lunghezza fissa). *Dato un insieme S di interi con valore massimo m , la codifica a lunghezza fissa usa $\lceil \log_2(m+1) \rceil$ bit per intero, ed è ottimale per la distribuzione uniforme $\mathcal{P}[x] = 1/m$.*

11.2 Codici di Elias: γ e δ

I codici γ e δ sono due codici *universali* per interi, introdotti negli anni '60 da Elias. L'aggettivo “universale” indica che la lunghezza del codice è $O(\log x)$ per qualsiasi intero x , cioè solo un fattore costante più lunga della rappresentazione binaria $B(x)$ di lunghezza $\lceil \log_2(x+1) \rceil$, con la proprietà aggiuntiva di essere prefix-free.

11.2.1 Il codice γ (gamma)

Il γ -codice rappresenta un intero x come una sequenza binaria composta da due parti:

1. Una sequenza di $|B(x)| - 1$ zeri (codifica unaria della lunghezza di $B(x)$).
2. La rappresentazione binaria $B(x)$ di x .

La sequenza iniziale di zeri è delimitata dal primo 1, che coincide con il primo bit di $B(x)$; i due componenti condividono quindi questo bit. La decodifica è semplice: si contano gli zeri consecutivi fino al primo 1 (sia c il loro numero); poi si leggono i successivi $c + 1$ bit (incluso l'1 delimitatore) e si interpreta questa sequenza come l'intero x .

Esempio 11.4. Per $x = 9$: $B(9) = 1001$, $|B(9)| = 4$, quindi $\gamma(9) = \underbrace{000}_{3 \text{ zeri}} \underbrace{1001}_{B(9)} = 0001001$, per un totale di 7 bit.

Il numero di bit del γ -codice è $|\gamma(x)| = 2|B(x)| - 1 = 2\lceil \log_2(x+1) \rceil - 1$.

Teorema 11.5 (γ -codice). *Il γ -codice di un intero positivo x richiede $2\lceil \log_2(x+1) \rceil - 1$ bit, ed è ottimale per la distribuzione $\mathcal{P}[x] \approx 1/x^2$. Questo è entro un fattore 2 dalla lunghezza $|B(x)| = \lceil \log_2(x+1) \rceil$ del codice binario di x .*

Dimostrazione. La lunghezza è $2\lceil \log_2(x+1) \rceil - 1$, che si ottiene direttamente dalla costruzione. Dal criterio di Shannon, ponendo $|U(\ell)| = \ell = |B(x)|$ e risolvendo $|U(\ell)| + \ell - 1 = \log_2 \frac{1}{\mathcal{P}[x]}$ rispetto a $\mathcal{P}[x]$, si ottiene $\mathcal{P}[x] \approx 1/x^2$. $\square$

11.2.2 Il codice δ (delta)

L'inefficienza del γ -codice risiede nella codifica unaria di $|B(x)|$, che diventa costosa al crescere di x . Il codice δ mitiga questo problema applicando il γ -codice alla lunghezza $|B(x)|$ anziché il codice unario. Il δ -codice è quindi composto da due parti:

1. $\gamma(|B(x)|)$: il γ -codice della lunghezza della rappresentazione binaria.
2. $B(x)$: la rappresentazione binaria di x (le due parti non condividono bit con la prima).

Poiché $|B(x)| \geq 1$ e il γ -codice si applica a $|B(x)|$, la prima e la seconda parte non condividono bit. La decodifica è: si decodifica il γ -codice iniziale ottenendo $\ell = |B(x)|$; poi si leggono i successivi ℓ bit e si interpreta la sequenza come x .

Esempio 11.6. Per $x = 14$: $B(14) = 1110$, $\ell = |B(14)| = 4$, $\ell' = |B(\ell)| = |B(4)| = 3$. Quindi $\delta(14) = \underbrace{00}_{U(\ell')} \underbrace{1}_{B(\ell)} \underbrace{001110}_{B(14)}$, per un totale di 9 bit.

Teorema 11.7 (δ -codice). *Il δ -codice di un intero positivo x richiede circa $1 + \log_2 x + 2 \log_2 \log_2 x$ bit, ed è ottimale per la distribuzione $\mathcal{P}[x] \approx \frac{1}{x(\log x)^2}$. Questo è entro un fattore $1 + o(1)$ dalla lunghezza $|B(x)|$ del codice binario.*

Il δ -codice è quindi universale ed è preferibile al γ -codice per interi grandi. Entrambi i codici sono tuttavia lenti da decodificare per via delle numerose operazioni di bit-shift. Nella pratica si preferiscono spesso codici allineati ai byte.

11.3 Rice Code

Il codice di Rice è un codice *parametrico* che dipende da un intero positivo k . È particolarmente efficiente quando gli interi sono concentrati attorno a un valore diverso da zero, situazione in cui γ e δ perdono efficacia.

Per un intero $x > 0$ e parametro k , il codice di Rice $R_k(x)$ separa x in due componenti:

- Il *quoziente* $q = \lfloor (x-1)/2^k \rfloor$, codificato in unario con $q+1$ bit.
- Il *resto* $r = x - 1 - 2^k \cdot q$, codificato in binario con esattamente k bit.

Il quoziente è in lunghezza variabile, il resto in lunghezza fissa. Più 2^k è vicino al valore tipico degli interi in S , più il quoziente è piccolo e quindi la codifica è compatta.

Esempio 11.8. Per $x = 83$ e $k = 4$: $q = \lfloor 82/16 \rfloor = 5$, $r = 82 - 16 \cdot 5 = 2$. Quindi $R_4(83) = U(6) \cdot B_4(2) = 000001\ 0010$, per un totale di 10 bit.

La lunghezza totale del codice è $|R_k(x)| = q + k + 1$ bit. Il codice di Rice è un caso particolare del codice di Golomb, ed è ottimale per la distribuzione geometrica $\mathcal{P}[x] = p(1-p)^{x-1}$, con la scelta $2^k \simeq \frac{\ln 2}{p} \simeq 0.69 \times \text{media}(S)$.

11.4 PForDelta

PForDelta è un metodo di codifica orientato alla velocità di decompressione, particolarmente efficace quando gli interi di S seguono approssimativamente una distribuzione gaussiana concentrata in un intervallo $[base, base + 2^b - 2]$.

L'idea è la seguente:

- Si sceglie un valore b tale che circa il 90% degli interi di S cada nell'intervallo $[base, base + 2^b - 2]$. Questi interi vengono codificati con esattamente b bit.
- Gli interi fuori intervallo (*eccezioni*) vengono segnalati con un *simbolo di escape* (la configurazione $2^b - 1$, riservata) e memorizzati separatamente usando w bit (una parola macchina intera).

Ogni intero di S è quindi codificato in lunghezza fissa: o b bit (se nell'intervallo) oppure $b + w$ bit (se eccezione), dove w è la dimensione della parola macchina. Questa proprietà rende la decodifica estremamente rapida, perché si possono processare w/b interi in parallelo con operazioni su word.

11.5 Variable-Byte Code and (s, c) -Dense Codes

Il *variable-byte code* è un codice allineato ai byte, introdotto per il motore AltaVista. La codifica di un intero x è costruita come segue:

1. Si prende la rappresentazione binaria $B(x)$ e la si estende (padding di zeri a sinistra) affinché la lunghezza sia multipla di 7.
2. Si partiziona in blocchi da 7 bit ciascuno.
3. A ciascun blocco si antepone un *flag bit*: 0 se è l'ultimo byte (stopper), 1 altrimenti (continuer).

La decodifica scorre i byte finché trova uno stopper (flag = 0), poi rimuove i flag bit e interpreta la sequenza binaria risultante come l'intero. Il numero minimo di bit è 8, quindi il codice è adatto per interi grandi.

Il variable-byte code partiziona le $2^8 = 256$ configurazioni di ogni byte in $s = 128$ stopper e $c = 128$ continuer. Questa scelta non è necessariamente ottimale: i codici (s, c) -densi generalizzano l'idea permettendo di scegliere $s + c = 2^b$ (con b non necessariamente 8) e di variare la ripartizione tra stopper e continuer.

Con un codice (s, c) -denso:

- Un byte codifica i primi s interi.
- Due byte codificano i successivi $s \times c$ interi.
- k byte codificano fino a $s \times \frac{c^k - 1}{c - 1}$ interi totali.

La scelta ottimale di s e c dipende dalla distribuzione degli interi da codificare.

11.6 Interpolative Code

Il codice interpolativo è una tecnica di codifica per sequenze *crescenti* di interi positivi. A differenza dei codici precedenti, è un codice *adattivo*: la codifica di un intero s'_i dipende dalla distribuzione degli altri interi nella sequenza. Inoltre, il codice interpolativo *non* è prefix-free.

Data una sequenza $S = s_1, \dots, s_n$ (eventualmente con ripetizioni), si trasforma in una sequenza crescente $S' = s'_1, \dots, s'_n$ ponendo $S'[i] = \sum_{j=1}^i S[j]$ (somme prefisse), così che $s'_1 < s'_2 < \dots < s'_n$.

Il codice interpolativo è particolarmente efficiente quando la sequenza S' presenta *cluster* di valori vicini tra loro, situazione tipica nelle posting list dei motori di ricerca.

Schema ricorsivo. L'algoritmo procede ricorsivamente sulla sottosequenza $S'_{l,r} = (s'_l, s'_{l+1}, \dots, s'_r)$ conoscendo quattro parametri: gli indici l e r , e i bound low e hi tali che $low \leq s'_l < \dots < s'_r \leq hi$.

Ad ogni chiamata ricorsiva:

1. Si calcola l'indice mediano $m = \lfloor (l + r)/2 \rfloor$.
2. Si codifica s'_m usando $\lceil \log_2 B \rceil$ bit, dove $B = hi - low - r + l + 1$ è la dimensione dell'intervallo dei valori possibili per s'_m . L'intero codificato è la differenza $s'_m - (low + m - l)$, cioè lo scarto rispetto al lower bound di s'_m .
3. Si procede ricorsivamente sulle due sottosequenze $(s'_l, \dots, s'_{m-1})$ e $(s'_{m+1}, \dots, s'_r)$ con bound aggiornati.

Osservazione 11.9. Quando la sottosequenza ha la forma $(low, low + 1, \dots, low + n - 1)$, l'intervallo B vale 1 per ogni elemento e l'algoritmo non emette alcun bit, ottenendo un vantaggio di compressione significativo per cluster densi.

Il codice interpolativo è un codice adattivo e non prefix-free, proprietà che lo rendono molto diverso dai codici universali. Sorprendentemente, per sequenze dense il codice interpolativo può risultare *più corto* del codice di Huffman (che è ottimo tra i codici statici prefix-free).

11.7 Elias–Fano Encoding

La codifica di Elias–Fano è una delle rappresentazioni più importanti per sequenze monotone crescenti di interi, perché offre simultaneamente buona compressione e accesso random efficiente (tramite opportune strutture dati compresse di supporto).

Sia $S' = s'_1 \leq s'_2 \leq \dots \leq s'_n$ una sequenza monotona non decrescente di interi in $[0, u)$, dove u è l'universo. Si pone $\ell = \lfloor \log_2(u/n) \rfloor$. Ogni intero s'_i viene separato in due parti:

- I *bit bassi* (*lower bits*): gli ℓ bit meno significativi di s'_i , memorizzati esplicitamente in un array L di $n \cdot \ell$ bit totali.
- I *bit alti* (*upper bits*): i restanti $\lceil \log_2 u \rceil - \ell$ bit più significativi, memorizzati in una sequenza H di bit usando una codifica unaria.

Per i bit alti, si definisce $h_i = \lfloor s'_i / 2^\ell \rfloor$ (la parte alta di s'_i). La sequenza H è una stringa binaria di lunghezza $n + u/2^\ell \leq n + 2n = 3n$ bit, costruita come segue: per ogni valore $v = 0, 1, \dots$, si scrivono tanti bit 0 quanti sono gli s'_i con $h_i = v$, seguiti da un bit 1 (separatore). In pratica, H contiene esattamente n zeri (uno per ogni elemento) e circa $u/2^\ell$ uni (uno per ogni “bucket”).

Spazio totale. L’occupazione di spazio è:

$$n \cdot \ell + n + \frac{u}{2^\ell} \leq n \cdot \lceil \log_2(u/n) \rceil + 2n + \frac{u}{2^\ell} \approx n \log_2(u/n) + 2n \text{ bit.}$$

Questa quantità è notevolmente vicina all’ottimo informazione-teoretico: il numero minimo di bit necessari per rappresentare una sequenza monotona di n interi in $[0, u)$ è $\log_2 \binom{u}{n} \approx n \log_2(u/n) + n \log_2 e$ bit (per $n \ll u$). La codifica di Elias–Fano usa quindi solo circa $2n$ bit in più rispetto all’ottimo.

Accesso random. Il grande vantaggio di Elias–Fano rispetto al codice interpolativo è la possibilità di accedere al j -esimo elemento in tempo $\mathcal{O}(1)$ usando strutture dati di supporto per le operazioni **Select** sulla sequenza H dei bit alti, che verranno approfondite nel Capitolo 15. In particolare:

- **Access**(j): restituisce s'_j combinando i bit alti (ottenuti con una **Select** su H) e i bit bassi (letti direttamente da L).
- **NextGEQ**(x): trova il più piccolo $s'_j \geq x$ sfruttando la struttura dei bit alti.

Teorema 11.10 (Codifica di Elias–Fano). *Data una sequenza monotona non decrescente di n interi in $[0, u)$, la codifica di Elias–Fano occupa al più $n \lceil \log_2(u/n) \rceil + 2n$ bit, e supporta l’accesso al j -esimo elemento in tempo $\mathcal{O}(1)$ con strutture dati ausiliarie di $o(n)$ bit.*

Osservazione 11.11. A differenza del codice interpolativo, la codifica di Elias–Fano non dipende dalla distribuzione degli interi all’interno dell’universo: lo spazio è garantito indipendentemente dalla “clusterizzazione” della sequenza. Per sequenze molto dense (cluster stretti), il codice interpolativo può essere più compatto; per sequenze sparse con necessità di accesso random, Elias–Fano è preferibile.

11.8 Summary and Comparison

Riassumiamo le principali codifiche di interi e le rispettive proprietà:

Codice	Bit per intero x	Distribuzione ottimale	Prefix-free
Unario $U(x)$	x	$\mathcal{P}[x] = 2^{-x}$	Sì
$\gamma(x)$	$2\lceil \log_2(x+1) \rceil - 1$	$\mathcal{P}[x] \approx 1/x^2$	Sì
$\delta(x)$	$\approx \log_2 x + 2\log_2 \log_2 x$	$\mathcal{P}[x] \approx \frac{1}{x(\log x)^2}$	Sì
Rice $R_k(x)$	$\lfloor (x-1)/2^k \rfloor + k + 1$	geometrica	Sì
Variable-byte	$8\lceil B(x) /7 \rceil$	$\mathcal{P}[x] \approx 1/\sqrt[8]{x}$	Sì
Interpolativo	variabile (adattivo)	cluster densi	No
Elias–Fano	$\approx \log_2(u/n) + 2$ per intero	qualsiasi	No

11.9 Exercises

1. Calcolare la codifica γ , δ e unaria degli interi $x = 1, 5, 13, 42, 100$. Per ciascuno, indicare il numero totale di bit e verificare che la decodifica sia corretta.
2. Dimostrare che il γ -codice è prefix-free, cioè che nessun codeword è prefisso di un altro. Suggerimento: mostrare che la lunghezza del codeword è univocamente determinata dai primi bit.
3. (Esame 2023) Spiegare la struttura della codifica di Elias–Fano per una sequenza monotona di n interi in $[0, u)$. Calcolare lo spazio occupato e confrontarlo con il lower bound informazione-teoretico $\log_2 \binom{u}{n}$.
4. Data la sequenza crescente $S' = (3, 7, 11, 14, 19, 25)$ con universo $u = 32$, costruire manualmente la codifica di Elias–Fano calcolando ℓ , l'array dei bit bassi L e la sequenza dei bit alti H .
5. Illustrare il funzionamento del codice interpolativo sulla sequenza crescente $S' = (1, 2, 3, 5, 7, 9, 11, 15, 18, 19, 20, 21)$. Mostrare l'albero delle chiamate ricorsive e i bit emessi ad ogni passo.
6. Confrontare la codifica di Rice con il γ -codice per la sequenza $S = (3, 5, 4, 6, 3, 7, 4, 5)$, scegliendo il parametro k ottimale per Rice. Quale codice produce meno bit totali?

Chapter 12

Statistical Coding

In questo capitolo affrontiamo il problema della *codifica statistica* di una sequenza S di simboli tratti da un alfabeto Σ . A differenza del capitolo precedente, qui i simboli non sono necessariamente interi: possono essere caratteri (compressione di testo), basi del DNA, o qualunque elemento discreto.

La compressione statistica si articola concettualmente in due fasi:

- **Fase di modellazione** (*modeling*): si calcolano le proprietà statistiche della sequenza di input e si costruisce un *modello* probabilistico.
- **Fase di codifica** (*coding*): il modello viene usato per assegnare codeword ai simboli di Σ e comprimere la sequenza.

In questo capitolo ci concentreremo principalmente sulla fase di codifica, esaminando i tre compressor statistici più importanti: la codifica di Huffman, la codifica aritmetica e la codifica range. Nella parte finale introdurremo il modello PPM (*prediction by partial matching*), che combina una fase di modellazione sofisticata con la codifica aritmetica.

Entropia. L'*entropia di ordine 0* di una sorgente che emette simboli da un alfabeto Σ con probabilità $\mathcal{P}[\sigma]$ è definita come:

$$\mathcal{H} = \sum_{\sigma \in \Sigma} \mathcal{P}[\sigma] \log_2 \frac{1}{\mathcal{P}[\sigma]}.$$

L'entropia rappresenta il numero medio minimo di bit per simbolo necessari a codificare l'output della sorgente. L'entropia è massima ($\mathcal{H} = \log_2 |\Sigma|$) quando i simboli sono equiprobabili, ed è nulla quando un solo simbolo ha probabilità 1.

12.1 Huffman Coding

La codifica di Huffman, pubblicata nei primi anni '50, è stata per decenni il metodo di riferimento per la compressione dati, fino all'avvento della codifica aritmetica.

12.1.1 Huffman Tree Construction

L'algoritmo di Huffman è uno schema *greedy* che costruisce un albero binario le cui foglie sono i simboli $\sigma \in \Sigma$, ciascuno dotato della probabilità $\mathcal{P}[\sigma]$ di occorrenza nella sequenza da comprimere.

1. Si inizializza un *candidate set* contenente tutte le foglie (simboli con le rispettive probabilità).
2. Ad ogni passo, si selezionano i due nodi con probabilità minima dal *candidate set*.
3. Si crea un nodo padre la cui probabilità è la somma delle probabilità dei due figli. Il padre viene inserito nel *candidate set*, mentre i due figli vengono rimossi.
4. Si ripete fino a quando il *candidate set* contiene un unico nodo (la radice).

L'albero finale ha $|\Sigma|$ foglie e $|\Sigma| - 1$ nodi interni, per un totale di $2|\Sigma| - 1$ nodi. Per derivare il codice, si assegnano etichette binarie (0 e 1) ai due archi uscenti da ogni nodo interno. Il codeword di un simbolo σ è la concatenazione delle etichette lungo il cammino dalla radice alla foglia σ , e la sua lunghezza è $L(\sigma) = \text{profondità della foglia}$. Il codice risultante è *prefix-free*.

File compresso. Il file compresso è composto da due parti: il *preambolo* (contenente l'albero di Huffman o le probabilità dei simboli, di dimensione $\Theta(|\Sigma|)$) e il *body* (contenente i codeword dei simboli di S).

12.1.2 Optimality of Huffman Coding

Sia $L_C = \sum_{\sigma \in \Sigma} L(\sigma) \times \mathcal{P}[\sigma]$ la lunghezza media del codeword prodotta da un codice prefix-free C .

Lemma 12.1. *Sia $\mathcal{F}$ un insieme di alberi binari pesati con $|\Sigma|$ foglie e profondità media minima. Allora esiste un albero $T \in \mathcal{F}$ in cui le due foglie con probabilità minima sono alla profondità massima e sono figlie dello stesso nodo.*

Lemma 12.2. *Sia T un albero binario pesato e R il suo albero ridotto ottenuto eliminando le due foglie x, y con probabilità minima e sostituendole con il loro padre z avente probabilità $\mathcal{P}[z] = \mathcal{P}[x] + \mathcal{P}[y]$. Allora $L_T = L_R + \mathcal{P}[x] + \mathcal{P}[y]$.*

Dimostrazione. Sommando le lunghezze pesate di tutti i cammini radice-foglia in T e in R , la differenza è esattamente il contributo aggiuntivo delle foglie x e y rispetto al loro padre z : ciascuna contribuisce con un livello in più, pesato dalla rispettiva probabilità. $\square$

Teorema 12.3 (Ottimalità di Huffman). *Se C è un codice di Huffman, allora L_C è la più breve lunghezza media possibile tra tutti i codici prefix-free C' , ovvero $L_C \leq L_{C'}$.*

Dimostrazione. Per induzione sul numero di simboli $n = |\Sigma|$. *Base:* $n = 2$: qualsiasi codice prefix-free assegna almeno 1 bit a ciascun simbolo, e Huffman assegna esattamente 1 bit a ciascuno. *Passo:* sia $n > 2$; si assuma che Huffman sia ottimale per $n - 1$ simboli. Sia C un codice ottimale per Σ e H il codice di Huffman. Per il Lemma 12.1, esiste un codice ottimale in cui le due foglie con probabilità minima (x, y) sono figlie dello stesso nodo. Si considerano gli alberi ridotti R_C e R_H , ottenuti eliminando x e y e sostituendoli con il padre z . Per il Lemma 12.2, $L_H = L_{R_H} + \mathcal{P}[x] + \mathcal{P}[y]$ e $L_C = L_{R_C} + \mathcal{P}[x] + \mathcal{P}[y]$. Per ipotesi induttiva $L_{R_H} \leq L_{R_C}$, quindi $L_H \leq L_C$. $\square$

Teorema 12.4 (Relazione con l'entropia). *Sia $\mathcal{H}$ l'entropia della sorgente. La lunghezza media L_H del codice di Huffman soddisfa:*

$$\mathcal{H} \leq L_H < \mathcal{H} + 1.$$

Dimostrazione. La disuguaglianza $\mathcal{H} \leq L_H$ discende dal teorema di codifica senza rumore di Shannon. Per la disuguaglianza superiore, si definisce $\ell_\sigma = \lceil \log_2(1/\mathcal{P}(\sigma)) \rceil$, che è la più piccola lunghezza intera che soddisfa la condizione di ottimalità di Shannon. Poiché $\sum_\sigma 2^{-\ell_\sigma} \leq 1$ (disuguaglianza di Kraft), esiste un codice prefix-free C con queste lunghezze, la cui lunghezza media è $L_C = \sum_\sigma \mathcal{P}(\sigma) \times \ell_\sigma < \mathcal{H} + 1$. Per l'ottimalità di Huffman, $L_H \leq L_C < \mathcal{H} + 1$. $\square$

Osservazione 12.5. Il Teorema 12.4 mostra che il codice di Huffman può perdere fino a 1 bit per simbolo rispetto all'entropia. Per distribuzioni sbilanciate (*skewed*, $\mathcal{H} \approx 0$), questo extra-bit è significativo. Per distribuzioni quasi uniformi ($\mathcal{H} \gg 1$), la perdita è trascurabile. Il miglior *compression ratio* ottenibile da Huffman partendo dalla rappresentazione binaria a $\log_2 |\Sigma|$ bit è $1/\log_2 |\Sigma|$: per ASCII ($|\Sigma| = 256$) non si scende sotto 12.5%.

12.1.3 Canonical Huffman

L'albero di Huffman classico presenta due limitazioni pratiche: richiede di memorizzare la struttura dell'albero (costoso per alfabeti grandi) e la decodifica è lenta (ogni bit richiede una discesa nell'albero con possibili cache miss).

L'*Huffman canonico* risolve entrambi i problemi ristrutturando l'albero in modo che le profondità delle foglie non decrescano da sinistra a destra. La costruzione si basa su cinque passi:

1. Si calcolano le lunghezze $L(\sigma)$ per ogni $\sigma \in \Sigma$ con l'algoritmo classico.
2. Si costruisce l'array **symb**[ℓ]: lista dei simboli con codeword di lunghezza ℓ .
3. Si costruisce l'array **num**[ℓ]: numero di simboli con codeword di lunghezza ℓ .
4. Si calcola l'array **fc**[ℓ]: il primo codeword di lunghezza ℓ bit, tramite operazioni aritmetiche (senza usare l'albero).
5. Si assegnano implicitamente codeword consecutivi ai simboli in **symb**[ℓ].

Lo spazio richiesto è solo $\max^2$ bit per **fc** e $(|\Sigma| + \max) \log_2(|\Sigma| + 1)$ bit per **symb** (dove $\max$ è la lunghezza massima del codeword). La decodifica è estremamente rapida e richiede al più 2 cache miss per simbolo decodificato, operando solo sugli array **fc** e **symb**.

12.2 Arithmetic Coding

La codifica aritmetica supera la limitazione fondamentale di Huffman: Huffman assegna un numero *intero* di bit per simbolo ($L(\sigma) \geq 1$), mentre la codifica aritmetica può usare un numero *frazionario* di bit per simbolo, avvicinandosi arbitrariamente all'entropia.

Idea di base. L'intera sequenza $S = s_1 s_2 \dots s_n$ viene codificata come un singolo numero reale in $[0, 1)$. L'intervallo $[0, 1)$ viene partizionato in sotto-intervalli proporzionali alle probabilità dei simboli. Ad ogni simbolo letto, l'intervallo corrente viene ristretto al sotto-intervallo corrispondente.

Procedura di codifica. Si mantiene un intervallo $[lo, hi)$ inizializzato a $[0, 1)$. Si fissa un ordine sui simboli $\sigma_1, \dots, \sigma_{|\Sigma|}$ e si definiscono le probabilità cumulate $F(\sigma_i) = \sum_{j < i} \mathcal{P}[\sigma_j]$. Per ogni simbolo s_k letto:

$$\begin{cases} range = hi - lo, \\ hi \leftarrow lo + range \times (F(s_k) + \mathcal{P}[s_k]), \\ lo \leftarrow lo + range \times F(s_k). \end{cases}$$

Dopo aver processato tutti i simboli, si emette un numero binario nell'intervallo finale $[lo, hi)$ usando $\lceil \log_2(1/(hi - lo)) \rceil + 1$ bit.

Teorema 12.6 (Compressione della codifica aritmetica). *La codifica aritmetica di una sequenza S di n simboli produce al più $n\mathcal{H} + 2$ bit, dove $\mathcal{H}$ è l'entropia di ordine 0 della sorgente.*

Sketch. La dimensione dell'intervallo finale è $\prod_{k=1}^n \mathcal{P}[s_k]$. Il numero di bit necessari per identificare un punto in questo intervallo è $\lceil -\log_2 \prod_{k=1}^n \mathcal{P}[s_k] \rceil + 1 = \lceil \sum_{k=1}^n \log_2 \frac{1}{\mathcal{P}[s_k]} \rceil + 1$. Mediando su tutte le possibili sequenze, il costo medio per simbolo è circa $\mathcal{H}$, con un overhead costante di al più 2 bit totali. $\square$

Implementazione a precisione finita. L'uso di numeri reali a precisione arbitraria è impraticabile. Le implementazioni reali operano con interi a 32 o 64 bit, emettendo bit non appena *lo* e *hi* condividono i bit più significativi (*bit-stuffing* o *renormalization*). Questo rende la codifica e decodifica incrementali e in tempo $\mathcal{O}(n)$.

Confronto con Huffman.

- Huffman è ottimale tra i codici *prefix-free statici*; la codifica aritmetica è quasi ottimale tra *tutti* i codici.
- Huffman perde fino a 1 bit/simbolo rispetto all'entropia; la codifica aritmetica perde solo $\mathcal{O}(1/n)$ bit/simbolo.
- Huffman è più veloce in decodifica (specialmente nella versione canonica); la codifica aritmetica è più lenta per via delle moltiplicazioni.
- Per distribuzioni molto sbilanciate ($\mathcal{H} \ll 1$), la codifica aritmetica è significativamente migliore.

12.3 Higher-Order Entropy and PPM

L'entropia di ordine 0 assume che i simboli siano indipendenti. In pratica, i testi naturali mostrano forti dipendenze tra simboli consecutivi. L'*entropia di ordine k* cattura queste dipendenze condizionando la probabilità di ogni simbolo ai k simboli precedenti (*contesto*):

$$\mathcal{H}_k = \sum_{c \in \Sigma^k} \frac{|c|_S}{|S|} \sum_{\sigma \in \Sigma} \mathcal{P}[\sigma \mid c] \log_2 \frac{1}{\mathcal{P}[\sigma \mid c]},$$

dove la somma è su tutti i contesti c di lunghezza k e $|c|_S$ conta le occorrenze di c in S .

Il modello PPM. Il *Prediction by Partial Matching* (PPM) è una tecnica di modellazione che stima la probabilità del prossimo simbolo basandosi sui k simboli precedenti, usando un meccanismo di *escape* per gestire i contesti mai visti: se il contesto di ordine k non è stato osservato, si emette un simbolo di escape e si “scende” al contesto di ordine $k - 1$, e così via fino all’ordine 0. Il modello viene poi accoppiato con un codificatore aritmetico.

PPM raggiunge tassi di compressione molto vicini all’entropia di ordine k , ed è considerato uno dei migliori compressor statistici per testo naturale.

12.4 Exercises

1. Costruire l’albero di Huffman per l’alfabeto $\Sigma = \{a, b, c, d, e, f\}$ con probabilità $\mathcal{P}[a] = 0.05$, $\mathcal{P}[b] = 0.10$, $\mathcal{P}[c] = 0.15$, $\mathcal{P}[d] = 0.30$, $\mathcal{P}[e] = 0.25$, $\mathcal{P}[f] = 0.15$. Calcolare la lunghezza media del codeword e confrontarla con l’entropia.
2. Dimostrare per induzione il Teorema 12.3 (ottimalità di Huffman) in tutti i dettagli, utilizzando i Lemmi 12.1 e 12.2.
3. Trasformare l’albero di Huffman dell’esercizio 1 nella sua forma canonica. Calcolare gli array `symb`, `num` e `fc`. Tracciare l’esecuzione dell’algoritmo di decodifica canonica sulla sequenza di bit 011001000001.
4. (Esame 2021) Spiegare perché il codice di Huffman non può raggiungere un compression ratio migliore di $1/\log_2 |\Sigma|$. Illustrare come lo schema di *blocking* di Shannon (raggruppare k simboli alla volta) migliora questo limite, e discutere il trade-off con la dimensione dell’albero.
5. Eseguire la codifica aritmetica della stringa $S = \text{abbc}$ con $\mathcal{P}[a] = 0.5$, $\mathcal{P}[b] = 0.3$, $\mathcal{P}[c] = 0.2$. Mostrare l’evoluzione dell’intervallo $[lo, hi)$ ad ogni passo e determinare il codeword binario finale.
6. Confrontare la codifica di Huffman e la codifica aritmetica su una sorgente binaria con $\mathcal{P}[0] = 0.99$ e $\mathcal{P}[1] = 0.01$. Calcolare l’entropia e la lunghezza media dei codeword per entrambi i metodi. Perché Huffman è particolarmente inefficiente in questo caso?

Chapter 13

Dictionary-Based Compression

In questo capitolo affrontiamo un approccio alla compressione dati radicalmente diverso da quello statistico: anziché stimare le probabilità dei simboli, i compressori *basati su dizionario* derivano un *dizionario di stringhe* dalla sequenza di input S e sostituiscono ogni occorrenza di una stringa già vista con un riferimento alla sua posizione precedente o con un identificatore nel dizionario. Il dizionario è costruito *dinamicamente*, partendo da vuoto, e cresce man mano che l'input viene processato.

A partire dal 1977, Abraham Lempel e Jacob Ziv introdussero una famiglia di compressori che costituiscono la base dei più diffusi strumenti di compressione: `gzip`, `7zip`, `Brotli`, `LZ4`, `LZMA` e `ZSTD`.

13.1 LZ77

LZ77 utilizza una *sliding window* $W[1, w]$ che contiene una porzione della sequenza già processata, e un *look-ahead buffer* B che contiene il suffisso ancora da comprimere. Il compressore opera in due fasi: *parsing* (decomposizione in frasi) ed *encoding* (codifica delle frasi in bit).

Parsing. LZ77 cerca il più lungo prefisso α di B che occorre come sottostringa di $W \cdot B$. Se α è trovato a distanza d dall'inizio di B (cioè nella finestra), con lunghezza $|\alpha|$ e carattere successivo $c = B[|\alpha| + 1]$, allora LZ77 emette la *trippla* $\langle d, |\alpha|, c \rangle$. La finestra W avanza di $|\alpha| + 1$ posizioni.

Se non viene trovato alcun match ($|\alpha| = 0$), viene emessa la tripla $\langle 0, 0, B[1] \rangle$.

Esempio 13.1. Comprimere $S = \text{aabbababbaababaabbbaa\$}$ con LZ77 e finestra illimitata:

$W \mid B$	Output
aabbababab...	$\langle 0, 0, a \rangle$
a abbababab...	$\langle 1, 1, b \rangle$
aab bababab...	$\langle 1, 1, a \rangle$
aabba babab...	$\langle 2, 3, EOF \rangle$

Variante LZss. Una variante popolare, dovuta a Storer e Szymanski (1982), emette *coppie* anziché triple: $\langle d, |\alpha| \rangle$ per un match, oppure $\langle 0, c \rangle$ per un singolo carattere letterale. Questa variante elimina la ridondanza della tripla quando $|\alpha| = 0$.

13.1.1 Implementation: gzip

Il problema algoritmico chiave nell'implementazione di LZ77 è la *ricerca veloce* del più lungo prefisso α di B che occorre in W . L'approccio di **gzip** utilizza una *hash table* che indicizza tutti i 3-grammi (triplette di caratteri consecutivi) in $W \cdot B[1, 2]$.

Per ogni 3-gramma, la hash table memorizza la lista delle posizioni in W dove occorre. Quando si cerca α :

1. Si cerca il 3-gramma $B[1, 3]$ nella hash table. Se non presente, si emette la coppia $\langle 0, B[1] \rangle$ e si avanza di 1.
2. Altrimenti, si scorre la lista $\mathcal{L}$ delle occorrenze e si confronta simbolo per simbolo $S[i, n]$ con B per trovare il match più lungo.
3. Si emette la coppia $\langle p - i^*, |\alpha| \rangle$, dove p è la posizione corrente e i^* la posizione del match migliore.

Per velocizzare la ricerca, **gzip** ordina le occorrenze dalla più recente alla più vecchia e si ferma dopo aver esaminato un numero sufficiente di candidati. Le opzioni da -1 a -9 controllano la dimensione della finestra W (da 100 KB a 900 KB circa), realizzando un trade-off tra rapporto di compressione e velocità.

Encoding. **gzip** codifica le frasi con due alberi di Huffman: uno per le lunghezze dei match (e i caratteri letterali) e uno per le distanze.

Decompressione. La decompressione è estremamente rapida: ogni coppia $\langle d, \ell \rangle$ corrisponde a una copia di memoria di ℓ byte a distanza d , eseguibile con una semplice iterazione:

```
for i = 0 to L-1 do { S[s+i] = S[s-d+i]; }
s = s + L;
```

La velocità di decompressione è inversamente proporzionale al numero di frasi: finestre più grandi producono meno frasi e quindi decompressione più veloce.

13.2 LZ78

LZ78 (1978) adotta un approccio diverso: anziché usare una sliding window, costruisce *esplicitamente* un dizionario $\mathcal{D}$ che contiene un sottoinsieme delle sottostringhe di S .

Parsing e costruzione del dizionario. Il dizionario è inizialmente vuoto (contiene solo la stringa vuota con $\text{id} = 0$). Ad ogni passo, LZ78:

1. Cerca il più lungo prefisso α di B che è una frase nel dizionario $\mathcal{D}$.
2. Sia $c = B[|\alpha| + 1]$ il carattere successivo. Emette la coppia $\langle \text{id}(\alpha), c \rangle$.
3. Aggiunge la nuova frase αc al dizionario con il prossimo identificatore disponibile.

Il dizionario è *prefix-complete*: contiene tutti i prefissi di tutte le sue frasi. Questo consente di implementarlo con un *trie non compattato*, dove ogni arco è etichettato con un singolo simbolo. La ricerca del più lungo prefisso corrisponde a una discesa nel trie.

Esempio 13.2. Parsing LZ78 di $S = \text{aabbababbbbaababaa}$:

Input	Output	Dizionario
a	$\langle 0, a \rangle$	1 : a
ab	$\langle 1, b \rangle$	2 : ab
b	$\langle 0, b \rangle$	3 : b
aba	$\langle 2, a \rangle$	4 : aba
bb	$\langle 3, b \rangle$	5 : bb
ba	$\langle 3, a \rangle$	6 : ba
abab	$\langle 4, b \rangle$	7 : abab
aa	$\langle 1, a \rangle$	8 : aa

Gestione dei dizionari grandi. Il dizionario cresce con la lunghezza dell'input. Quando raggiunge una dimensione massima, si possono adottare diverse strategie: (i) congelarlo; (ii) svuotarlo e ricominciare; (iii) eliminare le frasi meno recentemente utilizzate (LRU).

13.3 LZW

LZW (Welch, 1984) è una variante popolare di LZ78 con un'ottimizzazione chiave: elimina il secondo componente della coppia, emettendo solo l'id della frase. Per rendere ciò possibile:

- Il dizionario è inizializzato con tutte le stringhe di un singolo simbolo ($\text{id} = 0, \dots, |\Sigma| - 1$).

- Si cerca il più lungo prefisso α di B che è nel dizionario. Sia $c = B[|\alpha| + 1]$.
- Si emette $\text{id}(\alpha)$ (un solo intero, non una coppia).
- Si aggiunge αc al dizionario con il prossimo id disponibile.
- La prossima frase da cercare inizia da c (non da $B[|\alpha| + 1 + 1]$): parsing e aggiornamento del dizionario sono *disallineati*.

Decodifica. Il disallineamento rende la decodifica più complessa. Il decodificatore ricostruisce il dizionario in modo incrementale, ma la frase corrente potrebbe fare riferimento a una frase non ancora completa nel dizionario. Questo caso speciale ($\alpha'' = f$ dove f è la frase appena inserita) si risolve osservando che $f[1] = \alpha'[1]$: il primo carattere della frase mancante è sempre il primo carattere della frase precedente.

Esempio 13.3. LZW encoding di $S = \text{aabbabab}$ (ASCII: $a = 97$, $b = 98$):

Input	Output	Dizionario
a	97	256 : aa
a	97	257 : ab
b	98	258 : bb
b	98	259 : ba
ab	257	260 : aba
aba	260	261 : abab

Lo stream compresso è: 97, 97, 98, 98, 257, 260.

LZW è alla base del formato di immagine GIF.

13.4 LZ77 vs. LZ78 Comparison

Proprietà	LZ77	LZ78/LZW
Dizionario	implicito (sliding window)	esplicito (trie)
Dimensione dizionario	limitata da $ W $	cresce con l'input
Output per frase	coppia $\langle d, \ell \rangle$	coppia $\langle \text{id}, c \rangle$ o solo id
Compressione	in genere migliore	leggermente peggiore
Velocità compressione	più lenta (ricerca in W)	più veloce (lookup in trie)
Velocità decompressione	molto veloce	veloce
Strumenti	gzip, 7zip, ZSTD, Brotli	compress, GIF

Entrambe le famiglie sono compressor *universali*: al crescere della lunghezza dell'input, il loro rapporto di compressione converge all'entropia della sorgente (sotto ipotesi di ergodicità e stazionarietà). Per file testuali tipici, raggiungono un rapporto di compressione di circa un terzo.

13.5 Exercises

1. Eseguire il parsing LZ77 (variante LZss) della stringa $S = \text{abababababab\$}$ con finestra illimitata. Quante coppie vengono emesse? Discutere il caso in cui il match si estende dal buffer nella finestra (*overlapping copy*).
2. Eseguire il parsing LZ78 e la costruzione del dizionario (trie) per la stringa $S = \text{aababcbabcbabc\$}$. Disegnare il trie risultante.
3. (Esame 2024) Descrivere le differenze tra LZ77 e LZ78 in termini di struttura del dizionario, schema di codifica e prestazioni. In quali scenari un approccio è preferibile all'altro?
4. Descrivere come `gzip` implementa la ricerca del match più lungo in LZ77 usando una hash table dei 3-grammi. Qual è il trade-off tra dimensione della finestra e velocità/qualità della compressione?
5. Eseguire la codifica e decodifica LZW sulla stringa $S = \text{ababcbababc}$ con alfabeto ASCII. Mostrare lo stato del dizionario ad ogni passo e lo stream di `id` emessi. Illustrare il caso speciale nella decodifica quando il decodificatore incontra un `id` non ancora presente nel dizionario.
6. Spiegare perché i compressori Lempel–Ziv sono detti “universali”. Quale relazione esiste tra il rapporto di compressione di LZ77/LZ78 e l'entropia della sorgente?

Chapter 14

Block-Sorting Compression

I compressori basati su dizionario (Capitolo ??) e quelli statistici (Capitolo 12) affrontano il problema della compressione con strategie complementari. Nel 1994, Mike Burrows e David Wheeler proposero un approccio radicalmente diverso: una *trasformazione reversibile* della stringa di input che ne riorganizza i simboli in modo da rendere facile la compressione successiva, senza perdere informazione. Questa tecnica, nota come *Burrows–Wheeler Transform* (BWT), è alla base del compressore **bzip2** e ha aperto la strada a una nuova classe di strutture dati compresse (come l’FM-index, trattato nel capitolo successivo).

L’idea chiave è che la BWT produce una permutazione dei simboli dell’input che risulta *localmente omogenea*: lunghe sequenze di simboli uguali o simili tendono a raggrupparsi, e tali sequenze sono facilmente comprimibili con tecniche semplici come *Move-to-Front* (MTF) e *Run-Length Encoding* (RLE). Il tutto viene infine codificato con un encoder statistico (Huffman o aritmetico). La pipeline completa è:

$$S \xrightarrow{\text{BWT}} L \xrightarrow{\text{MTF}} L^{\text{MTF}} \xrightarrow{\text{RLE}} \text{coppie} \xrightarrow{\text{Huffman/Aritmetico}} \text{bitstream compresso}$$

14.1 The Burrows–Wheeler Transform

La BWT non è un algoritmo di compressione in senso stretto: non riduce la dimensione dell’input. È una *permutazione* (trasformazione lossless) dei simboli, concepita affinché la stringa risultante sia particolarmente adatta alla compressione tramite algoritmi semplici come MTF + RLE. La BWT consiste di due trasformazioni inverse: una *forward transform* che riorganizza i simboli, e una *backward transform* che ricostruisce la stringa originale a partire dall’output della prima.

14.1.1 Forward Transform

Sia $S[1, n]$ una stringa su un alfabeto ordinato Σ . Si appende a S un simbolo speciale $\$$ che non appartiene a Σ ed è definito come il più piccolo nell'ordinamento totale di $\Sigma \cup \{\$\}$.

La trasformata diretta procede come segue:

1. Costruire la stringa $S\$$.
2. Considerare la matrice *concettuale* $\mathcal{M}$ di dimensione $(n + 1) \times (n + 1)$, le cui righe sono tutti gli *shift ciclici sinistri* di $S\$$. La matrice $\mathcal{M}$ è detta *matrice delle rotazioni* di S .
3. Ordinare le righe di $\mathcal{M}$ lessicograficamente (da sinistra a destra), rispetto all'ordinamento su $\Sigma \cup \{\$\}$. La matrice ordinata è chiamata $\mathcal{M}'$.
4. L'output della BWT è la coppia $\text{BWT}(S) = (\hat{L}, r)$, dove $\hat{L}$ è l'ultima colonna di $\mathcal{M}'$ privata del simbolo $\$$, e r è la posizione di $\$$ nell'ultima colonna.

Nota 14.1. La matrice $\mathcal{M}$ è puramente concettuale: la sua dimensione sarebbe $\Theta(n^2)$, il che la renderebbe impraticabile per input di dimensioni anche moderate. Vedremo che $\mathcal{M}'$ può essere costruita in tempo e spazio $\mathcal{O}(n)$, sfruttando la costruzione del Suffix Array (Capitolo ??).

Esempio 14.2. Consideriamo $S = \text{abracadabra}$ (con $n = 11$). Si costruisce $S\$ = \text{abracadabra\$}$. La matrice delle rotazioni $\mathcal{M}$ ha 12 righe, ciascuna ottenuta da uno shift ciclico sinistro. Dopo l'ordinamento lessicografico si ottiene $\mathcal{M}'$:

Riga	F	Riga di $\mathcal{M}'$	L
1	\$	\$abracadabra	a
2	a	a\$abracadabr	r
3	a	abra\$abracad	d
4	a	abracadabra\$	\$
5	a	acadabra\$abr	r
6	a	adabra\$abrac	c
7	b	bra\$abracada	a
8	b	bracadabra\$a	a
9	c	cadabra\$abra	a
10	d	dabra\$abrac	c
11	r	ra\$abracadab	b
12	r	racadabra\$ab	b

La prima colonna F (First column) contiene i simboli di $S\$$ in ordine lessicografico: $F = \$aaaaabbcdr$. L'ultima colonna è $L = \text{ard}\$rrcaaaabb$, da cui $\widehat{L} = \text{ardrcaaaabb}$ (rimuovendo $\$$) e $r = 4$. Si noti la *proprietà di omogeneità locale*: l'ultima colonna presenta raggruppamenti di simboli uguali ($aaaabb$).

La proprietà di omogeneità locale non è casuale. Le righe di $\mathcal{M}'$ sono ordinate lessicograficamente, il che equivale a ordinare i *suffissi* di $S\$$. Righe consecutive che iniziano con lo stesso contesto (stessa sottostringa iniziale) tendono ad avere lo stesso simbolo *predecessore* in S , che si trova esattamente nell'ultima colonna L . Poiché i testi naturali (e molte sequenze reali) presentano elevata ripetitività nei contesti, L risulta fortemente comprimibile.

14.1.2 Backward Transform

Il cuore dell'invertibilità della BWT risiede in due proprietà fondamentali che collegano la prima colonna F e l'ultima colonna L della matrice ordinata $\mathcal{M}'$.

Definizione 14.3 (Fatto 14.1). Per $1 \leq i \leq n+1$, sia $S[k_i, n]$ il suffisso (eventualmente vuoto) di S che forma il prefisso della riga i -esima di $\mathcal{M}'$. Chiaramente questo suffisso è seguito dal simbolo $\$$, e poi dal prefisso $S[1, k_i - 1]$, a causa dello shift ciclico sinistro.

Teorema 14.4 (Proprietà 1). *Il simbolo $L[i]$ precede il simbolo $F[i]$ nella stringa S , per ogni riga i tranne quella che termina con $\$$, per la quale $F[i] = S[1]$.*

Dimostrazione. Dalla definizione di $\mathcal{M}'$, la riga i -esima è uno shift ciclico di $S\$$. L'ultimo simbolo della riga è $L[i] = S[k_i - 1]$ e il primo è $F[i] = S[k_i]$. Quindi $L[i]$ precede immediatamente $F[i]$ nella stringa originale S . $\square$

Teorema 14.5 (Proprietà 2). *Tutte le occorrenze di uno stesso simbolo c in L mantengono lo stesso ordine relativo che in F . Cioè la k -esima occorrenza di c in L corrisponde alla k -esima occorrenza di c in F .*

Dimostrazione. Siano t e t' due stringhe con $t \prec t'$ (lessicograficamente). Fissiamo un simbolo c che occorre in S . Se c occorre una sola volta, la proprietà è immediata.

Se c occorre almeno due volte, consideriamo due posizioni $F[i]$ e $F[j]$ con $i < j$ entrambe uguali a c . Siano $r(i)$ e $r(j)$ le righe corrispondenti in $\mathcal{M}'$. Allora:

- $r(i) \prec r(j)$ per l'ordinamento di $\mathcal{M}'$ e il fatto che $i < j$.
- Entrambe iniziano con c , quindi $r(i) = c\alpha$ e $r(j) = c\beta$ con $\alpha \prec \beta$.

Ruotando ciclicamente di una posizione a sinistra, otteniamo le righe $r(i') = \alpha c$ e $r(j') = \beta c$, che compaiono in L con il simbolo c in ultima posizione. Poiché $\alpha \prec \beta$, si ha $r(i') \prec r(j')$ nella matrice ordinata, e quindi la posizione di c in L relativa a $r(i')$ precede quella relativa a $r(j')$. L'ordine è preservato. $\square$

Queste due proprietà permettono di definire la funzione chiave per la ricostruzione.

Definizione 14.6 (LF-mapping). $LF[1, n+1]$ è un vettore di $n+1$ interi nell'intervallo $[1, n+1]$ tale che $LF[i] = j$ se e solo se il simbolo $L[i]$ corrisponde al simbolo $F[j]$. In altre parole, se $L[i]$ è la k -esima occorrenza del simbolo c in L , allora $F[LF[i]]$ è la k -esima occorrenza di c in F .

L'LF-mapping si costruisce in tempo $\mathcal{O}(n)$ con il seguente algoritmo:

Algoritmo 14.1: Costruzione dell'LF-mapping dalla colonna L

Input: Colonna $L[1, n+1]$

Output: Vettore $LF[1, n+1]$

// Fase 1: contare le occorrenze di ogni simbolo in L

for $i = 1, \dots, n+1$ **do** $C[L[i]] \leftarrow C[L[i]] + 1$

// Fase 2: somma cumulativa — $C[c]$ diventa il numero di simboli $< c$ in L , più 1

temp $\leftarrow 0$; *sum* $\leftarrow 1$ **for** $i = 1, \dots, |\Sigma| + 1$ **do** *temp* $\leftarrow C[i]$ $C[i] \leftarrow \text{sum}$ *sum* $\leftarrow \text{sum} + \text{temp}$

// Fase 3: assegnare LF scorrendo L

for $i = 1, \dots, n+1$ **do** $LF[i] \leftarrow C[L[i]]$ $C[L[i]] \leftarrow C[L[i]] + 1$

La Fase 1 conta le occorrenze n_c di ogni simbolo c in L . La Fase 2 trasforma questi conteggi in somme cumulative: dopo questa fase, $C[c]$ indica la posizione in F della prima occorrenza di c (ricordando che F è L ordinata). La Fase 3 scorre L da sinistra a destra: ogni volta che incontra $L[i] = c$, assegna $LF[i] = C[c]$ e incrementa $C[c]$, cosicché la prossima occorrenza di c verrà mappata alla posizione successiva in F . La complessità totale è $\mathcal{O}(n)$.

Con l'LF-mapping, la ricostruzione di S è immediata:

Algoritmo 14.2: Ricostruzione di S da $\text{BWT}(S) = (\widehat{L}, r)$

Input: Colonna $L[1, n+1]$ (derivata da $\widehat{L}$ reinserendo $\$$ in posizione r), posizione r

Output: Stringa $S[1, n]$

Calcolare $LF[1, n+1]$ tramite Algoritmo 14.1 $k \leftarrow 1$; $i \leftarrow n$ **while** $i > 0$ **do** $S[i] \leftarrow L[k]$ $k \leftarrow LF[k]$ $i \leftarrow i - 1$

L'algoritmo parte dalla riga 1 di $\mathcal{M}'$ (che corrisponde a $\$S$, cioè la riga che inizia con $\$$) e usa l'LF-mapping per “saltare” attraverso la matrice, ricostruendo S dall'ultimo simbolo al primo. Ad ogni passo, $L[k]$ è il simbolo che precede il contesto corrente in S , e $LF[k]$ ci porta alla riga di $\mathcal{M}'$ il cui contesto inizia con quel simbolo.

Teorema 14.7. *La stringa originale S può essere ricostruita dalla sua BWT in tempo e spazio $\mathcal{O}(n)$. L'Algoritmo 14.2 genera al più un cache miss per simbolo ricostruito.*

Dimostrazione. L'Algoritmo 14.1 esegue tre scansioni lineari, ciascuna in $\mathcal{O}(n)$. L'Algoritmo 14.2 esegue esattamente n iterazioni, ognuna in $\mathcal{O}(1)$. Lo spazio è dominato dai vettori L , LF e C , tutti di dimensione $\mathcal{O}(n)$ (o $\mathcal{O}(|\Sigma|)$ per C). Per quanto riguarda le cache miss, ogni iterazione accede a $L[k]$ e $LF[k]$: se i due array sono memorizzati consecutivamente, ogni accesso a posizione k genera al più un cache miss, poiché $L[k]$ e $LF[k]$ possono risiedere in blocchi diversi. $\square$

Esempio 14.8. Riprendiamo $S = \text{abracadabra}$ con $L = \text{ard\$rrcaaaabb}$ (incluso $\$$). Calcolando l'LF-mapping e partendo dalla riga $k = 1$ (che contiene $\$$ come primo simbolo di F), $L[1] = \text{a}$ è l'ultimo simbolo di S ($S[11] = \text{a}$). Poi $LF[1]$ ci manda alla riga 2, dove $L[2] = \text{r}$ fornisce $S[10] = \text{r}$. Proseguendo: $LF[2]$ porta alla riga 11, dove $L[11] = \text{b}$, cioè $S[9] = \text{b}$. Iterando $n = 11$ volte si ricostruisce $S = \text{abracadabra}$.

14.1.3 Connection with the Suffix Array

La matrice ordinata $\mathcal{M}'$ è strettamente legata al Suffix Array SA della stringa $S\$$. Le righe di $\mathcal{M}'$ ordinate lessicograficamente corrispondono ai suffissi di $S\$$ ordinati, il che è esattamente la definizione di SA . Pertanto:

- La colonna F di $\mathcal{M}'$ è la sequenza dei primi caratteri dei suffissi in ordine di SA , cioè $F[i] = S[SA[i]]$.
- La colonna L soddisfa $L[i] = S[SA[i] - 1]$ per $SA[i] > 1$, e $L[i] = \$$ per $SA[i] = 1$.

Di conseguenza, la BWT può essere calcolata in tempo $\mathcal{O}(n)$ costruendo prima il Suffix Array con un algoritmo lineare (come DC3/Skew, Capitolo ??), e poi derivando L direttamente. Viceversa, la BWT e le sue strutture ausiliarie sono alla base del FM-index, un indice compresso per la ricerca di pattern che esamineremo nel prossimo capitolo.

14.2 Two Auxiliary Transforms: MTF and RLE

La BWT produce una stringa L che esibisce lunghe sequenze di simboli uguali o simili. Per sfruttare questa proprietà, il compressore **bzip2** applica due trasformate intermedie prima della codifica finale.

14.2.1 Move-to-Front (MTF)

La trasformata *Move-to-Front* converte una stringa di simboli in una stringa di interi, sfruttando il principio di *località temporale*: se un simbolo è appena apparso, è probabile che riappaia presto.

Definizione 14.9 (MTF). Si mantiene una lista dinamica $\mathcal{L}^{MTF}$ contenente tutti i simboli dell'alfabeto Σ , inizializzata in ordine lessicografico. Per ogni simbolo $S[i]$ della stringa di input:

1. Si trova la posizione p di $S[i]$ in $\mathcal{L}^{MTF}$ (contando da 1).
2. Si emette p come i -esimo elemento dell'output S^{MTF} .
3. Si sposta $S[i]$ in *testa* a $\mathcal{L}^{MTF}$.

Esempio 14.10. Consideriamo $S = \text{bananacocco}$ con $\Sigma = \{a, b, c, n, o\}$ e indici iniziali $\{1, 2, 3, 4, 5\}$.

Passo	$\sigma = S[i]$	p	$\mathcal{L}^{MTF}$ dopo lo spostamento
1	b	2	$\{b, a, c, n, o\}$
2	a	2	$\{a, b, c, n, o\}$
3	n	4	$\{n, a, b, c, o\}$
4	a	2	$\{a, n, b, c, o\}$
5	n	2	$\{n, a, b, c, o\}$
6	a	2	$\{a, n, b, c, o\}$
7	c	4	$\{c, a, n, b, o\}$
8	o	5	$\{o, c, a, n, b\}$
9	c	2	$\{c, o, a, n, b\}$
10	c	1	$\{c, o, a, n, b\}$
11	o	2	$\{o, c, a, n, b\}$

L'output è $S^{MTF} = (2, 2, 4, 2, 2, 2, 4, 5, 2, 1, 2)$. Si noti l'abbondanza di valori piccoli (1 e 2), che sono facilmente comprimibili.

Quando MTF è applicata alla colonna L della BWT, le sotto-sequenze localmente omogenee di L (lunghe ripetizioni dello stesso simbolo) si trasformano in sequenze di valori molto piccoli (prevalentemente 1, cioè il simbolo è già in testa alla lista). Questo è il punto di forza della combinazione BWT + MTF: si trasformano le regolarità strutturali della BWT in una distribuzione degli interi fortemente sbilanciata verso lo zero, ideale per la compressione.

La trasformata MTF è invertibile: dato S^{MTF} e l'ordine iniziale di $\mathcal{L}^{MTF}$, si può ricostruire la stringa originale ripercorrendo il processo inverso.

14.2.2 Run-Length Encoding (RLE)

La *Run-Length Encoding* (RLE) è una tecnica di compressione elementare che sostituisce ogni *run* (sequenza massimale di simboli consecutivi uguali) con una coppia (σ, ℓ) , dove σ è il simbolo e ℓ è la lunghezza del run.

Esempio 14.11. La stringa `aaabbbccccca` viene codificata come: $(a, 3), (b, 2), (c, 5), (a, 1)$.

RLE è efficace solo quando l'input contiene lunghi run. Applicata direttamente a un testo generico, non produce compressione significativa. Tuttavia, dopo la combinazione BWT + MTF, l'output L^{MTF} contiene tipicamente lunghe sequenze di zeri (il valore 1 nella nostra notazione con indici da 1) o di valori molto piccoli, e RLE risulta estremamente efficace.

Nella pratica, `bzip2` applica RLE in due punti:

1. **Prima della BWT:** un primo passo di RLE sull'input originale per ridurre run molto lunghi.
2. **Dopo MTF:** un secondo passo di RLE sull'output di MTF per compattare le lunghe sequenze di zeri.

14.3 The bzip2 Pipeline

Il compressore `bzip2`, disponibile nella maggior parte dei sistemi operativi, implementa la seguente pipeline:

1. **RLE iniziale:** si codificano i run dell'input originale (opzionale, per gestire casi estremi).
2. **BWT:** si applica la trasformata di Burrows–Wheeler, ottenendo la colonna L e la posizione r .
3. **MTF:** si applica Move-to-Front a L , producendo una sequenza di interi piccoli L^{MTF} .
4. **RLE:** si applica Run-Length Encoding a L^{MTF} , codificando le sequenze di zeri.
5. **Codifica Huffman:** si codifica l'output di RLE con Huffman canonico (o, in alcune varianti, con codifica aritmetica).

L'input viene suddiviso in blocchi di dimensione fissa (tipicamente 900 KB). Ogni blocco viene trasformato e compresso indipendentemente, il che permette accesso quasi-random e limita l'uso di memoria.

Osservazione 14.12. Il compressore **bzip2** raggiunge tipicamente un rapporto di compressione migliore di **gzip** (basato su LZ77) su file testuali, producendo output circa il 20–30% più piccoli. Questo perché la BWT è in grado di catturare correlazioni a lunga distanza che la finestra scorrevole di LZ77 non può sfruttare. In termini formali, la compressione basata su BWT raggiunge la λ -ottimalità per ogni $k \geq 0$, con λ arbitrariamente piccolo: il rapporto di compressione converge a $H_k(S) + \varepsilon$ per $\varepsilon \rightarrow 0$. Questo è un risultato teoricamente più forte di quello ottenibile con LZ78.

Osservazione 14.13. La decompressione di **bzip2** inverte la pipeline: si decodifica Huffman, si inverte RLE, si inverte MTF ottenendo L , e infine si inverte la BWT ricostruendo S . Ogni passo è lineare, quindi la decompressione avviene in tempo $\mathcal{O}(n)$.

14.4 Theoretical Results on BWT

La BWT ha proprietà teoriche notevoli che la distinguono dai compressori LZ:

1. **λ -ottimalità:** una versione modificata di LZ78 combinata con RLE e BWT è 3-ottimale rispetto all'entropia di ordine 0 (H_0), ma è λ -ottimale per ogni $k \geq 1$ con λ arbitrariamente piccolo. Questo significa che per ogni ordine k , il rapporto di compressione della BWT converge all'entropia empirica di ordine k .
2. **Connessione con i Suffix Array:** la costruzione della BWT in tempo lineare è possibile grazie alla stretta relazione con il Suffix Array. Qualsiasi algoritmo che costruisce il SA in $\mathcal{O}(n)$ produce automaticamente la BWT in $\mathcal{O}(n)$.
3. **Invertibilità efficiente:** la BWT è invertibile in $\mathcal{O}(n)$ tempo e spazio, con al più un cache miss per simbolo ricostruito (Teorema 14.7).

14.5 Exercises

1. Calcolare la BWT della stringa $S = \text{mississippi}$. Scrivere la matrice delle rotazioni $\mathcal{M}$, la matrice ordinata $\mathcal{M}'$, le colonne F e L , e la posizione r .
2. Data la BWT $\text{BWT}(S) = (\text{annb\$aa}, 4)$, ricostruire la stringa originale S applicando l'Algoritmo 14.2. Mostrare il calcolo dell'LF-mapping e ogni passo della ricostruzione.
3. Applicare la trasformata MTF alla stringa $L = \text{aaaabbccc}$ con alfabeto $\Sigma = \{a, b, c\}$ e lista iniziale (a, b, c) . Discutere perché l'output contiene molti valori piccoli e come questo favorisca la compressione.
4. (Esame 2023) Descrivere la pipeline completa di **bzip2**: $\text{BWT} \rightarrow \text{MTF} \rightarrow \text{RLE} \rightarrow \text{Huffman}$. Per ciascun passo, spiegare il ruolo nel processo di compressione e la complessità temporale.

5. Dimostrare la Proprietà 2 della BWT: le occorrenze di uno stesso simbolo c mantengono lo stesso ordine relativo in L e in F . Spiegare perché questa proprietà è fondamentale per la costruzione dell'LF-mapping.
6. Spiegare la connessione tra BWT e Suffix Array. Come si può calcolare la BWT in tempo $\mathcal{O}(n)$ sfruttando la costruzione del SA? Mostrare la relazione $L[i] = S[SA[i] - 1]$.

Chapter 15

Compressed Data Structures

Nel capitolo precedente abbiamo introdotto l'FM-index come versione compressa del Suffix Array. In realtà, la letteratura offre oggi strutture dati compresse per array, alberi, grafi e molto altro. In questo capitolo presentiamo i mattoni fondamentali di questo paradigma: le operazioni RANK e SELECT su array binari, la loro implementazione succinta, e l'FM-index come applicazione principe alla ricerca su testi compressi. Un effetto collaterale importante è l'introduzione della *pointer-less programming*: l'idea di rinunciare ai puntatori espliciti e sostituirli con strutture dati compresse su array binari, ottenendo le stesse prestazioni asintotiche con un'occupazione di spazio notevolmente ridotta.

15.1 Compressed Representation of Binary Arrays

Consideriamo un dizionario $\mathcal{D}$ di n stringhe di lunghezza totale m , mappate in una singola stringa $T[1, m]$ (senza separatori). Vogliamo supportare due operazioni:

- $\text{ACCESS_STRING}(i)$: restituire l' i -esima stringa di $\mathcal{D}$.
- $\text{WHICH_STRING}(x)$: dato un indice x in T , restituire l'indice della stringa che contiene $T[x]$.

L'approccio classico usa un array di n puntatori (offset) alle stringhe di $\mathcal{D}$ in T , occupando $\Theta(n \log m)$ bit. Un approccio ortogonale e più compatto usa un *array binario* $B[1, m]$ dove $B[i] = 1$ se e solo se $T[i]$ è il primo simbolo di una stringa del dizionario. Le operazioni si traducono in:

- $\text{ACCESS_STRING}(i) \Rightarrow \text{SELECT}_1(i)$: trovare la posizione dell' i -esimo bit 1 in B .
- $\text{WHICH_STRING}(x) \Rightarrow \text{RANK}_1(x)$: contare quanti bit 1 precedono (o includono) la posizione x .

15.1.1 Rank and Select Definitions

Definizione 15.1 (Rank e Select). Sia $B[1, m]$ un array binario.

- $\text{RANK}_b(i)$: il numero di bit uguali a $b \in \{0, 1\}$ in $B[1, i]$. Formalmente, $\text{RANK}_1(i) = \sum_{j=1}^i B[j]$. Si noti che $\text{RANK}_0(i) = i - \text{RANK}_1(i)$.
- $\text{SELECT}_b(i)$: la posizione dell' i -esima occorrenza del bit b in B . Formalmente, $\text{SELECT}_b(i) = \min\{j : \text{RANK}_b(j) = i\}$.

Esempio 15.2. Consideriamo $B = (0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0)$ di lunghezza $m = 11$.

- $\text{RANK}_1(6) = 2$ (ci sono due 1 in $B[1, 6]$, nelle posizioni 3 e 5).
- $\text{SELECT}_1(3) = 8$ (il terzo 1 si trova nella posizione 8).

15.1.2 Succinct Rank Implementation

L'obiettivo è una struttura dati che supporti RANK_1 in tempo $\mathcal{O}(1)$ usando spazio $o(m)$ bit *in aggiunta* all'array B stesso. Questo tipo di struttura è detta *succinta*: lo spazio addizionale è asintoticamente trascurabile rispetto a B .

La struttura si basa su una decomposizione gerarchica di B in tre livelli:

Livello 1 — Big blocks. Si divide logicamente B in blocchi grandi (*big blocks*) di dimensione $Z = (\log m)^2$ bit ciascuno. Per ogni big block i si memorizza il *rank assoluto* r_i , ovvero il numero totale di 1 in B *prima* dell'inizio del blocco i -esimo:

$$r_i = \text{RANK}_1(Z \cdot (i - 1)).$$

Ogni r_i richiede al più $\log m$ bit, e ci sono m/Z blocchi, quindi lo spazio totale per i rank assoluti è:

$$\frac{m}{Z} \cdot \log m = \frac{m}{\log^2 m} \cdot \log m = \frac{m}{\log m} = o(m) \text{ bit.}$$

Livello 2 — Small blocks. Ogni big block è ulteriormente suddiviso in blocchi piccoli (*small blocks*) di dimensione $z = \frac{1}{2} \log m$ bit. Per ogni small block j all'interno del big block i si memorizza il *rank relativo* $r_{i,j}$, cioè il numero di 1 nel prefisso del big block fino all'inizio dello small block j :

$$r_{i,j} = \text{RANK}_1(Z \cdot (i - 1) + z \cdot (j - 1)) - r_i.$$

Ogni $r_{i,j}$ è limitato da $Z = (\log m)^2$, quindi richiede $\mathcal{O}(\log \log m)$ bit. Il numero totale di small blocks è m/z , per uno spazio totale di:

$$\frac{m}{z} \cdot \mathcal{O}(\log \log m) = \frac{m}{\frac{1}{2} \log m} \cdot \mathcal{O}(\log \log m) = \mathcal{O}\left(\frac{m \log \log m}{\log m}\right) = o(m) \text{ bit.}$$

Livello 3 — Tabella di lookup. Per rispondere alle query all'interno di uno small block, si precalcola una tabella R indicizzata da tutte le possibili configurazioni binarie di z bit e da tutte le posizioni all'interno del blocco. La tabella $R[b, o]$ fornisce il conteggio degli 1 nei primi o bit della configurazione b .

La tabella ha $2^z \cdot z$ entrate, ciascuna di $\mathcal{O}(\log z)$ bit. Poiché $z = \frac{1}{2} \log m$:

$$2^z \cdot z \cdot \mathcal{O}(\log z) = 2^{\frac{1}{2} \log m} \cdot \frac{1}{2} \log m \cdot \mathcal{O}(\log \log m) = \sqrt{m} \cdot \mathcal{O}((\log m)(\log \log m)) = o(m) \text{ bit.}$$

Risposta a una query. Per calcolare $\text{RANK}_1(x)$:

1. Determinare il big block $i = 1 + \lfloor (x-1)/Z \rfloor$ e leggere r_i .
2. Determinare lo small block $j = 1 + \lfloor ((x-1) \bmod Z)/z \rfloor$ e leggere $r_{i,j}$.
3. Determinare l'offset $o = 1 + ((x-1) \bmod z)$ all'interno dello small block e accedere a $R[B_{i,j}, o]$, dove $B_{i,j}$ è la configurazione binaria dello small block.
4. Restituire $\text{RANK}_1(x) = r_i + r_{i,j} + R[B_{i,j}, o]$.

Ogni passo richiede tempo $\mathcal{O}(1)$ (accessi diretti a tabelle), per un totale di $\mathcal{O}(1)$.

Nota 15.3. Nella pratica, se z entra in una parola macchina, il conteggio degli 1 nello small block può essere implementato con l'istruzione hardware **popcount** (population count), evitando del tutto la tabella R .

Teorema 15.4. *L'occupazione di spazio della struttura dati per RANK è $o(m)$ bit in aggiunta all'array $B[1, m]$. L'operazione RANK richiede tempo costante nel caso peggiore e accede all'array B in sola lettura.*

15.1.3 Select Implementation

L'implementazione di SELECT segue un approccio simile a quello di RANK, ma con una differenza fondamentale: la suddivisione in blocchi non è per posizione ma per *numero di bit a 1*.

Si fissa $K = \log^2 m$ e si definiscono i *big blocks* come segmenti di B che contengono esattamente K bit a 1 ciascuno. La dimensione di un big block varia: sia Z_i la lunghezza del big block i -esimo.

- Se il big block è **sperso** ($Z_i > K^2$), le posizioni dei suoi K bit a 1 vengono memorizzate esplicitamente. Lo spazio è $\mathcal{O}(K \log m)$ bit per big block, e il numero totale di bit a 1 in tutti i big block sparsi è al più m/K , per uno spazio totale di $o(m)$ bit.
- Se il big block è **denso** ($Z_i \leq K^2$), si procede ricorsivamente suddividendolo in small blocks di $k = (\log \log m)^2$ bit a 1 ciascuno, e si applica una strategia analoga (esplicita per i sotto-blocchi sparsi, tabella di lookup per quelli densi). Lo spazio totale è ancora $o(m)$ bit.

Teorema 15.5. *L'operazione SELECT può essere supportata in tempo $\mathcal{O}(1)$ usando $o(m)$ bit di spazio aggiuntivo rispetto all'array $B[1, m]$.*

15.2 The FM-index

L'FM-index, introdotto da Ferragina e Manzini nel 2000, è il primo *indice full-text compresso*: combina compressione e indicizzazione in un'unica struttura dati. Può essere visto come una versione compressa del Suffix Array, oppure come una versione “ricercabile” di un file compresso con bzip2. L'FM-index supporta tre operazioni fondamentali:

- $\text{COUNT}(P)$: restituisce il numero di occorrenze del pattern $P[1, p]$ nella stringa indicizzata $S[1, n]$. Più precisamente, restituisce l'intervallo $[first, last]$ di righe di $\mathcal{M}'$ i cui suffissi sono prefissati da P , con $last - first + 1$ occorrenze.
- $\text{LOCATE}(P)$: restituisce la lista delle posizioni in S dove P occorre.
- $\text{EXTRACT}(i, j)$: restituisce la sottostringa $S[i, j]$ accedendo alla rappresentazione compressa.

15.2.1 Backward Search for Count

L'operazione $\text{COUNT}(P)$ è implementata tramite un algoritmo detto *backward search*, che processa il pattern P dall'ultimo simbolo al primo. L'algoritmo mantiene un intervallo $[first, last]$ di righe della matrice ordinata $\mathcal{M}'$ e utilizza l'array C e la funzione RANK sulla colonna L della BWT.

Algoritmo 15.1: Backward Search — $\text{COUNT}(P[1, p])$ in S

Input: Pattern $P[1, p]$, array C , colonna L con supporto RANK

Output: Intervallo $[first, last]$ tale che le righe $first, \dots, last$ di $\mathcal{M}'$ corrispondono a suffissi prefissati da P

```

 $i \leftarrow p; c \leftarrow P[p] \text{ first} \leftarrow C[c]; \text{last} \leftarrow C[c + 1] - 1 \text{ while } \text{first} \leq \text{last} \text{ e } i > 1$ 
do  $c \leftarrow P[i - 1] \text{ first} \leftarrow C[c] + \text{RANK}(c, \text{first} - 1) \text{ last} \leftarrow C[c] + \text{RANK}(c, \text{last}) - 1$ 
 $i \leftarrow i - 1$  return  $(\text{first}, \text{last})$ 

```

L'array $C[c]$ memorizza la posizione della prima occorrenza del simbolo c nella colonna F di $\mathcal{M}'$ (equivalentemente, $C[c] = 1 + \sum_{x < c} n_x$, dove n_x è il numero di occorrenze di x in S). L'array C ha dimensione $|\Sigma|$ ed è trascurabile.

Invariante. All' i -esima iterazione, first punta alla prima riga di $\mathcal{M}'$ prefissata dal suffisso $P[i, p]$, e last punta all'ultima. Inizialmente $i = p$ e l'intervallo copre tutte le righe che iniziano con $P[p]$. Ad ogni passo si estende il pattern di un simbolo a sinistra, restringendo l'intervallo.

Correttezza. L'aggiornamento usa la Proprietà 2 della BWT e la Definizione 14.6 dell'LF-mapping. $\text{RANK}(c, \text{first} - 1)$ conta le occorrenze di c in $L[1, \text{first} - 1]$, e $\text{RANK}(c, \text{last})$ conta quelle in $L[1, \text{last}]$. La differenza dà il numero di occorrenze di c nell'intervallo corrente di L , che tramite l'LF-mapping corrisponde esattamente alle righe di $\mathcal{M}'$ prefissate dal pattern esteso.

Esempio 15.6. Su $S = \text{abracadabra}$ cerchiamo $P = \text{ab}$. Si parte con $i = 2, c = \text{b}$: $\text{first} = C[\text{b}] = 7, \text{last} = C[c] - 1 = 8$ (le righe di $\mathcal{M}'$ che iniziano con b). Poi $i = 1, c = \text{a}$: si calcola $\text{RANK}(\text{a}, 6) = 1$ e $\text{RANK}(\text{a}, 8) = 3$, ottenendo $\text{first} = C[\text{a}] + 1 = 3, \text{last} = C[\text{a}] + 3 - 1 = 4$. L'intervallo $[3, 4]$ identifica 2 occorrenze di ab in S .

Teorema 15.7. *Data una stringa $S[1, n]$ su alfabeto Σ , esiste un indice compresso che supporta $\text{COUNT}(P[1, p])$ in tempo $\mathcal{O}(p \times t_{\text{rank}})$, dove t_{rank} è il costo di una singola operazione RANK sulla BWT di S . L'occupazione di spazio è limitata da $nH_k(S) + o(n \log |\Sigma|)$ bit, per qualsiasi $k = o(\log_{|\Sigma|} n)$.*

Per il Teorema 15.4 (adattato al caso di alfabeti generali), RANK può essere supportata in $\mathcal{O}(1)$ per $|\Sigma| = \mathcal{O}(\text{polylog}(n))$, ottenendo COUNT in tempo ottimo $\mathcal{O}(p)$.

15.2.2 Locate and Extract

Locate. Per ottenere le posizioni in S delle occorrenze trovate da COUNT , si utilizza un campionamento. Si fissa un parametro μ e si memorizzano esplicitamente le posizioni $\text{pos}(i)$ per le righe i di $\mathcal{M}'$ i cui suffissi iniziano in posizioni multiple di μ in S . Per una riga generica r , si applica iterativamente l'LF-mapping ($h \leftarrow LF^t(r)$) fino a trovare una riga campionata h , da cui $\text{pos}(r) = \text{pos}(h) + t$.

Teorema 15.8. *Data una stringa $S[1, n]$ su alfabeto Σ , esiste un indice compresso che supporta $\text{LOCATE}(P)$ in tempo $\mathcal{O}(\mu \cdot \text{occ})$ e $\mathcal{O}\left(\frac{n}{\mu} \log n\right)$ bit di spazio aggiuntivo, dove occ è il numero di occorrenze e μ è il passo di campionamento. Fissando $\mu = \log^{1+\varepsilon} n$, il tempo per occorrenza è polilogaritmico e lo spazio è sub-lineare.*

Extract. L'operazione $\text{EXTRACT}(i, j)$ ricostruisce $S[i, j]$ simulando la backward transform della BWT: si parte dalla riga di $\mathcal{M}'$ corrispondente al suffisso $S[j, n]$ (trovata tramite campionamento) e si applica l'LF-mapping ripetutamente per $j - i$ passi, leggendo un simbolo di S ad ogni passo. L'approccio è analogo all'inversione della BWT, ma con l'LF-mapping calcolato on-the-fly tramite RANK anziché memorizzato esplicitamente.

15.3 Summary and Perspectives

Le strutture dati compresse rappresentano un paradigma potente nell'algorithm engineering moderno:

Struttura	Spazio	Tempo query
RANK su $B[1, m]$	$m + o(m)$ bit	$\mathcal{O}(1)$
SELECT su $B[1, m]$	$m + o(m)$ bit	$\mathcal{O}(1)$
FM-index (COUNT)	$nH_k + o(n \log \Sigma)$ bit	$\mathcal{O}(p)$
FM-index (LOCATE)	$+\mathcal{O}\left(\frac{n}{\mu} \log n\right)$ bit	$\mathcal{O}(\mu \cdot \text{occ})$

L'FM-index combina compressione e indicizzazione full-text: occupa spazio proporzionale all'entropia di ordine k della stringa indicizzata, supporta il conteggio delle occorrenze in tempo ottimo $\mathcal{O}(p)$, e permette di localizzare le occorrenze e di estrarre sottostringhe in tempo efficiente. Come **bzip2**, incapsula una versione compressa del file originale (accessibile via **EXTRACT**), ma a differenza di un semplice compressore, consente la *ricerca* di pattern arbitrari senza decomprimere l'intero file.

15.4 Exercises

1. Dato l'array binario $B = (1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0)$, calcolare $\text{RANK}_1(10)$ e $\text{SELECT}_1(5)$. Mostrare i passi intermedi usando la decomposizione in big blocks ($Z = 4$) e small blocks ($z = 2$).
2. Dimostrare che lo spazio occupato dalla struttura dati succinta per RANK è $o(m)$ bit, utilizzando $Z = (\log m)^2$ e $z = \frac{1}{2} \log m$. Verificare che ciascuno dei tre livelli contribuisce $o(m)$ bit.

3. (Esame 2022) Descrivere l'algoritmo di backward search dell'FM-index. Spiegare il ruolo dell'array C , della colonna L e dell'operazione RANK nella ricerca. Eseguire la backward search per il pattern $P = \mathbf{bra}$ sulla stringa $S = \mathbf{abracadabra}$.
4. Spiegare come l'FM-index implementa l'operazione LOCATE utilizzando un campionamento del Suffix Array con passo μ . Discutere il trade-off spazio/tempo al variare di μ .
5. Confrontare l'FM-index con il Suffix Array non compresso in termini di spazio occupato, tempo di ricerca e funzionalità. In quali scenari l'FM-index è preferibile?
6. Spiegare il paradigma della *pointer-less programming* e come le operazioni RANK e SELECT possano sostituire i puntatori espliciti. Discutere vantaggi e svantaggi di questo approccio.

Chapter 16

Conclusions

Questo corso ha attraversato un ampio panorama di tecniche e risultati nell'Algorithm Engineering, partendo dai modelli di calcolo (RAM, two-level memory, cache-oblivious) fino alle strutture dati compresse e agli indici full-text. Il filo conduttore è stato il divario tra teoria e pratica: gli algoritmi “ottimi” nel modello RAM possono comportarsi in modo deludente su dati reali, e viceversa, soluzioni praticamente efficienti richiedono un'analisi teorica raffinata che tenga conto della gerarchia di memoria, della dimensione dell'alfabeto, della distribuzione dei dati e delle costanti nascoste nella notazione asintotica.

Richiamiamo i temi principali affrontati:

- **Modelli di memoria** (Capitoli 1–3): il modello two-level memory cattura il costo degli accessi a disco e giustifica l'uso di strutture I/O-efficienti come il B-tree e il multi-way MergeSort. Il modello cache-oblivious offre portabilità tra architetture diverse.
- **Ordinamento** (Capitoli 4–5): il MergeSort multi-way raggiunge il lower bound I/O di $\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$ trasferimenti. Il QuickSort, pur non ottimale in I/O nel caso peggiore, è spesso competitivo nella pratica.
- **Intersezione di insiemi e ordinamento di stringhe** (Capitoli 6–7): il galloping e il mutual partitioning sfruttano l'asimmetria tra insiemi. Il Multi-key QuickSort e il RadixSort sono le scelte naturali per stringhe.
- **Hashing e dizionari** (Capitolo 8): hashing universale, perfetto, cuckoo hashing e Bloom filter offrono soluzioni con diversi trade-off spazio/tempo/probabilità di errore.
- **Ricerca per prefisso e per sottostringa** (Capitoli 9–10): compacted trie, Patricia trie e String B-tree per la ricerca per prefisso; suffix array, LCP array, algoritmo di Kasai e DC3/Skew per la ricerca per sottostringa.

- **Compressione** (Capitoli 11–14): codifica di interi (gamma, delta, Rice, Elias–Fano), codifica statistica (Huffman, aritmetica, PPM), compressori a dizionario (LZ77, LZ78, LZW), e la trasformata di Burrows–Wheeler con la pipeline bzip2.
- **Strutture dati compresse** (Capitolo 15): Rank e Select su array binari, FM-index come paradigma di indicizzazione compressa e pointer-less programming.

L’Algorithm Engineering è una disciplina in continua evoluzione: nuove architetture hardware (GPU, SSD, memorie non volatili), nuovi paradigmi di calcolo (streaming, distributed, quantum-inspired) e nuove applicazioni (bioinformatica, web search, machine learning su dati strutturati) pongono costantemente sfide algoritmiche che richiedono la combinazione di rigore teorico e sensibilità ingegneristica che questo corso ha cercato di trasmettere.

“Beware of bugs in the above code; I have only proved it correct, not tried it.”

— Donald E. Knuth